Prompts

These are all the prompts of the main Claude Code session that built the project (development folder cs331-lb), verbatim and in order, each followed by the AI's written replies. The session ran in a terminal on one laptop, not on the claude.ai website. Tool calls and their raw output (commands run, file edits) are not shown.

38 prompts

Prompt 1

hey so me and my team (4 of us) built a project for our computer networks course called mini-katran, basically a toy version of facebooks katran, xdp load balancer with a hash ring, conntrack, dnat and checksum stuff, plus a tcp proxy and an af_packet forwarder as baselines. looking back i think we kinda did it backwards, we just copied katran without ever learning about load balancers in general or figuring out what problem we were actually solving, so we could build it but couldnt really explain why it was designed the way it was. i want to get better at that part. so we want to redo it properly, study lbs in general first, pick a specific well defined problem and then build something that solves it well. where do we even start, like what kinds of load balancers exist and what problem does each one solve

Claude Code

You're right that this is the gap. A load balancer is a set of answers to specific problems. If you know which problem each part answers, the design explains itself. So start from the problems.

1. The core problem, split into sub-problems

"One service, many servers, and clients shouldn't notice" breaks into separate problems. Each kind of LB exists because it handles some of them well:

1. **Distribution:** spread work so no server is overloaded.
2. **Failure handling:** stop sending traffic to dead or sick servers, quickly.
3. **Consistency (affinity):** packets of one connection, or requests of one session, must keep reaching the same server, even while the server set changes.
4. **Scaling the LB itself:** one LB box is a bottleneck and a single point of failure, so how do you run many of them?
5. **Overhead:** the LB adds latency, CPU and memory. How little can it add?
6. **Smart routing:** route by URL, user or region, retry failed requests, and so on.

2. The main kinds of load balancer

Kind	How it works	Problem it solves	What it costs / can't do
DNS / GSLB	Return different IPs per client or region	Coarse global distribution, steering users to a nearby datacenter	TTL caching makes failover slow; no per-connection control
Anycast + ECMP (routers)	Many sites or boxes announce the same IP; routers hash the 5-tuple across next hops	Scaling the LB tier horizontally (#4) with no extra box	Routers know nothing about health or load. **If the set of next hops changes, many flows get rehashed to a different box**
L4 LB (LVS/IPVS, Maglev, Katran,	Forwards packets by 5-tuple and never terminates TCP	High-throughput distribution with low overhead (#1, #5);	Blind to the application; balances *connections*, not requests

Kind	How it works	Problem it solves	What it costs / can't do
Unimog, AWS NLB)		can preserve connections (#3)	
L7 LB / reverse proxy (HAProxy, nginx, Envoy)	Terminates TCP/TLS, parses HTTP, opens its own connection to the backend	Per-request balancing, retries, routing on content (#6), TLS offload	Two connections per flow, much higher CPU and memory per byte
Client-side / service mesh (gRPC LB, sidecars)	The client or a local proxy picks the backend	Traffic between internal services ("east-west") with no central hop	Every client needs fresh membership and health information

Your TCP proxy baseline is basically a minimal L7-style terminating proxy, and the AF_PACKET forwarder is a naive L4 forwarder. Those are good baselines. They just weren't tied to a question.

Inside L4: the forwarding mode is a design decision

This is probably the most important "why" for your project:

- **NAT (DNAT + SNAT):** the LB rewrites the destination IP, and replies come back through the LB.
  - The LB needs conntrack for the reverse translation.
  - The LB carries both directions of traffic.
- **DSR (Direct Server Return):** the LB forwards to the backend, and the backend replies **straight to the client**.
  - The rewrite can be a MAC rewrite (LVS-DR) or IP-in-IP/GUE encapsulation (Katran, Maglev).
  - Why it matters: responses are usually much bigger than requests (think downloads, video). With DSR the LB only handles the small inbound side.

Real Katran uses IPIP encapsulation plus DSR, not DNAT. If your version used DNAT, you built a different design: all return traffic has to come back through the LB, which is exactly what Katran is built to avoid. That's not a criticism. It's a concrete example of "we copied it without knowing why."

Algorithms are a separate axis

The algorithm is independent of the LB's layer, and each one solves a different problem:

- **Round robin / weighted:** simple distribution when requests and servers are roughly uniform.
- **Least connections / least load:** handles uneven request cost.
- **Power of two choices:** pick 2 servers at random and take the less loaded one. You get nearly the benefit of least-load without global state, which matters when there are many LBs that each have stale information.
- **Consistent hashing** (ring, rendezvous/HRW, Maglev, jump hash): the same key maps to the same server, and changing the server set moves only about 1/N of the keys. This solves consistency (#3) **without shared state**.
- **Consistent hashing with bounded loads:** keeps the consistency but caps how overloaded any one server can get.

3. Katran, explained from the problems

Facebook's edge is a stack of layers:

1. Anycast VIP
2. Routers spreading traffic with ECMP
3. Many Katran L4 instances
4. L7 proxies, reached through IPIP encapsulation, which reply to clients directly (DSR)

Each of Katran's design choices follows from a problem:

- **Why many L4 instances?** One box can't take the traffic (#4), so ECMP spreads packets across many LBs.
- **Why consistent hashing (Maglev)?** When an LB instance is added or dies, ECMP reshuffles flows and some connections land on a *different* LB mid-stream. That LB has no state for the flow. With consistent hashing it still picks the same backend, so the connection survives.
- **Why conntrack as well?** When the *backend* set changes, consistent hashing moves some keys. The per-LB flow table pins existing connections on the LB that already saw them. So the two mechanisms cover each other: hashing covers LB changes, the table covers backend changes.
- **Why DSR + IPIP?** Asymmetric traffic (#5), and encapsulation lets the L7 proxies live anywhere in the datacenter, not just on the same L2 segment.
- **Why XDP?** Per-packet cost on ordinary servers. XDP runs before the kernel allocates its packet structures, but unlike DPDK it doesn't take the NIC away from the rest of the host.

If your team can give that explanation from memory, you understand Katran.

4. How to pick a well-defined problem

Write a one-paragraph problem statement **before** any code:

> *Workload* (what traffic) + *events* (what changes or fails) + *metric* (what you measure) + *baseline* (what you compare against) + *claim* (what you expect to show).

Then build the **experiment setup first**: network namespaces or VMs, a traffic generator, and a way to measure the metric. After that, build the simplest LB, measure it, and improve it.

Some candidate problems that fit a course project and can run on a laptop:

1. **Keeping connections alive while backends change.** Add or remove backends while long-lived connections are running. Compare mod-N hashing, a hash ring, Maglev and rendezvous hashing, each with and without a flow table.
   - Metrics: % of connections broken, load imbalance, lookup cost.
2. **Scaling the LB tier itself.** Run several LB instances behind simulated ECMP and add or remove LBs. Compare stateful designs with stateless consistent-hash designs (and a Beamer-style design if you want to go further).
   - Metric: connections broken during LB scale-out or failure.
   - This is the problem Katran is really built for. Done properly, it turns your old project into something you can fully explain.
3. **NAT vs DSR under asymmetric traffic.** Small requests, large responses.
   - Metrics: LB CPU and throughput as response size grows.
   - It's a clean and very visual result.
4. **Tail latency with uneven backends.** Make one backend slow. Compare round robin, least-connections, power of two choices and bounded-load hashing.
   - Metric: p99 latency.
5. **Connection-level vs request-level balancing.** Run HTTP/2 or gRPC through an L4 LB and an L7 LB.
   - Show the imbalance with long-lived multiplexed connections and when L7 is worth its cost.

My recommendation: do **#2, with #1 as the first step toward it**. It reuses your eBPF/XDP skills, it is the real problem Katran solves, and the result is easy to measure (broken connections when X changes).

5. Reading, in order

1. **Matt Klein, "Introduction to modern network load balancing and proxying"** (2017 blog post). The best overview of everything above. Start here.
2. **Maglev** (Google, NSDI 2016). The reference L4 design and the source of Maglev hashing.
3. **Katran open-source announcement** (Facebook engineering blog, 2018). Read it now with section 3 in mind.
4. **Ananta** (Microsoft, SIGCOMM 2013) and **Beamer** (NSDI 2018). Beamer is about stateless LBs and keeping connections alive while the LB tier changes, so it's directly relevant to #2.
5. **GitHub GLB** and **Cloudflare Unimog** blog posts. Real-world designs and the trade-offs they made.
6. **Mirrokni et al., "Consistent Hashing with Bounded Loads"**, and **Mitzenmacher on the power of two choices**. For the algorithms side.
7. **LVS docs** on NAT/DR/TUN modes. The clearest explanation of forwarding modes.

A good first exercise: have each of the four of you read one of items 2–5. Each person then gives a 10-minute talk on what problem that system solves and which design choices follow from it. After that, choosing your problem will be much easier.

I can turn this into a shared doc for your team, with a problem-statement template and reading assignments, if that would help.

Prompt 2

thanks, this is a really clear breakdown. no need for a doc yet, well split the reading between us like you suggested. before we pick between those options though we want to approach it from what real companies actually need. like what kind of load balancing would a company like stripe or amazon need, would it be the same problem as facebook's or something different

Claude Code

Mostly not the same problem. Facebook's edge problem is very specific: a huge amount of internet traffic coming into one company, measured in packets per second. Most companies have a different problem.

One caveat: I'm going on what these companies have published (blog posts, talks, the AWS Builders' Library), not on knowledge of their internals.

Questions that tell the problems apart

Four questions separate one company's load-balancing problem from another's:

1. **What is the bottleneck?** Raw packets per second, or the cost of each request?
2. **How much does a failure cost?** A dropped video stream versus a duplicated payment.
3. **Who shares the system?** One app, or many customers who must not hurt each other?
4. **What is the traffic like?** Short requests or long-lived connections, and uniform or highly variable cost per request?

Facebook / Meta edge

- **Bottleneck:** packets per second and the cost of the LB fleet.
- **Cost of failure:** a broken connection is annoying, and the client reconnects.
- **Main problem:** forwarding enormous volume cheaply, and keeping connections alive while LBs and backends change.
- **Result:** Katran. L4, XDP, consistent hashing, DSR.

Stripe (payments API)

Traffic volume is small next to Meta's, so an L4 forwarder at line rate isn't the hard part. A cloud LB or Envoy handles it fine. Stripe has written about moving to Envoy. The hard problems are higher up:

- **Overload protection.** Requests vary a lot in cost; a list query can cost far more than a status check. When backends saturate, you want to drop the *least important* traffic and keep charges flowing.
  - Stripe's blog post "Scaling your API with rate limiters" describes layered rate limiters and load shedders, with critical traffic prioritized.
- **Fairness between tenants.** One merchant's burst, say a flash sale or a buggy retry loop, must not raise everyone else's latency.
- **Retries that are correct.** An LB or client that retries a "charge card" request must not charge twice. That's why idempotency keys exist. LB retry policy and application correctness are tied together.
- **Tail latency across many internal services.** One API call fans out to many internal services, so per-request balancing (L7, least-loaded, power of two choices) matters more than per-connection hashing.

Amazon: two different problems

Amazon.com, or any large service-oriented company

- The main concern is *blast radius*: one bad customer, request or deploy must not take everything down. Public techniques:
  - **Cell-based architecture:** independent copies of the whole stack, each serving a slice of customers.
  - **Shuffle sharding:** each customer gets a random small subset of servers, so two customers rarely share their whole subset. A "poison" customer can only hurt the few others who overlap with it.
  - **Load shedding:** drop excess requests early rather than letting every request slow down.
  - **Retries with backoff and jitter:** so retries don't turn a small failure into a retry storm.
- The AWS Builders' Library covers all of these with short, readable articles.

AWS as an LB vendor (ELB/ALB/NLB)

- Here the LB *is* the product, and it's shared by millions of tenants. Problems:
  - Isolating noisy neighbours.
  - Scaling each tenant's LB up and down automatically.
  - Keeping per-connection state that survives instance failure. AWS has talked publicly about an internal system called Hyperplane that underlies NLB and NAT Gateway.
- This is closer to Facebook's problem than Stripe's is, but with multi-tenancy added.

Problems almost every company has

These are less glamorous but very real:

- **Draining connections during deploys.** Remove a server from rotation without breaking its in-flight requests or long-lived connections. Every rolling deploy hits this.
- **Long-lived connections** (websockets, gRPC, game servers). When you add servers, existing connections never move, so new servers sit idle and old ones stay hot.
- **Health checking that isn't wrong.** A server can pass health checks while failing real requests, or the opposite. A flapping health check causes more damage than a dead server.

What this means for your project

You can now frame the project around a realistic requirement instead of "build an LB":

Company-like scenario	Requirement	Layer	Fits your XDP skills?
Meta edge	Keep connections alive while LBs and backends change, at high packets/sec	L4	Yes. Options #1/#2 from before
Multi-tenant platform (AWS, or Stripe's merchants)	One tenant's flood must not hurt other tenants' latency	L4 or L7	**Yes.** Shuffle sharding plus per-tenant rate limiting can be done in XDP
Payments-style API	Under overload, shed low-priority work and protect critical requests' p99	L7	Partly. Needs request awareness, so a user-space proxy
Any company doing rolling deploys	Zero broken connections while draining or replacing backends	L4 or L7	Yes. Close to option #1

The multi-tenant row is the most interesting new option. Here's the problem statement written out:

> *A shared L4 LB serves N tenants. One tenant suddenly sends 10× its normal traffic (or poison requests that crash backends). Show that with shuffle sharding plus per-tenant rate limiting in XDP, other tenants' p99 latency and error rate stay flat, compared with a plain consistent-hash LB where they degrade.*

It is grounded in a real need (AWS, Stripe), it has a clear metric and baseline, it uses what you already know, and it's less of a copy of one existing system.

Extra reading for this angle

- **AWS Builders' Library:** "Workload isolation using shuffle-sharding", "Using load shedding to avoid overload", "Timeouts, retries, and backoff with jitter".
- **Stripe blog:** "Scaling your API with rate limiters".
- **Google SRE book:** the chapters on load balancing in the datacenter and on handling overload. They're free online, and they're the best general treatment of the "what does a real company need" question.

One way to settle the decision as a team: each of you picks one company-like scenario and writes the one-paragraph problem statement for it. The one where you can most clearly state the metric and baseline is usually the right one to pick.

Prompt 3

you said stripe's traffic volume is small next to meta's and the hard parts are overload and tail latency. but if theyre serving millions of customers a day wouldnt throughput still be the bigger concern? why is latency the thing that matters more there

Claude Code

Throughput does matter for Stripe. But "millions per day" is smaller than it sounds once you convert it to a rate. And the throughput that's hard for them is in the backends, not in the load balancer.

1. Per day is not per second

The numbers below are made up to show scale; they aren't Stripe's real figures.

- 100 million requests per day ÷ 86,400 seconds ≈ **1,200 requests/second** on average.
- Assume peaks are 10–20× the average (Black Friday, for example). That gives roughly **10–25k requests/second**.

A single well-tuned Envoy or nginx box can handle tens of thousands of HTTP requests per second. An XDP forwarder can handle millions of packets per second per machine. So even at the peak, the forwarding part fits on a handful of machines.

Meta's edge is several orders of magnitude bigger, and its LB tier is a large fleet in its own right. Making each packet 2× cheaper there saves real hardware. At Stripe, the same improvement saves almost nothing.

2. The LB's cost is tiny compared with the request's cost

Look at where the time goes in one payment request (rough orders of magnitude):

Step	Approximate cost
L4/L7 load balancing decision and forwarding	microseconds
Auth, validation, fraud checks across internal services	milliseconds to tens of ms
Database writes, durable logging	milliseconds to tens of ms
Call out to a card network or bank	tens to hundreds of ms

Even if the LB were 10× faster, the request would barely change. The throughput that limits Stripe is how many requests the **backends** can do: databases, fraud models, external partners. Buying more of that capacity is mostly a planning and money problem, not an LB design problem.

What the LB *can* change is **which** backend a request goes to and **whether** it gets admitted at all. That affects latency and survival under load far more than forwarding speed does.

3. Throughput limits show up as latency first

This is the key connection. As a server gets close to full capacity, the queue in front of it grows nonlinearly. In the simplest queueing model (M/M/1), average waiting time grows with $\rho/(1-\rho)$, where ρ is how busy the server is:

Server busy	Relative waiting time
50%	1×
80%	4×
90%	9×
95%	19×

So a system doesn't hit a hard throughput wall. It gets slow first, then clients time out and retry, and the retries add *more* load. That is how a traffic burst turns into an outage.

So "can we handle the throughput?" turns into two questions in practice:

- Do we keep latency bounded near capacity?
- Do we shed the right work when we go over?

Both are overload and latency problems.

4. Fan-out makes the tail dominate

One API call can touch many internal services. Suppose each service is slow 1% of the time (its p99), and one call touches 20 of them. Then:

> P(at least one slow) = $1 - 0.99^{20} \approx$ **18%**

So a *rare* slowness in each service becomes a *common* slowness for users. This is the argument of Dean and Barroso's **"The Tail at Scale"** (CACM 2013), and it's worth reading. At this point the LB's choices matter a lot:

- Avoid a backend that is currently slow (least-loaded, power of two choices).
- Send a backup request if the first one is slow ("hedged requests").
- Don't retry so aggressively that you overload the system.

5. Why a slow request is costly for a payments company

- The customer is in the middle of a checkout. A slow payment means an abandoned purchase, which costs the merchant money, which makes it Stripe's problem.
- A timeout is *ambiguous*: did the charge happen or not? The client retries, and now correctness (no double charges) depends on idempotency. Tail latency directly creates correctness risk.

The general lesson

The question to ask is "where is the scarce resource, and what does the LB control?", not "is the traffic big?":

- **Meta edge:** the LB fleet itself is the scarce resource. The LB should do the **cheapest possible** work per packet: XDP, hashing, no request parsing.
- **Stripe-like API:** backend capacity and tail latency are scarce. The LB should spend **more** CPU per request to make **better** decisions: L7, load-aware choices, admission control, priorities.

Both are "load balancers", but they are optimized in opposite directions, and that's why they look so different.

For your project, this gives you a sharp test for a problem statement. If your metric is throughput, say why the LB is the bottleneck. If your metric is p99 latency, say what decision the LB makes that changes it.

Prompt 4

that makes sense, especially the part about the lb deciding where load goes rather than just forwarding fast. the shuffle sharding idea from before is interesting too, but we want to look at a few more real cases before deciding. are there other real world stories like this from cloudflare, github, google etc that could make a good project, ideally something we can actually build and test ourselves rather than just read about

Claude Code

Here are seven real cases that each became a well-defined problem, and each can be rebuilt as a small version and measured. I checked that the main sources exist; links are at the bottom.

Group 1: keeping connections alive while things change (L4)

1. GitHub GLB: removing a server without breaking its connections

- **The story:** GitHub's old LB tier was a few big boxes scaled vertically, and it hit its limit. They wanted a tier of many LBs that could scale horizontally, and the ability to take a proxy out of rotation (for maintenance or deploys) without killing the long-running git/SSH connections on it.
- **What they built:**
  - The "director" LB tier keeps **no per-connection state**.
  - Rendezvous hashing gives each flow a **primary and a secondary** proxy.
  - The director sends each packet to the primary, encapsulated.
  - If the primary has no socket for that flow and the packet isn't a new SYN, it forwards the packet to the secondary. This works because the TCP state already stored in the proxies' kernels is the source of truth, so the LB doesn't need its own state.
- **What you'd build:** an XDP director with a primary/secondary table, plus a small redirect hook on each backend (tc-BPF or XDP) that checks "do I have a socket for this flow?"
- **Test:** keep 1,000 long-lived connections open, drain one backend, and add another.
  - Metric: connections broken.
  - Baselines: plain hashing, and hashing plus LB-side conntrack.

2. Cloudflare Unimog: spreading load evenly across servers that aren't equal

- **The story:**
  - At Cloudflare's edge, every server runs every service, and routers spread traffic across servers with ECMP.
  - But servers differ: different hardware generations, other background work. Equal traffic therefore gave *unequal* load.
  - They needed to shift traffic based on measured load, without breaking connections when they shift it.
- **What they built:**
  - An XDP L4 LB that runs on every server, with a table of buckets.
  - A controller moves buckets away from overloaded servers.
  - Each bucket remembers its previous owner. If the new server doesn't recognise a connection, it forwards the packet to the previous one. This is the same idea as GLB's secondary, which the Unimog post credits.
- **What you'd build:** the bucket table in XDP, a user-space controller that reads each backend's CPU, and the forward-to-previous-owner path.
- **Test:** give backends unequal capacity (cgroup CPU limits, or `stress-ng` on some), then ramp up load.
  - Metrics: spread of CPU across backends, p99 latency, connections broken during rebalancing.
  - Baseline: static equal weights.

Group 2: deciding where load goes

3. Cloudflare: "Why does one NGINX worker take all the load?"

- **The story:** Load balancing *inside one machine*. Several NGINX worker processes share a listening socket. Because of how Linux wakes processes up with epoll, most new connections go to the same busiest worker. Switching to `SO_REUSEPORT` balances the workers better, but has its own tail-latency problems.
- **What you'd build:**
  - A multi-worker server and a load generator.
  - Then your own fix: an eBPF `sk_reuseport` program that chooses which worker's socket gets each new connection, for example by queue length.
- **Test:**
  - Metrics: CPU per worker, p99 latency.
  - Compare a shared accept queue, plain `SO_REUSEPORT`, and your BPF policy.
- **Why it's good:** smallest setup of all seven (one machine), but it goes deep into the kernel. A good fallback if time gets short.

4. Vimeo + Google: consistent hashing with bounded loads

- **The story:** Vimeo routed video requests with consistent hashing so each cache server kept the same content (good cache hit rate). But popular videos overloaded some servers, so they gave up and switched to least-connections, which ruined cache locality. They then implemented Google's "consistent hashing with bounded loads" in HAProxy: keep the hash affinity, but cap any server at a set factor above average load.
- **What you'd build:** a few caching backends behind your LB. Use a skewed key popularity (Zipf distribution), because that is what real traffic looks like.
- **Test:**
  - Metrics: cache hit rate *and* max load per server.
  - Compare plain consistent hashing, least-connections, and bounded loads with different factors.
- **Why it's good:** a clean trade-off curve between affinity and balance. It makes a great results chart.

5. Twitter deterministic aperture / Google subsetting

- **The story:** With client-side load balancing, thousands of clients each connecting to thousands of backends means millions of connections. So each client connects to only a **subset** of backends. Choosing subsets randomly gives uneven load. Twitter's "deterministic aperture" picks subsets deterministically and combines that with power of two choices. Google has written about the same problem: deterministic subsetting in the SRE book, and a later CACM article on reducing connection churn.
- **What you'd build:** mostly a simulator, plus a small real version (a gRPC or HTTP client library).
- **Test:**
  - Metrics: load spread across backends, connections per backend, how many connections get reshuffled when a backend is added.
- **Note:** this is more about algorithms than packets, and less of a networking-course fit.

6. Google Prequal (YouTube): don't balance CPU, balance what predicts latency

- **The story:**
  - The usual approach is to balance CPU utilization, for example weighted round robin driven by CPU numbers.
  - Google found that this leaves bad tail latency.
  - Prequal sends cheap probes to backends, asking each for its requests-in-flight and estimated latency, and picks based on those answers.
  - On YouTube this cut tail latency and errors and let them run machines at higher utilization (NSDI 2024).
- **What you'd build:** an L7 proxy with pluggable policies: round robin, least-connections, power of two choices on requests-in-flight, and a Prequal-style prober.
- **Test:** backends with noisy, uneven slowness (inject delays with `tc netem`, or add CPU contention).
  - Metric: p99 latency at 50%, 70% and 90% utilization.
- **Why it's good:** the clearest real-world version of the "the LB decides where load goes" idea, and it's recent work.

7. Netflix adaptive concurrency limits: overload

- **The story:** Fixed rate limits are always wrong, either too low (wasted capacity) or too high (collapse). Netflix borrowed from TCP congestion control: watch latency, and when it rises, shrink the number of requests allowed in flight. That gives a limit that adjusts itself.
- **What you'd build:** admission control in a proxy that uses a Vegas- or gradient-style limit.
- **Test:** ramp the load to 2× capacity.
  - Metrics: goodput (successful requests per second) and latency.
  - Compare no limit, a fixed limit, and the adaptive limit.
- **Why it's good:** the cleanest demo of the queueing curve from earlier, and it connects naturally to TCP congestion control in your networks course.

Comparison

Case	Layer	Uses your XDP/eBPF skills	Setup size	Main metric
1. GitHub GLB	L4	Yes, heavily	Medium	Connections broken
2. Cloudflare Unimog	L4	Yes, heavily	Medium–high	Load spread + connections broken

Case	Layer	Uses your XDP/eBPF skills	Setup size	Main metric
3. NGINX worker imbalance	Within one host	Yes (`sk_reuseport`)	Small	Per-worker CPU, p99
4. Bounded-load hashing	L4 or L7	Optional	Small–medium	Hit rate vs max load
5. Aperture / subsetting	Client-side	No	Small (mostly simulation)	Spread, connection count
6. Prequal	L7	No	Medium	p99 at high utilization
7. Adaptive concurrency	L7	No	Small–medium	Goodput under overload

Notice the pattern: cases 1–2 (and Katran) are about keeping connections alive while things change, and cases 3–7 are about deciding well. Unimog does both.

Practical setup tips for any of these

- **Testbed:** Linux network namespaces with veth pairs work for everything. You don't need VMs.
- **Latency measurement:** use `wrk2` or a load generator that sends at a fixed rate. Ordinary `wrk` hides latency under overload, because it stops sending while waiting for slow responses (this is called coordinated omission).
- **Making backends slow or unequal:** `tc netem` for delay, cgroup CPU limits or `stress-ng` for capacity.

My suggestion

If you want to stay with XDP:

1. **First milestone, GLB-style:** build the primary/secondary tables with drain-without-breaking. Measuring "connections broken" is a solid result on its own.
2. **Main project, Unimog-style:** add a controller that rebalances buckets by measured load.

That combines both families, it's a real story with a clear "why", and it reuses what you built for mini-katran in a design you can actually defend.

If you'd rather try the L7 side, Prequal (case 6) is the most interesting choice and teaches the most about the tail-latency ideas.

Sources:

- GLB: GitHub's open source load balancer (2018)
- Introducing the GitHub Load Balancer (2016)
- Unimog – Cloudflare's edge load balancer (2020)
- Why does one NGINX worker take all the load? (Cloudflare, 2017)
- Vimeo: Improving load balancing with a new consistent-hashing algorithm
- Google Research: Consistent Hashing with Bounded Loads
- Twitter: Deterministic Aperture (2019)
- Finagle Aperture docs
- Reinventing Backend Subsetting at Google (CACM author listing)
- Prequal, NSDI 2024
- Netflix: Performance Under Load (2018)

Prompt 5

unimog and glb both sound really good to us, and prequal is interesting too. before going further, for context on hardware: we have 4 laptops and only one of them supports native xdp. we might be able to get raspberry pis later but that could be difficult. does the unimog / glb style idea still work with that setup, and which of the others would

Claude Code

Yes, both still work, and the hardware limit matters less than it sounds. None of the problems we've been discussing are about raw throughput. Their metrics are connections broken, load spread and p99 latency, and those don't need line-rate NICs.

Why native XDP isn't a blocker

- **Generic XDP (SKB mode)** runs the same program on any interface. It's slower because it runs after the kernel has already allocated its packet structures, but the behaviour is identical, so correctness experiments give the same results.
- **veth pairs support native XDP.** If you build your topology from network namespaces connected by veths on one laptop, you get native XDP inside it on any modern kernel.
  - Gotcha: for `XDP_REDIRECT` into a veth, attach an `XDP_PASS` program on the peer end, or the redirected frames get dropped.
- **tc-BPF** attaches to any interface, and it's the natural place for GLB's backend-side redirect hook anyway.
- **Linux VMs:** `virtio_net` supports native XDP, so a teammate without a Linux install can develop in a VM.

To check what a given interface supports, run `ethtool -i <iface>` to see its driver, then try `ip link set dev <iface> xdpdrv obj prog.o sec xdp`. If the driver lacks support, the command fails.

What the setup looks like

One laptop, a handful of namespaces, all scripted:

```
[client ns] ──veth── [router ns] ──veth──┬── [server1 ns: XDP LB + backend]
 load gen              Linux ECMP        ├── [server2 ns: XDP LB + backend]
                       (multipath route) ├── [server3 ns: ...]
                                         └── (bridge between servers for
                                              encapsulated forwarding)
```

- **Simulating ECMP is built into Linux.** Use a multipath route in the router namespace, and set `net.ipv4.fib_multipath_hash_policy=1` so it hashes on the 5-tuple. Adding or removing a next hop then rehashes flows, which is exactly the event GLB and Unimog are designed to survive.
- **Free extra baseline:** Linux "resilient" nexthop groups (kernel 5.13+, `ip nexthop add ... type resilient`) are the router's own answer to the same rehashing problem. You can compare your LB-tier fix against a router-tier fix.
- **Unimog maps naturally onto this.** Unimog runs on every server, so each server namespace runs your XDP program plus a backend service.
- **Unequal servers become controllable.** Use cgroup `cpu.max` to give servers different capacity, and pin each namespace's processes to separate cores. That's more controllable and repeatable than real unequal hardware, which is a plus for experiments.
- **Make the whole topology one script** (`setup.sh` / `teardown.sh`), so all four of you run exactly the same testbed.

Limits of a single laptop, and how to handle them

Issue	What to do
Load generator, LB and backends compete for the same CPU	Pin each to its own cores; report core assignments with the results
Absolute numbers (Mpps, µs) don't match real hardware	Report *relative* comparisons (baseline vs your design), not absolutes
Generic or veth XDP throughput isn't representative	Don't make throughput a headline claim; optionally add a short performance section using the native-XDP laptop
Measurement noise	Several runs per configuration; report medians and spread

Multi-laptop and Raspberry Pi

- **You don't need Pis.** They would mostly add problems: slow CPUs, 1 Gbps NICs, and uncertain XDP driver support.
- **Over Wi-Fi, avoid anything L4.** Wi-Fi client mode doesn't handle the L2 tricks well (multiple MAC addresses, bridging), and Wi-Fi latency jitter swamps p99 measurements.
- **If you want a physical demo,** a cheap Ethernet switch plus wired links works. Use the native-XDP laptop as the LB and the others as clients and backends. USB Ethernet adapters usually lack native XDP, but generic mode works on them. Treat this as a final demo, not the main testbed.

Which projects work on your hardware

Project	Works on your setup?	Notes
GLB-style (primary/secondary, drain without breaking)	Yes, fully	Namespaces + veth + tc-BPF; every teammate can develop locally
Unimog-style (load-aware rebalancing + forward to previous owner)	Yes, fully	Unequal capacity via cgroups is actually easier to control than real hardware
Prequal	Yes, easiest	All user space, no XDP needed; can even run across laptops if wired
NGINX / `sk_reuseport` imbalance	Yes	Needs a laptop with enough cores (8+) to show the effect clearly
Bounded-load hashing	Yes	Hardware doesn't matter
Adaptive concurrency limits	Yes	Hardware doesn't matter
Aperture / subsetting	Yes	Mostly simulation
Anything with "faster than X" as the main claim	Only on the native-XDP laptop with a real NIC	Not one of the options we've been discussing

Practical takeaway

The hardware limit pushes you toward problems about correctness and decision quality, not raw speed. That's the right direction anyway, and it fits GLB, Unimog and Prequal well.

One working pattern for a team of four:

- Everyone develops on their own laptop inside namespaces, using generic or veth XDP.
- Results come from one agreed machine running the shared setup script.
- The native-XDP laptop is reserved for an optional performance section at the end.

Before you commit, check each laptop's kernel version (`uname -r`; 5.15+ is comfortable) and whether BTF is available (`ls /sys/kernel/btf/vmlinux`). Those matter more for eBPF development than which NIC you

have.

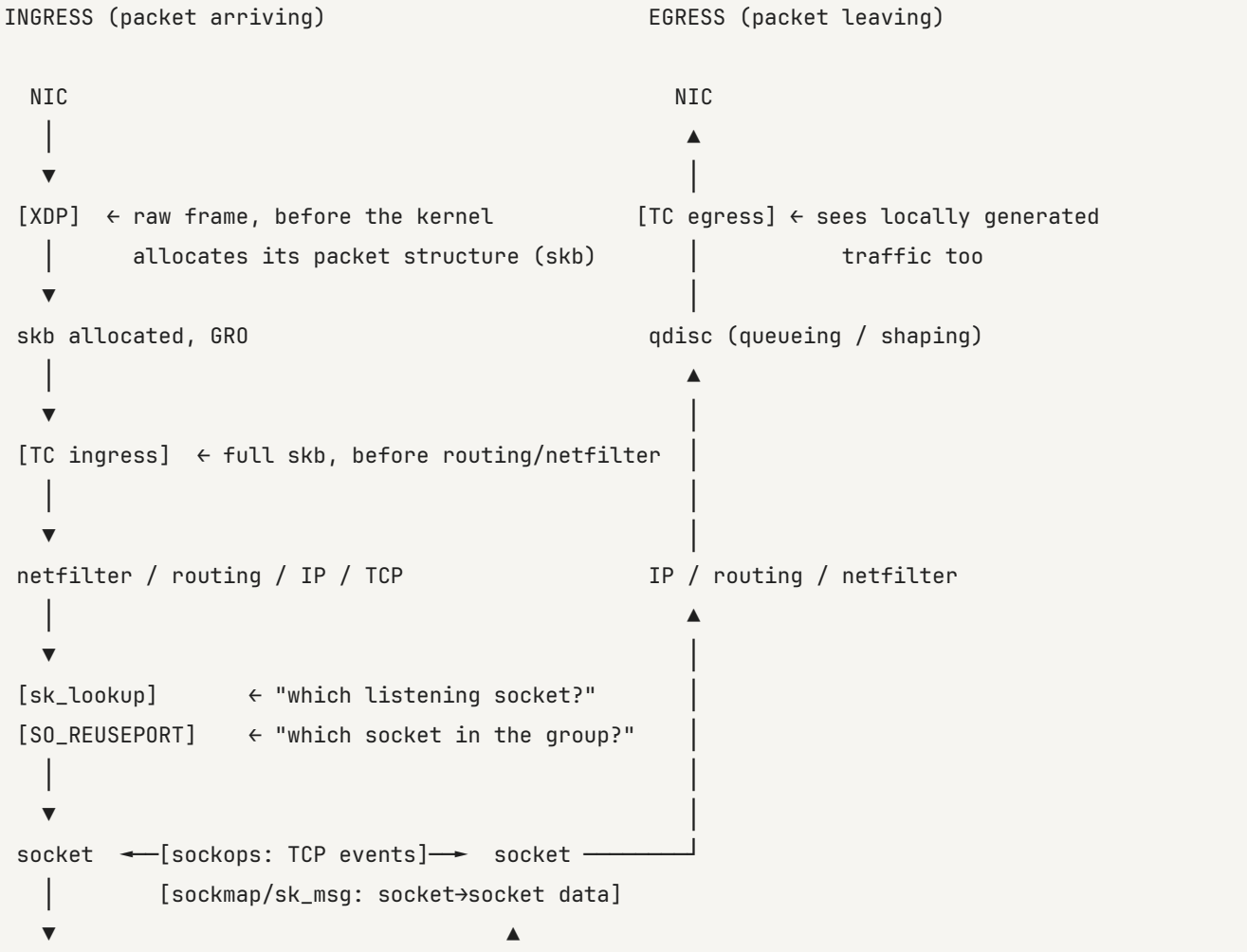
Prompt 6

good to know generic xdp and tc work anywhere. you mentioned tc-bpf for the backend side hook and sk_reuseport earlier. can you explain the main ebpf hook points besides xdp more broadly? when would you use tc vs xdp vs the socket level hooks, and what can each one do that the others cant

Claude Code

The way to think about it: each hook sits at a different point on the path from the NIC to the application. Earlier hooks are cheaper per packet but know less. Later hooks know more (which socket, which process, which connection) but run after more work has already been done. The design rule is **use the earliest hook that has the information your decision needs.**

Where the hooks sit



The main hooks

XDP: per-packet, ingress, earliest

- **Runs:** in the driver's receive path (or in generic mode right after), on every received frame.
- **Can do:** drop, bounce back out the same interface (`XDP_TX`), redirect to another interface, a CPU or an AF_XDP socket, rewrite any bytes, and add or remove headers (encap/decap).

- **Unique strength:** cheapest per packet, because there's no skb allocation yet. That's why DDoS filtering and L4 LBs (Katran, Unimog) live here.
- **Can't:**
  - See outgoing traffic, including anything generated locally.
  - Use the skb's extra information (mark, priority, timestamps for pacing).
  - Interact with queueing.
  - Socket lookup does work in XDP (`bpf_sk_lookup_tcp` / `bpf_skc_lookup_tcp`), but you have to parse everything yourself.

TC (cls_bpf / tcx): per-packet, both directions, with an skb

- **Runs:** after the skb exists, on ingress before routing and netfilter, and on **egress**. Newer kernels have `tcx`, a cleaner way to attach.
- **Unique strengths:**
  - **Egress.** You can act on replies and on traffic the host itself generates: SNAT, encapsulating outgoing packets, marking traffic.
  - **Rate limiting by pacing.** You set each packet's departure timestamp (`skb→tstamp`) and the `fq` qdisc sends it at that time. Cilium's bandwidth manager works this way. This would be the egress side of per-tenant rate limiting.
  - **Integration with the kernel stack:** tunnel metadata helpers, `bpf_redirect_peer` (jump straight into a container namespace through its veth), `bpf_redirect_neigh` (let the kernel fill in the next-hop MAC).
- **Cost:** more expensive than XDP because the skb is already allocated. And after GRO, one skb can be a merged "super packet" made from many wire packets, which you have to keep in mind when rewriting headers.

cgroup sock_addr hooks (`connect4/6`, `sendmsg4/6`, `getpeername`): per-connection, at the syscall

- **Runs:** when a local process calls `connect()` (or `sendmsg()` for UDP).
- **Unique strength: load balancing without NAT.** The client calls `connect(VIP)`, your program rewrites the destination to a real backend *once*, and every packet after that goes directly. No per-packet translation, no conntrack table, no checksum fixups. The `getpeername` hook can even make the app still see the VIP. Cilium's "socket-LB" replacement for kube-proxy works this way.
- **Can't:** handle traffic from other machines. It only works when the client runs on a host you control, which means internal (east-west) service-to-service traffic, not traffic from the internet.
- **Why this matters to you:** it's a good "why" lesson in light of mini-katran. All the DNAT, conntrack and checksum work exists because an L4 LB doesn't control the client. When you *do* control the client, you make the decision once at `connect()` and none of that work is needed.

`sk_lookup`: which listening socket receives this?

- **Runs:** when the kernel looks for the socket that should receive an incoming packet or connection.
- **Unique strength:** steer traffic for any IP or port to any socket, without binding to it. One socket can serve thousands of addresses, or you can move traffic between server processes. Cloudflare added this hook upstream.
- **Can't:** modify packets. It only chooses a socket.

`SO_REUSEPORT` programs (`sk_reuseport`): which socket in the group?

- **Runs:** when several sockets share a port via `SO_REUSEPORT` and a new connection (TCP) or datagram (UDP) arrives.

- **Unique strength:** load balancing **across processes or threads on one host** with your own policy, for example least queue depth. Also used for zero-downtime restarts (steer new connections to the new process while the old one drains) and for QUIC (steer by connection ID).
- **Relevance:** this is the fix for the "one NGINX worker takes all the load" problem.

`sockops`: TCP connection events

- **Runs:** on TCP lifecycle events: connection established, RTT updates, retransmits, state changes.
- **Unique strength:**
  - Per-connection visibility and control from inside the kernel: read RTT and retransmits.
  - Set congestion control or TCP options per connection.
  - Write and read custom TCP header options. For example, a backend could put its load in a TCP option, and the LB could read it in-band.
  - Often used to fill a sockmap.

`sockmap` / `sk_msg` / `sk_skb`: move data between sockets

- **Runs:** on data being sent or received on sockets registered in a sockmap or sockhash.
- **Unique strength: splice data from one socket to another inside the kernel.** A TCP proxy can forward client↔backend bytes without copying through user space.
- **Relevance:** your TCP proxy baseline could become an in-kernel proxy with sockmap. That would be an interesting middle point between "full user-space proxy" and "L4 forwarding".

Observability hooks (tracepoints, kprobes, fentry)

These don't change packets, but they're very useful for **measuring** your experiments. For example:

- `sock:inet_sock_set_state` shows every TCP state change.
- `tcp:tcp_receive_reset` shows resets.

So you can count "connections broken" precisely from the kernel instead of guessing from client errors.

Side-by-side

Hook	Granularity	Direction	Knows about sockets?	Modifies packets?	Key thing only it does well
XDP	per packet	ingress	lookup only	yes	Cheapest per-packet drop/forward/encap
TC	per packet	ingress + egress	lookup + skb info	yes	Egress, pacing/shaping, stack integration
cgroup `connect` / `sendmsg`	per connection	local client	yes (the caller's)	no, rewrites the address	LB with no NAT for local clients
`sk_lookup`	per new flow	ingress	chooses a socket	no	Any address/port → any socket
`sk_reuseport`	per new flow	ingress	chooses within a group	no	Balancing across workers, hot restart
`sockops`	per TCP event	both	yes	TCP options only	Per-connection metrics and tuning
`sockmap` / `sk_msg`	per message	socket↔socket	yes	the payload stream	In-kernel proxying
tracepoints	per event	n/a	yes	no	Precise measurement

How to choose

Ask three questions:

1. **Who controls the client?** Nobody (internet traffic): you need a packet-level hook (XDP/TC). You control the client host: decide at `connect()` and skip per-packet work.
2. **What information does the decision need?** Only header fields: XDP. Socket or connection state: TC, or a socket-level hook. Which worker should handle it: `sk_reuseport`.
3. **Which direction?** Only incoming: XDP is fine. Outgoing or replies: TC egress.

Mapped to your candidate projects

- **GLB-style:**
  - Director: XDP. It receives traffic it doesn't control, decides on headers only, and encapsulates.
  - Backend redirect: XDP or TC ingress. It decapsulates, calls `bpf_skc_lookup_tcp` on the inner 5-tuple, and if there's no socket and the packet isn't a SYN, re-encapsulates it towards the secondary. Both hooks can do this. TC is easier to debug and works alongside kernel tunnel devices; XDP is faster.
  - Measurement: tracepoints to count resets.
- **Unimog-style:** XDP on every server, with the bucket table in a BPF map. A user-space controller updates the map from load measurements.
- **Prequal:** mostly user space. Optionally, `sockops` could collect per-backend RTTs cheaply, or `sockmap` could speed up the proxy.
- **Per-tenant isolation:** XDP for ingress drops or rate limits per tenant; TC egress with pacing if you shape replies.

One practical note: the socket-level hooks (cgroup, `sk_lookup`, `sk_reuseport`, `sockops`) need no special NIC support at all. They work the same on every laptop, which fits your hardware.

Prompt 7

> the tracepoint idea for counting broken connections is really useful, we hadnt thought of measuring it from the kernel side. on that note, we want to learn how research papers on load balancing define their problem and evaluate it, testbed, tools, baselines, metrics, basically the whole workflow from problem to results. which papers should we read and in what order, given were leaning towards the unimog / glb direction

Claude Code

Below is a reading order for the GLB/Unimog direction. Read each paper as an example of method, not just for its design: how does it turn "connections break" into a precise problem, and how does it prove the fix works?

Step 0: how to read papers (everyone, about an hour)

- **S. Keshav, "How to Read a Paper"** (CCR 2007). The three-pass method. Do pass 1 (about 10 minutes) on every paper below before going deep on any one.
- **Gernot Heiser, "Systems Benchmarking Crimes"** (web page). A short list of common evaluation mistakes. Reading it first makes you much better at judging the evaluations in the papers that follow.
- **Gil Tene, "How NOT to Measure Latency"** (talk). Covers coordinated omission and why averages lie. Essential before you measure any latency.
- *Reference, not cover-to-cover:* Raj Jain, *The Art of Computer Systems Performance Analysis*, for experiment design and statistics when you need it.

An extraction template to fill for every paper

Have whoever reads a paper fill this in and share it. Together the templates become your related-work section and the basis of your own evaluation plan.

```
Paper:
1. Problem (one sentence):
2. Why it matters (what evidence? production data? an incident?):
3. Assumptions / what's out of scope:
4. Key idea (one sentence):
5. Testbed (machines, NICs, software, topology):
6. Workload (traffic type, flow sizes, connection lengths, attack traffic?):
7. Events tested (drain, add, LB failure, scale-out, flood):
8. Baselines (what they compare against, and why those):
9. Metrics:
10. Main figures (what does each one prove?):
11. Limitations (stated, and ones you notice):
12. What we can reuse for our project:
```

Reading order

Phase 1: the baseline designs everyone compares against

1. **Maglev** (Google, NSDI 2016), "Maglev: A Fast and Reliable Software Network Load Balancer". Read it for:
   - How it evaluates its consistent hashing: Maglev hashing vs a hash ring vs rendezvous hashing, scored on how much traffic stays put when backends change and how even the load is.
   - Two numbers that pull against each other, both measured. You'll need that same pattern.
2. **Ananta** (Microsoft, SIGCOMM 2013), "Ananta: Cloud Scale Load Balancing". Read it for:
   - How it motivates its design from the needs of a cloud provider: many tenants, cost, scaling.
   - How it splits work between a central tier and the hosts. It's the conceptual ancestor of "let the hosts help the LB".

Phase 2: defining the problem precisely

3. **SilkRoad** (SIGCOMM 2017), "SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs". **Read mainly the problem and motivation sections.** The switch implementation matters less for you. Read it for:
   - The definition of **per-connection consistency (PCC)**: a connection must stay on the same backend even when the pool changes. That's exactly your problem, and the term gives you vocabulary.
   - How it uses production measurements to show that pool changes are frequent, not rare. That evidence turns "edge case" into "real problem".

Phase 3: the papers closest to GLB/Unimog (most important)

4. **Beamer** (NSDI 2018), "Stateless Datacenter Load-balancing with Beamer".
   - The academic version of the GLB/Unimog idea: the LB keeps no per-connection state, and backends forward stray packets to the previous owner ("daisy chaining").
   - Its evaluation is a good model to copy. From the slides: 2 LBs, 8 servers, 700 running connections, a 1 Mpps SYN flood, then drain 1, 2 and 4 backends. The stateful baseline broke 87, 148 and 351 connections; Beamer broke none.
   - That is one clear claim, a stressful condition, a scaled event, a direct baseline, and one countable metric.

5. **Faild** (Fastly, NSDI 2018), "Balancing on the Edge: Transport Affinity without Network State".
    - The industry version of the same goal, for a CDN edge.
    - Read it to see how constraints change the design: commodity hardware, robustness against resource exhaustion.
    - It also has a section on surprises from real traffic, a kind of lesson that testbeds can't teach.
6. **Then read the GLB and Unimog blog posts right after Beamer and Faild.**
    - Same idea, different presentation. Notice what the blogs leave out: baselines, controlled experiments, stated limitations.
    - That gap is exactly what your project can fill: rebuild the industry idea and evaluate it the way a paper would.

Phase 4: sharper and newer

7. **Cheetah** (KTH, NSDI 2020), "A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency".
    - Guarantees PCC while allowing *any* balancing policy, not just hashing. It encodes which server owns a connection in a cookie carried in the packets.
    - This bridges to Unimog's load-aware side. The paper evaluates both connection consistency and how well load is spread (flow completion time), the same two-sided evaluation you'd need.
8. **The XDP paper** (CoNEXT 2018), "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel".
    - Read for methodology: how to measure packet processing on Linux fairly. It also includes a load-balancer example.
    - It's the most directly relevant paper for your tools.

Phase 5: optional, to compare a different style

9. **Prequal** (Google, NSDI 2024). How a production paper evaluates tail latency and utilization. Good contrast with the L4 papers.
10. **Dean & Barroso, "The Tail at Scale"** (CACM 2013). Short, and shapes how you think about latency metrics.

Splitting it across four people

- **Everyone:** Step 0 plus pass 1 of every paper.
- **Deep reads:**
    - Person A: Maglev + Ananta
    - Person B: SilkRoad + Cheetah
    - Person C: Beamer + GLB blog
    - Person D: Faild + Unimog blog + the XDP paper
- **Then a session where each person presents their templates.**

Patterns you'll see across these evaluations

Element	What these papers typically do	Your equivalent
Motivation	Production data (how often pools change)	Cite their numbers and state your assumptions about change frequency
Testbed	A few servers, a traffic generator, the LB in DPDK/Click/P4	Namespaces + veth, XDP/tc, a scripted topology; state this honestly

Element	What these papers typically do	Your equivalent
Workload	Long-lived connections, realistic flow-size distributions, SYN floods as stress	Same; long-lived connections are what make breakage visible
Events	Drain/add backends, LB failure, LB scale-out	Same, plus ECMP next-hop changes in the router namespace
Baselines	Stateless hashing, a stateful conntrack LB, prior work (Maglev)	mod-N hashing, Maglev + conntrack (your old design!), Linux resilient nexthops
Metrics	Connections broken, load imbalance (max/mean), throughput per core, memory	Connections broken (tracepoints), max/mean load, p99 latency, map memory
Sensitivity	Vary the amount of change, number of backends, connection lifetime	Sweep these; this is where most of the insight comes from
Presentation	Time series around an event, CDFs, tables per configuration	Same

How this becomes your workflow

1. **Problem + evidence:** one paragraph using the PCC vocabulary, with numbers cited from SilkRoad, Beamer and the industry blogs.
2. **Plan the figures before writing code.** Write down the 4–6 figures you want in the final report, with axes and baselines. For example: "connections broken vs number of backends drained, for 4 designs", or "per-server CPU over time as Unimog-style rebalancing happens".
3. **Build the measurement setup** (topology script, load generator, tracepoint counters) and get the baselines running. Your old mini-katran design becomes a baseline, which is a nice way to reuse it.
4. **Build your design**, then run the planned experiments, then the sensitivity sweeps.
5. **Write up limitations honestly:** single laptop, generic XDP, synthetic traffic.

Step 2 is the habit research groups use most and students skip most often. If you can't draw the figure in advance, the problem isn't well defined yet.

Sources:

- Beamer, NSDI 2018 (USENIX page)
- Beamer paper PDF
- Beamer slides
- Faild, NSDI 2018
- Cheetah, NSDI 2020
- KTH blog post on Cheetah
- SilkRoad paper PDF
- Prequal, NSDI 2024

Prompt 8

the extraction template and the "plan the figures first" idea are great, were going to use both. can you go through maglev's and beamer's evaluation sections in more detail? like what experiments they run in what order, what they compare against, the testbed, traffic tools, metrics and how many runs, so we can model ours on them

Claude Code

I have the full text of both papers. Maglev's evaluation is done; now I'm reading Beamer's evaluation section.

Both papers' evaluation sections are read, and I'm writing up the breakdown.

I worked from the papers' full text, not summaries. That turned up an error in my last message: I quoted Beamer's SYN-flood results as 87 / 148 / 351 broken connections (from the slides via a search summary). The paper itself says **87±2, 184±8, 351±21**. Details below.

Maglev (NSDI 2016), Section 5

Framing: "We evaluate Maglev's efficiency and performance," using one production cluster plus testbed microbenchmarks. There are three subsections, each backing one claim.

5.1 Load balancing (production data, no baseline)

- **Data:** connections per second to 458 backends in one European cluster over a typical day, sampled every 5 minutes and normalised by the daily average.
- **Metrics:**
  - Mean and standard deviation of load per backend.
  - **Coefficient of variation** (standard deviation / mean): 6–7% most of the time.
  - **Overprovision factor** (max load / mean load): below 1.2 more than 60% of the time, higher off-peak. They explain why: it is harder to balance little traffic.
- **Figure:** one time series over the day (Fig. 7).
- **Why it matters:** max/mean is the metric that matters operationally, because the busiest backend sets how much capacity you must buy. Steal this metric.

5.2 Single-machine throughput (testbed)

- **Testbed:** 2 senders, 2 receivers, 1 Maglev machine, all in one Ethernet domain. Each machine had two 8-core CPUs, a 10 Gbps NIC and 128 GB RAM. Maglev got one CPU socket to itself; the OS and everything else ran on the other.
- **Method:**
  - Senders slowly increase their rate. Throughput is defined as the **maximum packets/s before Maglev starts dropping**.
  - Minimum-size packets, because packets per second (pps) is what limits a forwarder.
  - Each forwarding thread is pinned to its own core.
  - Main knob: the number of forwarding threads, 1–7.
- **Traffic generator:** not named.
- **Experiments, in order:**
  1. **Kernel bypass vs the normal Linux stack** (Fig. 8). This is an *ablation*: the baseline is their own system with one design choice removed. With bypass the NIC becomes the bottleneck at 5 or more threads; through Linux, Maglev reaches under 30% of that.
  2. **Traffic type** (Fig. 9):
     - SYN-only: every packet needs a backend choice, which is what a SYN flood looks like.
     - Non-SYN: only a connection-table lookup.
     - A single constant 5-tuple: best case for the cache.
     - Result: SYNs need 6 threads to saturate the NIC instead of 5. Each result is explained by which code path the packet takes.
  3. **NIC speed** (Fig. 10): swap in a 40 Gbps NIC to find the real ceiling (a bit over 15 Mpps) and name the new bottleneck (their packet-steering module).

5.3 Consistent hashing (simulation, no packets)

- **Baselines:** Karger's ring hashing (1000 views) and rendezvous hashing.
- **Parameters:** 1000 backends; table sizes 65,537 and 655,373.

- **Metric 1, balance:** maximum and minimum % of table entries per backend (Fig. 11). With the small table, Karger needs 29.7% overprovisioning and rendezvous 49.5%; with the large table, 10.3% and 12.3%. Maglev is near-perfect at both sizes.
- **Metric 2, resilience:** % of table entries that change vs % of backends failing at once (Fig. 12).
  - Remove k random backends, rebuild the table, count the changed entries, **repeat 200 times** and average.
  - Only Maglev is measured, because the other two guarantee no collateral change.
- **Stated trade-off:** a bigger table is more resilient, but building it takes 22.9 ms instead of 1.8 ms.

What Maglev doesn't do

- **No experiment counts broken connections end to end.** Resilience is only a table-level simulation. They say the connection table is "the primary means of protection" and never test it under backend or LB churn.
- **No latency measurement.**
- **Traffic tool not named.** Only the 200 simulation runs are stated; the testbed numbers give no run count.

Beamer later fills exactly the first gap and finds Maglev hashing breaks 2.3–3.3% of unrelated ("innocent") connections when nodes are removed.

Beamer (NSDI 2018), Section 6 and Appendix

Framing: "test the performance, correctness, fault tolerance and deployability." Four properties, and every experiment maps to one of them. The headline claims are stated before the details: no connections dropped, twice the best software dataplane, negligible latency.

Testbed:

- 20 Xeon servers connected to a 48-port 10 Gbps BGP router.
- For the mux-churn experiment, an IBM 8264 switch acts as the border router.
- Amazon EC2 for the controller-scale experiment: 100 muxes, 64K backends, 6.4M buckets.
- **Implementation:** a Click/FastClick software mux on netmap, a P4 mux (software model and NetFPGA simulator), and a server-side kernel module that does decapsulation plus daisy chaining.

The key baseline decision: Ananta's and Maglev's code isn't public, so they **built "Stateful": their own mux with a per-flow hash table added.** Same codebase, one design difference, so the comparison isolates the idea. They also cross-check against Maglev's published numbers (2.8 Mpps per core, similar to Stateful).

6.1 Microbenchmarks: what does a packet cost?

Experiment	Setup	Knob	Metric / result
Forwarding vs packet size (Fig. 13)	1 core, 1000 buckets, 10G Intel 82599 NIC, netmap `pkt-gen` as generator and receiver, one packet size per run	Packet size	pps; Beamer is 2–3× faster than Stateful with 1M active flows
Stateful vs number of flows (Fig. 15)	Same	Active flows: 1K → 100M	3.9 → 2.3 Mpps; the flow table thrashes the CPU cache
Core scaling	Same	Cores	Line rate at ≥2 cores; explains why 64B packets fall short of 14.88 Mpps (encapsulation overhead)
Big box (Fig. 14)	4×10G NICs, 4 clients, 4 servers	Number of buckets (up to 1M)	5.6 → 5.1 Mpps per core; 23–33 Mpps per box

Experiment	Setup	Knob	Metric / result
Realistic traffic	Replay tool using packet sizes from MAWI traces (real HTTP uplink)	n/a	36 Gbps using 7 of 10 cores; extrapolated to "one mux serves 50–500 servers"
Server overhead	10G `iperf` into a backend	Kernel module on/off	CPU 7% → 9%
Latency (Fig. 16)	Mux on 1 core with 64B background traffic; high-frequency `ping` through it	Background rate	CDF; under 0.2 ms until saturation; at 600 Kpps over capacity, 1.5 ms and 14% loss (and they explain why)
P4	Software model used only to check correctness (they say its performance numbers are meaningless); NetFPGA simulator	n/a	About 60 Mpps estimated

6.2 Scalability and robustness: the core of the paper

1. **Mux churn** (Fig. 17): 5 clients and 6 servers running `iperf` to the virtual IP. Start with the P4 mux alone, add 3 software muxes a minute apart, kill a Click mux at 240 s, then kill the P4 mux. Plot total throughput over time with the events marked. Result: **not a single iperf connection broken.**
2. **Server churn** (Fig. 18): 64B packets into one mux. Start with 1 server and add one every 30 s. Plot per-server throughput over time, showing load spread evenly after each change.
3. **Connection affinity** (a table):
   - Workload: 7 clients × 100 persistent HTTP connections, each continuously downloading 1 MB files.
   - Event: start with 2 muxes and 8 servers, remove N servers, wait 30 s, then remove one mux.

Backends removed	0	1	2	4
Stateful	0	54±7	103±14	214±48
Beamer	0	0	0	0

   They explain *why* Stateful breaks: after a mux disappears, its flow state is gone, the survivor rehashes those flows, and the new backend replies with RST.
4. **SYN flood** (a table): same setup, only backends removed, with a 1 Mpps SYN flood running alongside.
   - With no flood, both designs break zero connections.
   - With the flood: Stateful **87±2 / 184±8 / 351±21** for 1 / 2 / 4 backends removed; Beamer 0. The flood pushes real flows out of Stateful's table.
   - They also measure the flood's effect on the servers: +1% CPU, and flow completion time (FCT) median 1 → 1.3 ms, p99 1.4 → 1.7 ms.

6.3 Full scenario with MPTCP

- **Workload:** `siege`, 4 clients × 10 parallel downloads of a 100 MB file, each restarting as soon as it finishes ("closed loop").
- **Script:** 4 muxes and 1 server. Add a server every minute until there are 6, then kill one mux per minute until 1 is left.
- **Figures:**
  - Median and p99 FCT over time, Beamer (Fig. 19) vs Stateful (Fig. 20).
  - Cumulative packets per server (Fig. 21).
  - **Packets daisy-chained per server** (Fig. 22): about 30K total, mostly ACKs. This shows the mechanism actually ran and is cheap.

- **Cross-check:** `siege` reported no failures, but they noticed MPTCP could *hide* failures. So they inspected per-subflow statistics: no resets, but timeouts that MPTCP covered for. This is exactly why your kernel tracepoints are a good idea: a client-side metric can lie.

6.4 Controller scale (EC2)

Time to add or remove backends vs how many already exist (a table: about 0.6 s at 1 backend, about 9.8 s at 32K), plus CDFs.

Appendix A.2: hashing comparison

- **Parameters:** reused exactly from Maglev's §5.3 (1000 servers, 65,537 buckets) "for fair comparison".
- **Metrics:** imbalance (max/mean), number of hardware rules needed, and innocent connections broken when servers are removed (Maglev 2.3–3.3%).

What Beamer doesn't do

- **The ± values are never defined**: not how many runs, nor whether they are standard deviations or confidence intervals.
- No run counts for the testbed experiments.
- The P4 hardware result comes from a simulator.

Both papers are in Heiser's "benchmarking crimes" territory on statistics. Don't copy that part.

Side by side

	Maglev	**Beamer**
Claims → experiments	3 subsections, one claim each	4 named properties, every experiment tied to one
Baselines	Its own ablation (Linux vs bypass); other hashing algorithms in simulation	**Own stateful variant in the same codebase**, plus Maglev's published numbers and parameters
Testbed	5 machines, one L2 domain	20 servers + a real BGP router; EC2 for scale
Tools	Unnamed generator	netmap `pkt-gen`, `iperf`, `ping`, `siege`, a MAWI trace replayer
Throughput method	Ramp rate until drops	Fixed packet size per run, measured at the receiver
Correctness under churn	Table-level simulation only	End-to-end broken-connection counts, scaled by event size
Stress / adversarial	SYN-only traffic (throughput)	SYN flood during churn (correctness)
Figure types	Time series (production), bar charts, line plots	Tables of counts, **timelines with marked events**, CDFs
Runs / statistics	200 repeats for the simulation only	± with no definition

Lessons to copy

1. **Name your claims, then give each one an experiment.** For example: forwarding cost, connection consistency, load spread, overhead on backends.
2. **Order the experiments from the bottom up:**
    1. Cost per packet.
    2. Scaling knobs (flows, buckets, cores).
    3. Realistic traffic.
    4. Correctness under single events, scaled by event size.

5. The same under stress.
6. A full timeline scenario.
7. Control-plane cost.

3. **Build your baseline in your own codebase,** differing only in the idea being tested. Your old mini-katran design (stateful connection table + hashing) is your "Stateful".
4. **Reuse a prior paper's parameters** for anything you can compare directly (e.g. 1000 backends / 65,537 entries for the hashing simulation).
5. **Scale the event:** 1, 2 and 4 backends removed, not just one.
6. **Show the mechanism working,** not only the outcome (Beamer's daisy-chained packet counts).
7. **Cross-check the metric** with a second source (client errors vs kernel tracepoints).
8. **Explain every odd number** (why it's below line rate, why latency jumps).
9. **Do better than both on statistics:** state the number of runs and what the error bars mean.

Translated into your setup (namespaces, XDP/tc)

#	Experiment	Modelled on	How in your setup
E1	Cost per packet: your director vs Stateful, vs number of active flows	Beamer Figs. 13 & 15	Turn on BPF runtime stats (`sysctl kernel.bpf_stats_enabled=1` ; `bpftool prog show` then reports `run_time_ns` / `run_cnt` , i.e. ns per packet), or use `BPF_PROG_TEST_RUN` with repeats. Report *relative* cost only
E2	Hashing simulation: balance + innocent disruption	Maglev §5.3 / Beamer A.2	Pure user space, Maglev's parameters, 200 repeats
E3	Connection affinity: remove 1/2/4 backends, then one LB	Beamer affinity table	N clients × 100 persistent connections downloading in a loop; ECMP next-hop removal in the router namespace; count resets via tracepoints **and** client errors
E4	Same as E3, with a SYN flood	Beamer SYN-flood table	`hping3` flood. Size Stateful's LRU map so eviction actually happens, and say so
E5	Timeline: add/remove backends and LBs every 30 s	Beamer Figs. 17/18/22	Per-backend throughput over time, plus a "packets forwarded to the second-chance or previous owner" counter in a BPF map
E6	(Unimog stage) Load spread with uneven backends	Maglev Fig. 7 metric	cgroup CPU limits; **max/mean** load and coefficient of variation over time
E7	Overhead on backends	Beamer server overhead	`iperf3` into a backend with your tc/XDP hook on vs off
E8	Latency through the LB	Beamer Fig. 16	`wrk2` at fixed rates; CDF of p50/p99

Statistics rule for your project:

- At least 5 runs per configuration for the testbed experiments.
- Report mean ± standard deviation (or median with min–max) and *say which*.
- Fix random seeds for the simulations.
- Publish the setup scripts with the results.

The papers and extracted text are in `/tmp/claude-1000/-home-m4nny-Code-Courses-cs331-lb/43eb207f-3643-4f43-b206-23432dca2abe/scratchpad/papers/` (`maglev.pdf`, `beamer.pdf`, plus `.txt` versions). Copy them somewhere permanent; the scratchpad is temporary.

Prompt 9

this is exactly what we needed, well copy the papers into the repo. one thing that made us pause: your E6 measures success as even cpu (max/mean), and unimog itself balances by cpu, but we came across the prequal paper which argues you shouldnt balance cpu load at all. whats the reasoning behind that, and does it undermine the unimog approach or that metric

Claude Code

I've read Prequal's argument. Now checking what Unimog actually balances on, so I compare them accurately.

Short answer: Prequal's argument doesn't undermine the Unimog *approach*, because the two balance different things, at different timescales, for different goals. It does undermine using max/mean CPU as your *only* measure of success. That's worth fixing, and you can turn the disagreement into part of your project.

I checked both against their primary sources: the Prequal paper, now saved next to the other two in `/tmp/claude-1000/-home-m4nny-Code-Courses-cs331-lb/43eb207f-3643-4f43-b206-23432dca2abe/scratchpad/papers/prequal.pdf`, and the Unimog blog post.

Prequal's argument

The setting: Google's multi-tenant clusters. Each server "replica" runs in a VM with a guaranteed CPU *allocation* and shares the machine with other tenants' VMs ("antagonists"). The previous policy, weighted round robin (WRR), set each replica's weight to `QPS / CPU utilization`, so it equalised *replica* CPU utilization.

Why CPU balancing backfires, using their own example (§2):

- 100 replicas on identical machines, each allocated 40% of its machine.
- On machines 1 and 2, antagonists use the remaining 60%. The other 98 machines have spare capacity.
- Demand spikes to 1.1× the allocation, so on average each replica needs 44%.
- CPU balancing pushes *every* replica to 44%:
  - On 98 machines that's fine; they borrow idle cycles.
  - On machines 1 and 2 there's nothing to borrow. CPU isolation kicks in and throttles those replicas.
- So about 0.18% of the load causes problems for 2% of all queries, and p99 latency spikes.
- **Equal CPU was exactly the wrong target, because the machines had very different headroom.**

Three reasons CPU is a poor signal for this goal:

1. **It's a trailing signal.** Utilization only means something averaged over a window, so it's always somewhat stale.
2. **Coarse sampling hides overload.** Their Fig. 3: at 1-minute samples every replica looks within its limit. At 1-second samples, replicas often burst to nearly 2× the limit. "At sufficiently small timescales, there is nearly always some replica in overload."
3. **It misses other contention** (locks, memory bandwidth) that also drives latency.

What they use instead:

- **Requests in flight (RIF):** exact at the moment of a probe, and a leading indicator of future load and memory use.
- **Estimated latency.**

- Both are gathered by probing a few servers ("power of d choices") and choosing per request.

Results on YouTube Homepage vs WRR: roughly 2× lower tail latency, 5–10× lower tail RIF, 10–20% lower tail memory, 2× lower tail CPU, and errors from load imbalance nearly gone.

Their own concession: equalising utilization "can be a great idea if all replicas always stay within their allocation, as this maximizes the amount of traffic we can serve." The problem is overload combined with uneven headroom.

Why that doesn't undermine Unimog

	Prequal	Unimog
Goal	Tail latency per request, at high utilization	**Efficiency**: use every server's capacity evenly across mixed hardware generations; LB overhead under 1% of CPU
What "CPU" means	*Replica* (VM) utilization, blind to antagonists on the same machine	*Whole-server* processor utilization from Prometheus (with "some intricacies"). The blog says it is designed to include "other activities ... not directly driven by connections"
Decision unit	Each request (RPC)	Each *new connection* (L4 can't see requests); existing connections never move
Timescale	Milliseconds, using live probes	A control loop over periodic metrics: seconds or more
Heterogeneity it handles	Unpredictable, short-lived contention from other tenants	Persistent hardware differences and background work

1. **Unimog measures the thing Prequal says WRR missed.** Prequal's trap was balancing the replica's *own* CPU while the *machine* was contended. Prequal's footnote 2 even says balancing whole-machine CPU would fight Google's isolation system. But Cloudflare's servers aren't shared with other tenants, so whole-machine CPU is the honest measure of how much capacity is left.
2. **The goals differ.** Unimog asks: over minutes, are all my servers doing their fair share, so I don't buy hardware for hot spots? Prequal asks: right now, which replica will answer this request fastest? Both are called "load balancing", but they optimise different things. It's the same lesson as Meta vs Stripe earlier: define the objective first.
3. **An L4 balancer can't act on millisecond bursts anyway.** It only steers new connections, through a slow control loop. A precise per-request signal like RIF would mostly be wasted at L4. Fast tail-latency control belongs at L7 (Prequal's layer). In a real stack the two are complementary.

Where the critique does apply to you

- **Your E6 metric.** Max/mean CPU sampled every 10 s could look perfect while 1-second bursts on one backend wreck p99. CPU balance is a means; latency and errors are what clients experience.
- **Your signal choice.** If your controller measures only the backend process's cgroup CPU, it falls into exactly Prequal's trap: blind to an antagonist sharing its cores.
- **Unimog's own limit.** Its signal is still averaged and lagging, and connections are sticky. A few heavy long-lived connections on one server can't be moved. Your evaluation should show that limit, not hide it.

How to revise E6

Report CPU balance at two timescales, plus what clients experience:

- Max/mean CPU and coefficient of variation at **1 s and 30–60 s** samples. The gap between them is a result in itself; it reproduces Prequal's Fig. 3 effect.
- **p99 latency and error rate**, overall and per backend.

Add a scenario built to break CPU balancing (Prequal's §2 example, scaled down):

- Each backend's server process gets a cgroup `cpu.max` limit, playing the role of the VM allocation.
- On backends 1–2, run a `stress-ng` antagonist in a separate cgroup pinned to the same cores (`cpuset`).
- Ramp demand past the total allocation.
- Prediction: a controller using the backend's own CPU keeps loading backends 1–2, and their p99 explodes. A controller using a contention-aware signal moves load away.

Make the load signal an experimental variable:

- Backend cgroup CPU (`cpu.stat` usage): what Prequal warns against.
- Whole-core CPU utilization (`/proc/stat` per core): closest to what Unimog uses.
- **CPU pressure (PSI):** `cpu.pressure` per cgroup, using `some avg10`. It reports how long tasks *waited* for CPU, so it captures contention directly and leads utilization. It's cheap and a good fit for an L4 controller.
- Active connections per backend: a cheap L4 counterpart to Prequal's RIF.

That turns a possible objection into your research question:

> *For a load-aware L4 balancer that only controls where new connections go, which load signal best keeps p99 latency low and avoids broken connections when servers differ in capacity and face contention from other workloads?*

The baselines are static weights and your old stateful hashing design. The comparison points are the four signals. The metrics are p99 latency, errors, CPU spread at two timescales, and connections broken during rebalancing. That's sharper than "we rebuilt Unimog". Your readers already see the tension, and your evaluation answers it.

Prompt 10

this reframing is really good, the four signals idea makes the question much sharper and the psi signal is new to us. before we lock it in, can you explain how unimog works in detail? especially the bucket table, how the controller moves buckets, and the daisy chaining / second hop part, and how that differs from what github glb does

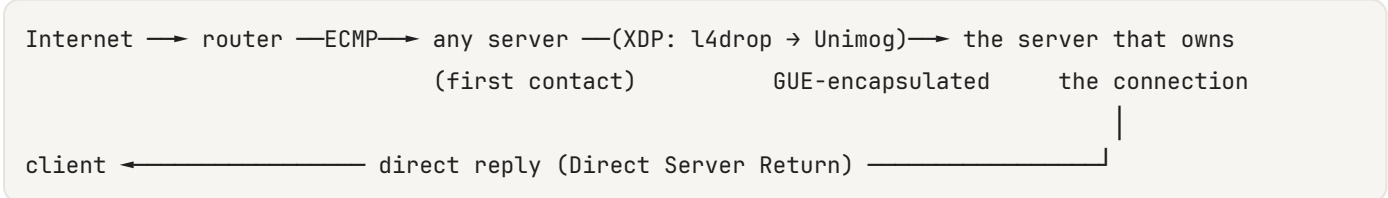
Claude Code

I've got the full Unimog design from Cloudflare's post. Next I'm reading GitHub's GLB post so the comparison is accurate, not from memory.

Both systems share one core idea: the LB keeps no per-connection state. Instead, every forwarding-table entry names two servers, and the server that receives a packet checks its own TCP sockets to decide whether to keep it or pass it on. They differ in what the second server *means*, who builds the table, and whether load is balanced dynamically. Everything below comes from Cloudflare's Unimog post and GitHub's GLB post, read in full. Where I'm inferring beyond what they say, I say so.

Unimog

1. Where it sits: every server is a load balancer



- **No separate LB tier.** The router spreads packets across all servers with ECMP. The server that receives a packet runs Unimog in XDP and forwards it, encapsulated, to the server that should handle it.
- Cloudflare gives two reasons:

1. They avoid specialised server roles.
2. Their DDoS filter (l4drop) already runs in XDP on every server, *before* Unimog, so attack packets are dropped before any forwarding happens.

- **ECMP rehashes no longer matter.** Every server holds the same forwarding table, so it doesn't matter which server the router picks.
- **Encapsulation is GUE** (UDP-based). They chose it so they could reuse GitHub's redirector. A 1500-byte packet becomes 1536 bytes, so they run jumbo frames inside the datacenter.

2. The forwarding table (buckets)

- A **power-of-two-sized array** of buckets. The bucket index is the **low N bits of a hash of the 4-tuple** (source/destination IP and port).
- There are **more than 100× as many buckets as servers**, so tens of thousands in a datacenter with hundreds of servers.
  - A server appears in many buckets, and **the number of buckets it owns is its share of new connections.**
  - That's how they get fine-grained weights, even though each bucket holds many connections and the per-bucket counts are only statistically equal.
- The table is **immutable**. The control plane builds a whole new table and sends it out (via Consul); every server swaps it in.
- There's **one table per "trafficset"** (a group of VIP ranges for one service), so different services can be balanced differently.

Each bucket has two slots:

```
bucket i:  [ first hop: current owner (gets NEW connections) | second hop: PREVIOUS owner ]
```

3. Forwarding with a second hop (daisy chaining, borrowed from Beamer)

Data plane on the server that receives the packet (XDP):

1. Is the destination a VIP? If not, pass it to the normal stack.
2. Look up bucket = hash(4-tuple) & (size − 1).
3. Encapsulate the packet towards the **first-hop** server, putting the **second-hop** address in a GUE extension header. That way the next server never needs to look at the table itself.

Redirector on the first-hop server:

```
if packet is SYN:                          → handle locally (new connections go to first hop)
elif local TCP socket matches 4-tuple:     → handle locally (existing connection here)
else:                                      → re-encapsulate to the second-hop address from the GUE
header
```

- In practice **under 1% of forwarded packets take the second hop**, even with long-lived connections.
- **History of the redirector:**
  - First they used GitHub's `glb-redirect` (an iptables/kernel module).
  - Then they rewrote it as `cls-redirect`**, a TC-BPF program** that also does the decapsulation.
  - They picked **TC instead of XDP on purpose**: XDP hides packets from `tcpdump`, which makes debugging hard, and the redirector passes most packets straight through, so XDP's speed wouldn't help.
  - They contributed it to the kernel's BPF self-tests, so you can read real code. Look for `test_cls_redirect` under `tools/testing/selftests/bpf/`.

4. How bucket changes interact with connections

Unimog keeps only two hops, while Beamer can chain further. So a connection survives **one** change to its bucket, not two:

```
t0: bucket = [A | –]      conn X established on A
t1: bucket = [B | A]      X: arrives at B → no socket → second hop to A ✔️
t2: bucket = [C | B]      X: arrives at C → no socket → second hop to B → no socket → RST X
```

Cloudflare's two answers:

1. **Swapping the two slots is free.** Changing `[B | A]` to `[A | B]` only changes who gets *new* connections. A's old connections land on A directly; B's connections arrive at A, find no socket there, and go on to B. Nothing breaks. Cloudflare says **most of their load balancing is done with swaps like this.**
2. **When a real reassignment is needed** (e.g. a new server needs buckets), change the **least-recently modified** buckets. Their connections are the most likely to have already finished.

They also deliberately left out Beamer's generation numbers. Those handle a corner case where an ECMP rehash coincides with a table update that's still propagating, and Cloudflare judged the damage negligible in their setting.

5. The controller ("conductor")

Inputs:

- Server inventory and status (including "draining").
- Node- and service-level health checks from Consul.
- **Load:** a Prometheus expression for processor utilization ("with some intricacies").
- VIP ranges.

The control loop (as published):

- Every period, compare each server's load with the **average**.
- Adjust its bucket count **in proportion to how far it is from the average**.
- So loads converge on the mean. This is a *proportional controller*.

What isn't published: the exact period, the gain, any dead band or smoothing, and how buckets are picked beyond "prefer swaps, then least-recently modified". You'd design those yourselves, and they're fair game for evaluation.

A real operational lesson they share:

- In a datacenter-wide overload, many servers looked unhealthy, so the conductor stopped sending them new connections.
- That overloaded the remaining servers, while the first group recovered.
- The datacenter could then **oscillate** between healthy and degraded even after demand dropped back.
- Their fix: tell isolated degraded servers apart from datacenter-wide problems.

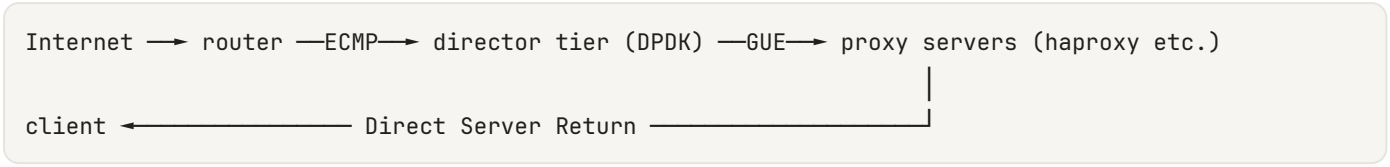
The lesson: a feedback controller adds its own failure modes.

6. UDP

- UDP has no SYN, so the redirector logic becomes "local socket? keep it : second hop". That means **new UDP flows go to the second-hop server**, so for UDP the controller puts the new owner in the *second* slot.
- Services must use **connected** UDP sockets so the redirector can match flows to them.
- Protocols with session IDs (QUIC, their WireGuard extension for WARP) use a pluggable "flow dissector" that hashes the session ID instead of the 4-tuple, so flows survive client IP changes.

GLB (GitHub)

1. Where it sits: a separate director tier



- **Directors** are dedicated LB machines running **DPDK** on an SR-IOV virtual function. A NIC feature ("flow bifurcation") sends GLB's VIP traffic to DPDK and everything else to Linux.
- Cloudflare cites the DPDK requirement as a main reason GLB didn't suit them, since every Cloudflare server is also an application server.

2. The forwarding table: rendezvous hashing with primary/secondary

- A fixed table of **65,536 rows** (about 512 KB). The 4-tuple hash picks a row.
- Each row holds **(primary, secondary)**.
- **The table is computed from the membership list, not chosen by a controller.** For each row:
  - hash (server IP, row number) for every server and sort by that value;
  - the top two become primary and secondary.
- This gives the invariants they want:
  - each server appears about equally often as primary and as secondary;
  - never both in the same row;
  - adding a server makes it primary in about 1/N of rows, with the old primary moving to secondary;
  - removing a server promotes the secondary wherever the removed server was primary.
- **No dynamic weights.** Every active server gets an equal share.

3. The redirector: "second chance"

- A Netfilter / iptables module on each proxy. It accepts the packet if it's a SYN or matches a local socket. Otherwise it forwards to the secondary, whose address is in the GUE private data.
- Decapsulation uses the kernel's `fou` /GUE support.
- They fill the outer **UDP source port with a per-connection hash**, so ECMP inside the datacenter and the receiving NIC's queue spreading (RSS) both spread load. IPIP can't do this, because most NICs can't look inside IPIP packets.

4. Server state machine

- States: `active` → `draining` → removed, and new servers go through `filling` → `active`.
- **Draining** = swap primary and secondary in the rows where that server is primary. The other server takes new connections; the draining server keeps its existing ones through the second chance. This is the same "swap is free" trick Unimog uses.
- **Constraint: at most one server may be in a non-active state at a time,** because each row only has two slots. Changes can only go as fast as the longest connection you want to keep.
- **Health failure** = the same swap, a "soft drain". If the health check was a false positive, connections just take a slightly longer path instead of breaking.

Side by side

	GLB	Unimog
Problem framing	Scale one IP across many proxies; drain/fill without breaking connections	Same, **plus** dynamic load balancing across mixed hardware, on servers that also do everything else

	GLB	Unimog
Where the LB runs	Separate director tier	Every server (no LB tier)
Data plane	DPDK on SR-IOV	XDP, chained after the DDoS filter
Table	65,536 rows, derived from membership via rendezvous hashing	Power-of-two buckets (>100× servers), **chosen by a controller**, immutable versions distributed via Consul
Meaning of the 2nd slot	The **next choice** in hash order (precomputed fallback)	The **previous owner** (history)
Weights	Equal	Bucket counts set by a CPU-driven proportional loop
Allowed changes	At most 1 server non-active at a time	Any update; each bucket survives **one** change; swaps are free; prefer least-recently modified buckets
Redirector	iptables kernel module + `fou` decap	`cls-redirect`, TC-BPF, does decap too
Protocols	TCP (+ ICMP "fragmentation needed" for path MTU discovery)	TCP, UDP (connected sockets), custom flow dissectors (QUIC-style)
Shared	GUE with the second-hop address in the header; Direct Server Return; state lives in backend sockets	same
Relation to Beamer	Two-hop daisy chaining	Two-hop daisy chaining, without generation numbers

In one sentence: GLB is daisy chaining with a *fixed* table, used for planned membership changes. Unimog is daisy chaining with a *controller-written* table, used continuously for load balancing. That turns the table from configuration into a control output.

What this means for your project

1. GLB is a natural first milestone, and Unimog the second.

- The redirector (TC: decap + `bpf_skc_lookup_tcp` + re-encapsulation) is identical in both.
- Start with a rendezvous-generated table and drain/fill, then replace the table generator with a controller.

2. Data structures (my suggestion, not from the posts):

- A BPF array map of `{first, second}` per bucket.
- To match Unimog's "immutable table, swapped whole", use **map-in-map** (`BPF_MAP_TYPE_ARRAY_OF_MAPS`). Build the new table in a fresh inner map and switch the outer pointer in one update, so packets never see a half-updated table.

3. There's a design tension in the controller that becomes part of your research question. A faster or more aggressive controller balances load better, but it changes buckets more often. Any connection whose bucket changes twice during its lifetime breaks. So "which signal?" grows into a two-dimensional design space:

Axis	Choices
Load signal	Backend cgroup CPU, whole-core CPU, PSI, active connections
Controller	Period, gain, dead band, swap-first vs reassign, bucket-selection policy

4. Metrics that fall out of this:

- Load spread at 1 s and 60 s (as before).
- p99 latency and errors.
- **Bucket changes per minute, and how many are swaps vs real reassignments.**

- **Connections broken by a second bucket change** (tracepoint resets correlated with your table-change log).
- **Time to converge and oscillation** after a step change in load. That's Cloudflare's oscillation story, reproduced on purpose.

5. Stay TCP-only. UDP flips the slot semantics and needs connected sockets. It's interesting, but best left as future work.

Sources: Unimog – Cloudflare's edge load balancer, GLB: GitHub's open source load balancer. Text copies are in `/tmp/claude-1000/-home-m4nny-Code-Courses-cs331-lb/43eb207f-3643-4f43-b206-23432dca2abe/scratchpad/web/` (`unimog.txt`, `glb.txt`), if you want them alongside the papers in the repo.

Prompt 11

thanks, the swap trick and the least recently modified rule are really clever. one worry though: if the servers are unequal but their capacity doesnt change, couldnt static weights by core count get almost all of the benefit? we want to make sure the problem is actually strong. how would we define what a good vs bad load balancer is here, ideally one headline number, in a way that shows when the dynamic controller is actually worth it

Claude Code

Your worry is right, and taking it seriously makes the problem stronger. **If server capacity is fixed and known, and every connection costs about the same, static weights get nearly all the benefit.** A dynamic controller then only adds bucket churn and the risk of oscillation. So the question to answer isn't "is dynamic better?" It's **"under which conditions does dynamic beat the best static weights, and by how much?"** A problem where the answer could be "no" is a strong one.

When static weights fall short

Cloudflare's own justification (from the Unimog post) names three reasons. A fourth follows from L4's constraints:

1. **The capacity estimate is wrong.** Core count isn't capacity. Hardware generations differ per core, and the ratio depends on the workload.
2. **Connections don't all cost the same.** Bucket share equals *connection* share, not *load* share. A few heavy long-lived connections skew it.
3. **Other work runs on the servers**: background jobs, antagonists, throttling. It changes available capacity over time.
4. **An L4 balancer can only steer new connections.** Dynamic control can react no faster than the longer of its control period and connection turnover time. Disturbances shorter than that can't be fixed at L4 at all; that's Prequal's territory.

The headline number: usable capacity fraction

Definition:

> **Capacity at SLO** = the highest offered load (requests/s or new connections/s) the cluster sustains while **p99 latency ≤ L** and **error rate (including resets) ≤ E**.
>
> **Usable capacity fraction = (capacity at SLO through the LB) / (sum of each backend's own capacity at SLO, measured directly)**

- A perfect balancer scores about 1.0. A balancer scoring 0.75 wastes a quarter of the hardware.

Why this number:

- **It's what operators pay for.** They provision hardware against peak load under a latency target. Maglev's overprovision factor and Prequal's "run at higher utilization" both measure this idea.
- **It folds everything into one outcome:**
  - imbalance (the busiest server saturates first);
  - tail latency (p99 is in the SLO);
  - connection breakage (resets count as errors, so an over-eager controller that breaks connections is penalised automatically).
- **It has a known ceiling.** You measure the denominator by sending load to each backend directly, so the score is an absolute fraction, not just "better than baseline".
- **It connects to the metric you already have.** At saturation, the cluster is limited by its busiest server relative to its own capacity, so usable fraction ≈ mean / max of capacity-normalised utilization ≈ 1 / Maglev's overprovision factor. Max/mean becomes a *diagnostic* (why a design scored what it did), and capacity at SLO is the *outcome*.

Worked example (made-up numbers): servers with 4, 4, 2 and 2 cores, so total capacity is 12 units.

Balancer	Saturates when	Capacity	Usable fraction
Equal weights	The 2-core servers hit 100% at 3 units each	4 × 2 = 8	0.67
Static by cores	All saturate together	about 12	about 1.0
An antagonist takes 1 core on one 4-core server; available capacity is now 11			
↳ Static by cores	That server still gets 1/3 of the load but has capacity 3 → saturates at L = 9	9	9/11 = 0.82
↳ Ideal dynamic	Load follows the available capacity	about 11	about 1.0

This shows both the case where static is enough (the first two rows) and the case where dynamic helps (the last two), using the same metric.

How to measure it:

- Use an **open-loop** load generator: `wrk2` at fixed rates, or a generator that opens connections at a fixed rate.
- Raise the load in steps. Hold each step about 60 s after warm-up and compute p99 and errors per step. Capacity is the highest step that meets the SLO; refine with a binary search if you need precision.
- In scenarios that change over time, evaluate the SLO **per 10-second window** across the whole run. For example, the SLO holds if at least 99% of windows meet it; otherwise transient spikes get averaged away.
- Measure the denominator (each backend directly) with the same procedure and the same SLO.
- Report results for 2–3 values of L, so nobody can say you picked the SLO to suit your design.

Baselines, from weakest to strongest

1. **Equal weights.** What ECMP or plain hashing gives you.
2. **Static weights by core count.** What a sensible operator would configure.
3. **Static oracle.** Weights from benchmarking each backend's capacity offline, the best any static scheme can do. **This is the baseline your dynamic controller must beat.** Beating only #1 or #2 would be a straw man.
4. **Dynamic, one run per signal:** backend cgroup CPU, whole-core CPU, PSI, active connections.

5. **Upper bound** = 1.0, the sum of measured backend capacities.

Map where dynamic wins

Don't run one scenario. Run sweeps that show where the boundary lies:

Axis	What you vary	Prediction
Capacity estimate error	Make the nominal capacity (cores) differ from real capacity (`cpu.max` quotas that don't match core counts)	Core-count weights lose; static oracle and dynamic tie
Per-connection cost spread	Uniform vs heavy-tailed cost per connection (a few long, heavy downloads or websockets)	All static weights lose some; dynamic recovers part of it
Disturbance timescale	An antagonist on 1–2 backends switching on and off with period T, from 1 s to 10 min	Dynamic wins only when T is longer than both the control period and connection turnover; for very short T nobody can help, and an aggressive controller may *hurt* (oscillation)
Connection lifetime	Short (HTTP-like) vs long (minutes)	Longer lifetimes slow down how fast dynamic can rebalance, and raise the risk of a second bucket change breaking connections

The key figure:

- x-axis: disturbance period T (log scale).
- y-axis: usable capacity fraction.
- One line each for equal weights, static by cores, static oracle, and dynamic with each signal.
- Two separate figures, one for short and one for long connection lifetimes.

The expected shape:

- Lines converge at both ends: when nothing changes, static is enough; when changes are too fast, nothing helps.
- In the middle, dynamic is clearly above the static oracle.
- The signals separate in that middle band, and that answers your four-signals question.

A sanity check that strengthens the claim

In the static scenario, the dynamic controller should *match* the static oracle within noise, with no broken connections. If it's worse there, you've measured its cost (churn, oscillation). Report that honestly, because it's part of the answer.

The resulting claim template

> *"On a cluster with unequal servers and time-varying contention, a load-aware L4 balancer using [signal] achieves X% of ideal cluster capacity at p99 ≤ L ms, compared with Y% for the best static weights. It wins when disturbances last longer than [boundary]. When capacity is static it matches static weights within Z%, and no configuration broke connections."*

Every blank in that sentence comes from a figure you can plan now, which fits the "plan the figures first" approach.

Main threat to validity: you're building the disturbance scenarios yourselves, so someone could argue you chose ones that favour dynamic. The sweeps help, the static-wins scenario helps, and so does tying each scenario's parameters to real evidence: Cloudflare's mixed hardware generations, Prequal's 1-second CPU bursts, heavy-tailed connection sizes. Explain where each parameter comes from in the write-up.

Prompt 12

usable capacity fraction is exactly the kind of headline number we wanted, and the static oracle baseline makes sense. now scope: is this reasonable for a course project with 4 people? we were thinking the main comparison is whole-core cpu (unimog style) vs active connections / requests in flight (prequal style), with backend cgroup cpu and psi as extra signals if time allows. also one design choice were unsure about: should we run the lb on every server like unimog with a simulated ecmp router, or one dedicated lb machine, which seems simpler and easier to move to real laptops later?

Claude Code

Yes, it's reasonable for four people, provided you scope it carefully. The code is moderate. The real costs are **experiment time** and **measurement noise on one laptop**, and those should drive your scoping more than the number of features. On topology, I'd go with **one dedicated LB, designed to be stateless** so a second LB is a cheap add-on. Reasons below.

1. The signal comparison: one correction, then a scoping rule

A correction to the framing. In your design the LB can't see requests in flight, or even reliably how many connections are open. It's stateless, and with Direct Server Return it never sees the responses. So **every signal is reported by the backends and collected by the controller**:

Signal	Where it comes from
Whole-core CPU	`/proc/stat` per core
Requests in flight (RIF)	Counter in your backend app
Active connections	Established TCP sockets in the backend's namespace (e.g. the `CurrEstab` counter, or `ss` output)
Cgroup CPU	The backend cgroup's `cpu.stat`
PSI	The backend cgroup's `cpu.pressure`

So "Prequal-style" here means *Prequal's signal used inside a Unimog-style slow controller*, not Prequal's per-request probing. Phrase your claims that way. It's still an interesting comparison, but it tests the signal, not Prequal.

The scoping rule: each extra signal is about 20 lines of code (read a number, return it). Each extra signal in the *full sweep* multiplies experiment time. So:

- **Implement all of them** behind one plugin interface.
- **Run the full sweeps** for whole-core CPU vs RIF, plus the three static baselines.
- **Run the extras** (cgroup CPU, PSI, connection count) **on the one or two scenarios that matter most**, e.g. the antagonist case where cgroup CPU is predicted to fail. That keeps your main question intact and still gets the Prequal-trap result if it shows up.

Rough experiment budget:

- One capacity-at-SLO measurement is about 8 load steps × ~70 s ≈ 10 minutes.
- A full sweep of 5 disturbance periods × 5 designs × 2 connection lifetimes × 5 runs ≈ 250 measurements ≈ **40+ hours** of machine time.

That's doable on a single machine running overnight, but only if the harness is fully automated early. Cut where it doesn't change the picture: binary search instead of fixed steps, and fewer runs at the ends of the sweep where results converge.

2. Topology: dedicated LB vs LB on every server

The research question doesn't depend on where the LB runs. Bucket table, controller, signals, redirector and second hop are identical in both. Only the first forwarding step moves.

	Dedicated LB	LB on every server (Unimog)
Measurement validity	Clean: backend CPU is only backend work	**Confounded**: forwarding work runs on the same CPUs you're measuring and using as a signal
Complexity	One XDP attach point	ECMP router namespace + shared L2 for re-forwarding + a "first hop is me" path
Move to real laptops	Natural: LB on the native-XDP laptop	Each laptop is both LB and backend; harder
Second hop still needed?	**Yes**: controller table changes break connections without it	Yes
What you lose	The "LB instances come and go" story	n/a

The confound is the deciding factor. Your headline number is capacity, and your main signal is CPU. Putting forwarding work on the backends muddies both, for no benefit to the question you're asking.

A middle ground worth building in from day one: keep the LB stateless, so every LB instance just reads the controller's current table. Then **two dedicated LBs behind a multipath route in a router namespace** is a small optional extension. It gives you the "kill an LB, no connections break" demo (GLB/Beamer-style) without the complexity of running the LB on every server.

Write-up: say explicitly that colocating LB and backends is out of scope and why (it confounds the CPU signal). Reviewers respect a scoped-out choice with a reason.

Single-laptop setup:

- Pin each part to its own physical cores: load generator, LB, backends.
- Antagonists share cores with their target backends, on purpose.
- Watch out for hyperthread siblings: two "cores" that are siblings interfere.
- Check LB CPU in every run, so you can show the LB was never the bottleneck.

3. Scope cuts I'd make

- **TCP over IPv4 only**, one VIP, one forwarding table.
- **Two hops only** (like Unimog). No generation numbers.
- **Your own simple UDP encapsulation** carrying the second-hop address. Your TC redirector decapsulates it itself (like `cls-redirect`), so you don't need GUE compatibility or `fou` setup.
- **No automatic health checking.** Drain only on command. Mention Cloudflare's oscillation lesson as future work.
- **A backend app you write yourselves:** tunable CPU cost per request (including heavy-tailed costs), and it exports RIF. This is easier than instrumenting nginx, and it makes "per-connection cost spread" a single parameter.

4. Who does what

Person	Owns
A, data plane	XDP director, bucket table via map-in-map (atomic whole-table swap), encapsulation
B, backend side	TC redirector (decap, `bpf_skc_lookup_tcp`, second hop), backend app (cost model, RIF), tracepoint counter for broken connections

Person	Owns
C, control plane	Signal plugins, proportional controller (period, gain, dead band), bucket selection (swap first, then least-recently modified), table publishing
D, testbed + harness	Namespace/cgroup/antagonist scripts, open-loop load generation, capacity-at-SLO search, per-window p99 analysis, plotting

All four write. D's role isn't a side job: the harness decides whether you get results at all.

5. Milestones, built so you always have a finished project

I don't know your deadline, so these assume about 10–12 weeks. Stretch or compress to fit.

Miles tone	Weeks (approx.)	Deliverable	Already a complete result?
M1	1–2	Testbed, backend app, load generator; **capacity at SLO for each backend directly** (the denominator); a noise check (same configuration 5×)	Shows your measurements are trustworthy
M2	3–4	XDP director with static tables: equal, by cores, static oracle	Yes: "usable capacity vs. heterogeneity for static schemes"
M3	5–6	Redirector + second hop; drain and table changes with **zero broken connections**	Yes: the GLB-style result on its own
M4	7–8	Controller with whole-core CPU and RIF; tune stability	The main contribution
M5	9–10+	Sweeps, extra signals on the key scenarios, optional second LB, write-up	Full project

If M4 slips, M1–M3 is still a coherent, well-evaluated project ("static weighting plus connection-preserving table changes"). That safety net is the main reason this scope is reasonable.

6. Risks to check early

1. **Noise on one laptop.** If p99 varies ±30% between identical runs, you can't resolve differences between designs. Measure this in M1 *before* building the LB. If it's bad: longer measurement windows, isolated cores, frequency scaling fixed (`performance` governor), turbo disabled.
2. **The LB becoming the bottleneck.** Generic XDP on veths is slower than native. Keep the offered load in a range where LB CPU stays well below saturation, and report LB CPU.
3. **Redirector correctness.** Write the drain test first: long-lived connections plus a table change should give zero resets. TC lets you use `tcpdump`, which is exactly why Cloudflare chose it.
4. **Harness time.** If the capacity search isn't fully automated by the end of M2, the sweeps won't fit.

So the plan holds: main comparison whole-core CPU vs RIF, extras on the key scenarios, a dedicated stateless LB with an optional second instance, and M3 as the safety net.

Prompt 13

ok heres where weve landed after all this, can you check it and tell us what you would change before we get into the detailed design.

problem: on a cluster where servers have unequal and changing capacity, carry as much load as possible within a p99 deadline, without breaking live connections and without per connection state on the lb.

design:
- one dedicated stateless lb running xdp. it hashes the 4-tuple into a power of two bucket table (map in map so we can swap the whole table at once), each bucket has a current owner and a previous owner
- it encapsulates packets to the current owner with our own small udp header that carries the previous owners address, and backends reply directly to the client (dsr)
- a tc redirector on each backend: syn or local socket means keep it, otherwise forward to the previous owner
- a backend app we write ourselves with tunable cpu cost per request, which exports requests in flight
- a controller: proportional loop, swap first then least recently modified buckets, with signal plugins
- baselines: equal weights, static by cores, static oracle
- tcp over ipv4 only, single laptop with namespaces first

research questions:
1. how much of the ideal capacity does each policy reach at the slo (usable capacity fraction), with static and with changing capacity
2. under which disturbance timescales does the dynamic controller beat the static oracle, and does it stay stable
3. which signal works best: whole core cpu vs requests in flight as the main comparison, with cgroup cpu and psi on the key scenarios
4. does the second hop keep connections alive through table changes, and how often does a second bucket change break them

we probably have less time than 10-12 weeks so we want the milestones tighter. wed also like to build the core pieces first and design the benchmark harness properly once they work, though we get your point about checking noise early. whats a good order of work given that, and what would you change

Claude Code

This is a solid plan. The problem is precise, the design follows from it, and every research question maps to something you can measure. My changes are mostly correctness gaps in the data path that would surface during your capacity tests, plus a few definitions to fix now so the core pieces log what the harness will later need.

1. Problem statement: two small edits

> On a cluster where servers have unequal and changing capacity, **an L4 load balancer that can only steer new connections** should carry as much load as possible **within a latency and error SLO (p99 ≤ L, error rate including resets ≤ E)**, without breaking live connections and without per-connection state on the LB.

- **"Can only steer new connections"** is the constraint that makes the problem interesting. It's why the dynamic controller has limits, and it separates your work from Prequal.
- **Putting errors in the SLO** means broken connections automatically count against capacity, which ties RQ4 to RQ1.

2. Design: what I'd change, most important first

a. Stop forwarding loops. Put a "this is already the second hop" flag in your UDP header.

- The second-hop server must never forward again; if it has no socket, it hands the packet to its stack, which sends a RST.
- Also define an "empty second slot" value. With no second hop, the first hop handles the packet locally.

b. SYN cookies will look like broken connections under load. This is the subtle one.

- At high load, the backend's listen queue overflows and Linux falls back to SYN cookies (`tcp_syncookies=1` is the default).
- The handshake's final ACK then matches **no socket**, so your redirector forwards it to the second hop, and the connection fails.
- That happens exactly during your capacity-at-SLO steps, so it would show up as errors and skew both RQ1 and RQ4.
- **Fix:** for a non-SYN packet with no established socket, look up the listening socket and call `bpf_tcp_check_syncookie`. If the cookie is valid, keep the packet locally. At minimum, size the backlog large, and check `TcpExtSyncookiesSent` after every run.
- Use `bpf_skc_lookup_tcp`, not `bpf_sk_lookup_tcp`. The `skc` variant also returns half-open (request) and TIME_WAIT sockets, so a handshake whose bucket changed mid-way still gets kept by the server that saw the SYN.

c. Fill the second slots from the start, GLB-style.

- If the initial table has empty second slots, the controller has **no free swaps** available, and every early adjustment is a real reassignment.
- Initialise each bucket's second slot with the rendezvous-hash second choice (a different server).
- This is safe: a stray packet sent to a server that never owned the connection just gets a RST, the same as having no second hop.
- Also rename the slots to "first hop / second hop" rather than "current / previous owner". After a swap, "previous" isn't literally true, and the naming will confuse you during debugging.

d. Details of Direct Server Return and encapsulation inside namespaces. These are easy to get wrong:

- **On backends:** the VIP on `lo`, `arp_ignore=1`/`arp_announce=2`, `rp_filter` off on the receiving interfaces, and replies routed to the client directly, not back through the LB.
- **MTU:** raise the MTU on the internal veths by your header size (the equivalent of jumbo frames), or clamp TCP MSS on the client side. Otherwise full-size packets get silently dropped once encapsulated.
- **Second-hop forwarding:** rewrite the outer destination and use `bpf_redirect_neigh`, so the kernel fills in MAC addresses.

e. Define the workload model now, before any measurement.

- The LB steers *connections*, but capacity and RIF are in *requests*.
- Fix: the rate of new connections, requests per connection, the per-request cost distribution, and the connection lifetime.
- Offered load is then in requests/s, generated as connections × requests.
- Without this, results from different milestones won't be comparable.

f. Controller parameters to write down (not to tune yet): control period, how signals are smoothed, gain, dead band, and a cap on bucket changes per period. The cap gives you a direct knob for the RQ2/RQ4 trade-off between balancing faster and breaking more connections.

g. Define the static oracle precisely: weights proportional to each backend's capacity at SLO, measured directly with no disturbance. That's the same number as your denominator, so it costs nothing extra.

3. Research questions

- **Rank them,** because time is short. **RQ1 (static part) and RQ4 (first part) don't need the controller**, so they're your guaranteed results. RQ2, RQ3 and the second part of RQ4 depend on it.
- **RQ2: define "stable"** now, e.g. settling time after a step change in load, oscillation amplitude of the load spread, and bucket changes per minute.
- **RQ3: name the key scenario.** I'd use the antagonist-on-shared-cores scenario, where cgroup CPU is predicted to fail and PSI should succeed.
- **RQ4 is also a trade-off with RQ2.** Report broken connections against controller aggressiveness (the change cap from 2f), not just as one count.
- **Report overhead without making it a research question:** the fraction of packets that take the second hop, plus LB and redirector CPU.

4. Order of work: core first, harness later, without the risk

Your preference is fine with one adjustment: **design the harness later, but decide now what it needs logged.** If the core pieces log the right things from day one, the harness is just analysis. Otherwise you'll be retrofitting instrumentation in the final weeks.

Log from day one:

- per-request latency with timestamps, from the load generator;
- RIF from the backend app;
- every table publish (timestamp, which buckets changed, swap or reassignment);
- tracepoint resets;
- second-hop counters in a BPF map.

The noise check doesn't need the harness either. It's about a day with off-the-shelf tools.

Phase	Approx. time	Work	Exit check
0. Groundwork	2–3 days, everyone	Topology script (namespaces, veths, MTU, VIP/ARP/ `rp_filter`, DSR routing, core pinning, `performance` governor); backend app v0; **noise check:** `wrk2` **straight at one backend, same configuration 5×**	p99 run-to-run variation small enough to resolve your expected differences (aim for under about 10%)
1. Core pieces in parallel	~2 weeks	**A:** XDP director with a static map-in-map table and encapsulation. **B:** TC redirector, first just decap + keep locally, then the second hop + loop flag + syncookie check. **C:** controller *offline*: table generator (static weights, rendezvous second slots), publishing to map-in-map, signal plugins, tested against fake signals with no data plane. **D:** full backend app (cost distribution, RIF export), tracepoint counter, choice of load generator	End to end through the LB with a static table, plus the logs listed above
2. Correctness + static results	~1 week	Drain test: long-lived connections + a table change = **zero resets** (RQ4 part 1). D builds harness v1: automated capacity search; measure the denominator and the three static baselines	**First complete result:** RQ1 static part + RQ4 part 1
3. Closing the loop	~1 week	Controller live with whole-core CPU and RIF; step-change tests for stability. D: harness v2 with the scenario runner (antagonist schedules), per-window SLO analysis	Controller settles without oscillating in the step test

Phase	Approx. time	Work	Exit check
4. Experiments	1–2 weeks	Sweeps run overnight; extra signals on the antagonist scenario; double-change breakage vs the change cap (RQ4 part 2)	All planned figures exist
5. Write-up	~1 week	Plus buffer	n/a

That's about 6–7 weeks. To compress further:

- Cut the connection-lifetime axis to one setting.
- Use 3 disturbance timescales (fast, medium, slow) instead of 5.
- Do 3 runs instead of 5.
- Run the extra signals only on the antagonist scenario.
- Drop the optional second LB and the physical-laptop demo.

Cut line: if phase 3 slips, phases 0–2 alone are still a complete project: static weighting evaluated by usable capacity, plus connection-preserving table changes. Plan your report structure so that version could be submitted as-is.

Summary of changes

- **Problem statement:** add "can only steer new connections" and an explicit SLO with errors.
- **Design:**
  - loop-prevention flag and an empty second-slot value;
  - syncookie handling (plus `skc` lookup);
  - rendezvous-initialised second slots;
  - rename the slots to first/second hop;
  - get the Direct Server Return and MTU setup right;
  - fix the workload model;
  - add a cap on bucket changes per period.
- **Research questions:** rank them; define stability; name the key scenario; treat RQ4 as a trade-off curve.
- **Process:** a one-day noise check at the very start, log everything from day one, and build the harness after the core works.

Prompt 14

this is really useful, especially the syncookie thing, we would never have caught that. lets go with all of those changes and the phase plan. before building, lets work out the controller properly: how exactly do we turn the signal numbers into bucket changes each period? for example how do we compare cpu across backends with different core counts, or rif, what gain, dead band and change cap do we start from, and how do we pick which buckets to swap vs reassign so it doesnt flip flop between backends

Claude Code

Here's a concrete design you can start from. It has four stages: **normalise each signal into "load relative to capacity"**, **compute target bucket counts with a damped proportional rule**, **turn targets into bucket moves (free swaps first, then the safest reassignments)**, and **stop flip-flopping with dead bands, dwell**

times and caps. All the starting values are meant to be tuned once in a short step test. That tuning run also feeds RQ2.

1. Normalise every signal to the same scale

Make each plugin return **u_i = load on backend i relative to its capacity**, where about 1.0 means saturated. The controller then just **equalises u_i**. Equal relative load means every backend hits its limit at the same time, which is exactly what maximises usable capacity.

Read cumulative counters and take the difference over the period, don't sample instantaneous values. That gives exact averages over the period with no sampling noise.

Signal	Raw counter	Normalisation → u_i
Whole-core CPU	Busy time of the backend's cores (`/proc/stat`), so antagonists on those cores count	Δbusy / ($cores_i$ × P)
Requests in flight	**Have the app export ∫RIF dt** (accumulated request-seconds, updated at each request start and end)	(Δ∫RIF / P) / $cores_i$
Cgroup CPU	`cpu.stat` `usage_usec`	Δusage / ($allocation_i$ × P), where the allocation is the `cpu.max` quota, or the cpuset size if there's no quota
PSI	`cpu.pressure` `some total=` (µs)	Δtotal / P, the fraction of time some task waited for CPU
Connections (extra)	Established sockets	count / $cores_i$

Why RIF/cores is comparable to CPU utilisation (Little's law): RIF = throughput × time in system. For a CPU-bound app, that's roughly (fraction of cores busy) + (queued requests / cores). So below saturation RIF/cores tracks utilisation, and near saturation it rises faster as the queue builds. That's what makes it a leading signal. It also automatically reflects slower cores and antagonists, because both stretch service time.

PSI is a different kind of signal. It stays near 0 until there's contention, then rises. At low load it carries no information, and that's fine, because the low-load gate (below) keeps the controller idle there anyway. Use the `total=` counter yourselves rather than `avg10` ; `avg10` is a 10-second moving average, which adds lag.

A validity issue this exposed: how you emulate "unequal servers"

- **Fewer cores:** a smaller cpuset. Fine for every signal.
- **Slower cores (hardware generations): don't use `cpu.max` for this.**
  - A backend on 2 cores with a 1.2-core quota is saturated while its cores look 60% idle. Whole-core CPU would be blind to it.
  - That's an artefact of the emulation; real slow cores show 100% busy when saturated.
  - Instead, give the backend app a **per-backend slowness factor** (more CPU work per request). That's the faithful equivalent of an older CPU generation.
- **VM-style allocations (Prequal's scenario):** that's where `cpu.max` belongs. There, the gap between cgroup CPU and the true contention is the point of the experiment.

2. From signals to target bucket counts

Bucket share decides the share of new connections, so a backend's load scales roughly with its bucket count. To move u_i towards the target ū, scale its bucket count by ū/u_i, damped by a gain:

```
ū       = Σ c_i·u_i / Σ c_i          (capacity-weighted mean = what every u would be if balanced; c_i = cores)
target_i = n_i · (ū / max(u_i, U_MIN)) ^ GAIN     if |u_i - ū| > dead band, else n_i
```

```
then:   clamp each change to ±MAX_STEP·nᵢ, apply FLOOR, rescale so Σ = NBUCKETS,
        round with largest-remainder so the total stays exact
```

Gates:

- If ū < LOW_LOAD (e.g. 0.15), **do nothing**. Imbalance doesn't threaten the SLO at low load, and the signals are mostly noise there.
- Each backend keeps at least FLOOR buckets unless it's drained. A backend with zero buckets reads u ≈ 0, gets flooded with buckets on the next period, and starts an oscillation.

The key stability fact for an L4 controller: a bucket change only affects *new* connections. Load shifts over roughly one connection lifetime τ. If the period P is much shorter than τ, the controller doesn't yet see the effect of its last change and keeps pushing, which causes overshoot and oscillation. The effective gain per period is about GAIN × (1 − e^(−P/τ)). So:

- **choose P ≥ τ** (the typical connection lifetime), or
- **lower GAIN for long-lived connections.**

This is also why your connection-lifetime axis matters for RQ2.

3. Turning targets into moves

Donors have more buckets than their target; receivers have fewer. Each move takes one bucket from a donor and gives it to a receiver. Keep per-bucket metadata: `last_change` time and `second_since` (when the current second-hop server got there).

Step 1, free swaps. For donor d and receiver r, find buckets `[d | r]` and swap them to `[r | d]`.

- This breaks nothing: d's existing connections arrive at r, find no socket, and go on to d.
- Pair the largest surpluses with the largest deficits, but prefer pairs that have swappable buckets. Keep a count of buckets for each (first, second) pair so lookups are cheap.

Step 2, reassignments, safest first. For the remaining need, take a bucket `[d | x]` and make it `[r | d]`.

- x's connections in that bucket lose their second hop and **break**. These are connections that already survived one change.
- So choose buckets **with an empty second slot first, then the oldest** `second_since`. x's leftover connections in those buckets are the most likely to have already ended. This is the precise version of Cloudflare's "least-recently modified" rule.
- The old first hop becomes the new second hop. That also creates a future free swap back to d if needed.

Log every change: time, bucket, old and new slots, swap vs reassign. RQ4's "second change broke it" analysis is a join between this log and your tracepoint resets.

4. Stopping flip-flops

These work at different levels, so use all of them:

1. **Hysteresis dead band.** A backend starts moving only when $|u_i - \bar{u}| > \delta_{on}$, and keeps moving until it's within $\delta_{off} < \delta_{on}$. Noise around the threshold then doesn't cause back-and-forth.
2. **Dwell time per bucket.** A bucket changed recently isn't eligible again. This stops one bucket bouncing between two backends.
   - Short for swaps (they're harmless; this just prevents churn).
   - Long for reassignments (≥ τ, to protect connections).
3. **Change caps.** A global cap per period and a per-backend cap on the fraction of its buckets that can change. Large errors are corrected over several periods instead of in one jump.

4. **Gain < 1 and P ≥ τ**, from section 2.
5. **Low-load gate and floor**, from section 2.

5. Starting values

Parameter	Start at	Reasoning / how to refine
NBUCKETS	4096	More than 100× backends (Unimog's rule); with ≤8 backends each bucket is about 0.025% of traffic
Initial table	Static-by-cores weights; second slots from the rendezvous second choice	A sensible starting point; gives free swaps from the start
P (period)	2–5 s, and **≥ the typical connection lifetime**	Tune in the step test
GAIN	0.3	Sweep 0.1 / 0.3 / 0.6 / 1.0
δ_on / δ_off	**2× / 1× the noise level** of u_i (standard deviation per backend under static-oracle weights at steady state); start with 0.05 / 0.02 if you have nothing yet	Measure the noise in phase 2; this ties the dead band to data instead of a guess
U_MIN	0.05	Avoids dividing by near-zero
LOW_LOAD	0.15	n/a
Global change cap	2% of buckets per period (about 80)	It's also your RQ4 aggressiveness knob
Per-backend step	≤ 20% of its buckets per period	n/a
FLOOR	1% of buckets per active backend	Keeps every backend producing a signal
Swap dwell	2 periods	n/a
Reassign dwell	max(3 periods, p90 connection lifetime)	Protects connections from a second change

6. Pseudocode (one control period)

```
u = {i: plugin.read(i) for i in backends}            # normalised, ~1.0 = saturated
u_bar = sum(cores[i] * u[i] for i in backends) / sum(cores.values())
if u_bar < LOW_LOAD:
    return

target = {}
for i in backends:
    err = u[i] - u_bar
    moving[i] = abs(err) > (DELTA_OFF if moving[i] else DELTA_ON)   # hysteresis
    if not moving[i]:
        target[i] = n[i]
        continue
    t = n[i] * (u_bar / max(u[i], U_MIN)) ** GAIN
```

```
    target[i] = clamp(t, n[i] * (1 - MAX_STEP), n[i] * (1 + MAX_STEP))
target = integerize(rescale(target, NBUCKETS, floor=FLOOR))       # largest remainder

donors    = {i: n[i] - target[i] for i in backends if n[i] > target[i]}
receivers = {j: target[j] - n[j] for j in backends if target[j] > n[j]}
budget = CHANGE_CAP

for d, r in pairs_by_need(donors, receivers):                     # 1. free swaps
    for b in buckets(first=d, second=r, idle_for=SWAP_DWELL):
        if not (budget and donors[d] and receivers[r]):
            break
        table[b] = (r, d)
        log(b, "swap")
        donors[d] -= 1; receivers[r] -= 1; budget -= 1

for d, r in pairs_by_need(donors, receivers):                     # 2. safest reassignments
    for b in sorted(buckets(first=d, idle_for=REASSIGN_DWELL),
                    key=lambda b: (table[b].second is not None, second_since[b])):
        if not (budget and donors[d] and receivers[r]):
            break
        table[b] = (r, d)                                         # old second-hop dropped
        log(b, "reassign")
        donors[d] -= 1; receivers[r] -= 1; budget -= 1

if changed:
    publish(table)                                                # map-in-map swap
```

7. Tuning and checking it, before and with the data plane

Offline simulator (phase 1, person C, no data plane needed):

- A simple model:
  - new connections arrive at rate λ and are split by bucket share;
  - each lasts about τ;
  - backend load = active connections × cost / capacity$_i$, plus noise and scripted antagonists.
- Run the real controller code against it. You'll find gain/period instabilities in minutes instead of testbed hours, and the simulator doubles as your unit-test bed.

Step test on the testbed (phase 3):

1. Start balanced under static-oracle weights at about 70% load.
2. At t = 60 s, turn an antagonist on for one backend; at t = 180 s, turn it off.
3. Measure:
   - **settling time** (until max$|u_i - \bar{u}| < \delta_off$ and it stays there);
   - **overshoot**;
   - **bucket changes, split into swaps and reassignments**;
   - **resets**.
4. A small grid of GAIN {0.1, 0.3, 0.6, 1.0} × P {1, 2, 5, 10 s} at about 4 minutes per run takes about an hour. Pick the defaults from it and include the stability map in the report as RQ2 evidence.

8. One caveat that may become a finding

Equalising *utilisation* maximises *throughput*. Your SLO, though, is a *latency* deadline. A backend with slower cores has longer service times, so it reaches the p99 deadline at a **lower** utilisation than a fast one. So when servers differ in core speed, equalising CPU utilisation won't exactly maximise capacity at the SLO.

RIF partly accounts for this, because slower service raises RIF. Whole-core CPU doesn't. If your slowness-factor experiments show CPU-equalising losing usable capacity while RIF holds up, that's a meaningful, explainable result for RQ3, not a bug. Mention it in your hypotheses up front so it reads as a prediction, not a rationalisation.

Prompt 15

the offline simulator idea is great and the slowness factor point makes sense, well use that instead of cpu.max for slower cores and put the section 8 caveat into our hypotheses. next the data path details. what should our udp encapsulation header look like (fields, the second hop address, the loop flag, port), and how does the tc redirector decap it, bpf_skb_adjust_room or something else? also for the lb itself, xdp vs tc given it will often run in generic mode, and what changes between the two in terms of helpers and how we build the outer headers

Claude Code

Short version:

- **Header:** a 12-byte custom header inside a plain IPv4/UDP envelope, carrying the second-hop address, a hop flag, the bucket number and the table generation.
- **Redirector:** TC ingress program. Decide using the inner headers, then either remove the envelope with `bpf_skb_adjust_room` (keep locally) or rewrite the outer addresses and send it on with `bpf_redirect_neigh` (second hop).
- **LB:** write it in XDP. One correction to your premise, though: on your namespace testbed **it doesn't have to run in generic mode.** veth supports native XDP. Generic mode is only a fallback for odd interfaces (USB or Wi-Fi NICs), and the native-XDP laptop will run the LB in any physical demo.

1. The encapsulation format

```
outer:  Ethernet | IPv4 (src = LB IP, dst = first-hop backend, proto UDP)
                 | UDP (src = per-flow hash, dst = ENCAP_PORT, checksum = 0)
                 | encap header (12 B)
inner:  original IPv4 packet, unmodified (IP + TCP + payload)
```

```
#define ENCAP_PORT   9999      /* any fixed unused port, same on LB and all backends */
#define ENCAP_VER    1
#define F_HOPPED     0x01      /* already on its second hop: never forward again */

struct encap_hdr {
    __u8   version;            /* ENCAP_VER; drop anything else */
    __u8   flags;              /* F_HOPPED */
    __be16 bucket;             /* bucket index: debugging + RQ4 attribution */
    __be32 second_hop;         /* IPv4 of second-hop backend, 0 = none */
    __be32 table_gen;          /* table generation used; logging only */
};                             /* 12 bytes, naturally aligned */
```

Design decisions in this format:

- **Overhead:** 20 (IP) + 8 (UDP) + 12 = **40 bytes**.
  - Set the MTU on the LB↔backend links to 1600; keep the client side at 1500.
  - Replies go directly to the client and never get encapsulated, so they follow the client's normal MSS.
- **UDP checksum = 0.** That's legal in IPv4 and means you never checksum the payload in BPF. The inner TCP checksum is untouched because you never modify the inner packet. You only compute the 20-byte outer IP header checksum.
- **UDP source port = hash of the inner 4-tuple**, folded into 49152–65535. Packets of one flow stay in order, and a receiving NIC with multiple queues spreads flows across them (this matters on real hardware later). This is the same trick GLB uses.
- **Carry `bucket` and `table_gen`** even though no forwarding decision uses them. When a reset shows up in your tracepoints, you can tell which bucket and which table version caused it. That's RQ4's evidence.
- **No ICMP or fragment handling.** List it as a known limitation. GLB has to handle ICMP "fragmentation needed" for path MTU discovery; your fixed MTUs avoid the need.

2. The LB in XDP

Topology:

- The router namespace routes the VIP *via the LB's IP* (`ip route add VIP via LB_IP`). So only the LB's own address ever needs ARP.
- Backends carry the VIP on `lo` with `arp_ignore=1` / `arp_announce=2`, so they never answer ARP for it.
- LB and backends share one L2 bridge. The LB sends packets back out the interface they arrived on (`XDP_TX`).
- With two LBs later, the same route becomes multipath.

Per packet:

1. Parse Ethernet / IPv4 / TCP. If it's not TCP to a VIP, return `XDP_PASS`.
2. **Hash the 4-tuple with your own `jhash` and a fixed seed.** Never use the kernel's flow hash (`skb→hash`): it's seeded randomly at boot, so two LBs, or one LB after a reboot, would disagree on buckets.
3. Look up the table. The outer `ARRAY_OF_MAPS` at index 0 points to the current inner array. Read `inner[hash & (NBUCKETS-1)]` → `{first, second}`.
   - Put `table_gen` in one extra slot at the end of the inner map, so it's published atomically with the table.
4. `bpf_xdp_adjust_head(ctx, -(int)ENCAP_LEN)` to add 40 bytes of headroom in front. **Reload `ctx→data` / `data_end` and bounds-check again.** All earlier pointers are invalid after this call.
5. Write the new Ethernet header at the new start, then outer IPv4, UDP and the encap header. The inner IP header stays where it was.
6. Compute the outer IP checksum: `bpf_csum_diff(0, 0, (__be32 *)iph, sizeof(*iph), 0)`, then fold to 16 bits. Or a small unrolled sum.
7. **MAC addresses:** `bpf_fib_lookup()` works in XDP and gives you source and destination MAC for `first`.
   - Have the setup script install **permanent neighbour entries** for all backends, so it never returns `NO_NEIGH`. Count failures anyway.
   - The alternative is a static map from backend IP to MAC, filled by the setup script. Simpler, but it breaks when you move to real hardware.
8. Return `XDP_TX`.

Counters (per-CPU array): packets passed, encapsulated, FIB failures, dropped because they'd be over MTU, and unknown VIP.

Gotchas:

- **Native XDP on veth:** frames sent with `XDP_TX` / `XDP_REDIRECT` are delivered into the peer veth's receive path. On many kernels that requires the **peer end** (the bridge-side veth) to have an XDP program attached (a trivial `XDP_PASS`), or GRO enabled on it. Otherwise the frames are silently dropped. Check your encapsulated count against what the backends actually receive in your first test.
- **Generic mode, if you ever need it:** it runs after the kernel has built its packet structure. If GRO has merged several TCP segments into one large packet, generic XDP may hand you that as one big frame, and once encapsulated it's over the MTU. **Turn GRO off on the LB's ingress interface in generic mode** (`ethtool -K <if> gro off`), and keep the over-MTU drop counter.
- **Workload shape helps here:** small requests coming in (through the LB), large responses going out (Direct Server Return). Encapsulated traffic stays small and you avoid most of these problems.
- **Debugging:** `tcpdump` doesn't see packets the LB sends back out with `XDP_TX`. Use `xdpdump` from xdp-tools, and check the result on the backend side, where TC lets ordinary `tcpdump` work.

3. The redirector on each backend (TC ingress)

Decision logic: look at the inner headers *before* changing anything, because the forwarding path doesn't need decapsulation.

```
SEC("tc")
int redirector(struct __sk_buff *skb)
{
    /* parse eth / outer IPv4 / UDP; not (UDP to ENCAP_PORT && outer dst == MY_DIP) → TC_ACT_OK
*/
    /* parse encap_hdr (version check), inner IPv4 + TCP (bounds checks, inner IHL) */

    if (eh→flags & F_HOPPED)             { count(KEEP_HOPPED);  goto keep; }  /* 2nd hop: never
re-forward */
    if (tcp→syn && !tcp→ack)             { count(KEEP_SYN);     goto keep; }

    struct bpf_sock_tuple t = { .ipv4 = { inner→saddr, inner→daddr, tcp→source, tcp→dest } };
    struct bpf_sock *sk = bpf_skc_lookup_tcp(skb, &t, sizeof(t.ipv4), BPF_F_CURRENT_NETNS, 0);
    if (sk) {
        if (sk→state ≠ BPF_TCP_LISTEN) {                    /* established, SYN_RECV, TIME_WAIT
*/
            bpf_sk_release(sk); count(KEEP_SOCKET); goto keep;
        }
        if (tcp→ack && !tcp→syn && !tcp→rst &&              /* listener found: maybe a SYN
cookie ACK */
            bpf_tcp_check_syncookie(sk, inner, sizeof(*inner), tcp, sizeof(*tcp)) == 0) {
            bpf_sk_release(sk); count(KEEP_COOKIE); goto keep;
        }
        bpf_sk_release(sk);
    }
    if (!eh→second_hop || eh→second_hop == MY_DIP) { count(KEEP_NO_SECOND); goto keep; }
    count(FORWARD);
    return forward_second_hop(skb, eh→second_hop);          /* below */

keep:
```

```
    return decap(skb);                                          /* below */
}
```

Two details in that code:

- `bpf_skc_lookup_tcp` finds established sockets, half-open handshakes (`SYN_RECV`) and `TIME_WAIT` sockets. If there's no connection it returns the *listener*, and that's exactly the moment for the SYN cookie check. **Every socket returned must be released**, or the verifier rejects the program.
- **A hopped packet with no socket** (`KEEP_HOPPED` while the lookup would fail) means the connection is about to be reset. Count it separately. It's the kernel-side counterpart to your tracepoint resets for RQ4.

Decap (keep locally):

```
static int decap(struct __sk_buff *skb)
{
    if (bpf_skb_adjust_room(skb, -(int)ENCAP_LEN, BPF_ADJ_ROOM_MAC, 0))
        return TC_ACT_SHOT;                    /* count it */
    return TC_ACT_OK;                          /* stack sees: eth | inner IPv4 to VIP → local
delivery */
}
```

- `BPF_ADJ_ROOM_MAC` with a negative length removes the 40 bytes **right after the Ethernet header**, which is exactly outer IP + UDP + encap header. The inner packet becomes the network layer.
- Because inner and outer are both IPv4, no extra flags are needed.
- For local delivery, the VIP must be on `lo`, and `rp_filter` must be 0 (or 2, loose) on the receiving interface.
- **Checks after every run:** `TcpInCsumErrors` stays at 0 (proves the checksum state survived decap), and `TcpExtSyncookiesSent` (shows whether the syncookie path was exercised).

Forward to the second hop:

```
static int forward_second_hop(struct __sk_buff *skb, __be32 dst)
{
    /* rewrite outer: saddr = MY_DIP, daddr = dst; set F_HOPPED in encap_hdr;
       fix outer IP checksum: bpf_l3_csum_replace() for the two address changes
       (or recompute); UDP checksum stays 0 */
    return bpf_redirect_neigh(skb→ifindex, NULL, 0, 0);   /* FIB + neighbour fill in MACs, send
out */
}
```

- `bpf_redirect_neigh` does the routing and neighbour lookup for the *new* outer destination, so the redirector needs no MAC table. That's a big convenience of TC. It returns `TC_ACT_REDIRECT` on success.
- Write with `bpf_skb_store_bytes`, or with direct packet access after `bpf_skb_pull_data(skb, needed_len)`. Re-load pointers after any helper that can change the packet.

Reference code worth reading: Cloudflare's `cls-redirect` in the kernel's BPF selftests (`tools/testing/selftests/bpf/progs/test_cls_redirect.c`). It solves the same problem for GUE.

4. XDP vs TC for the LB

	XDP	TC
Making room for headers	`bpf_xdp_adjust_head(ctx, -len)`	`bpf_skb_adjust_room(skb, len, BPF_ADJ_ROOM_MAC, BPF_F_ADJ_ROOM_ENCAP_L3_IPV4 | BPF_F_ADJ_ROOM_ENCAP_L4_UDP)`; the flags record where the inner headers start, so the kernel can still split large merged (GSO) packets correctly
Writing headers	Direct packet writes; reload pointers after the adjust	Direct access (after `bpf_skb_pull_data`) or `bpf_skb_store_bytes`
Checksums	Compute yourself / `bpf_csum_diff`	Same, plus incremental `bpf_l3_csum_replace`
MAC addresses	`bpf_fib_lookup` or your own map	`bpf_redirect_neigh` does it for you
Sending	`XDP_TX`, or `bpf_redirect`/devmap → `XDP_REDIRECT`	`bpf_redirect(ifindex, 0)` or `bpf_redirect_neigh`
Merged (GRO/GSO) packets	Native mode: single frames. Generic: may get one large packet → turn GRO off	Handled properly with the ENCAP flags
Socket lookup	`bpf_skc_lookup_tcp(ctx, …)`	Same helper
Map-in-map, per-CPU counters	Yes	Yes
`tcpdump`	Doesn't see what XDP sent; use `xdpdump`	Sees the original packet
Cost	Native: before skb allocation (cheapest). **Generic: about TC's cost, sometimes worse** (copying to make headroom)	Always after skb allocation

Recommendation:

- **Write the LB in XDP and run it in native mode on veth** (with the peer-end gotcha handled). It's the design you're studying, and it carries straight over to the native-XDP laptop.
- **Keep header building and parsing in shared inline functions** that work on `(data, data_end)` pointers. Both program types support direct packet access, so the same code compiles into a TC version.
- That makes a **TC version of the LB a one-day fallback** if veth native XDP misbehaves on someone's kernel. In generic mode the two cost about the same anyway, so you'd lose nothing that matters for your metrics.
- The **redirector stays in TC**, for the same reasons Cloudflare gave: `tcpdump` works, `bpf_redirect_neigh` exists, and most packets pass through untouched, so XDP's speed wouldn't help.

5. Build order and tests for the data path

1. **Golden-packet tests before any namespace.** Build packets with Scapy, run each program through `BPF_PROG_TEST_RUN` (libbpf `bpf_prog_test_run_opts`), and compare the output bytes against the

expected encapsulated or decapsulated packet. GitHub did the same with Scapy for GLB. This catches header-layout bugs in seconds.
2. LB encapsulation + redirector **decap-only** → an end-to-end TCP connection through the LB with Direct Server Return.
3. Add the second hop + `F_HOPPED` → the drain test (long-lived connections + a table change = zero resets).
4. Add the syncookie path → force cookies with a tiny backlog (`listen()` backlog 1, plus a burst of SYNs), and confirm `KEEP_COOKIE` increases while no connections break.
5. Only then hand it to the controller.

Checks after every run:

- counters add up (packets encapsulated ≈ packets received by backends);
- `TcpInCsumErrors` = 0;
- over-MTU drops = 0;
- `FIB fail` = 0.

Prompt 16

got it, native xdp on veth with an xdp_pass program on the peer, we actually ran into that gotcha in our old project. the golden packet tests with scapy are a great idea too. last thing before we build: the single laptop setup. this laptop is an i7-14650HX with p cores and e cores (24 threads). can we use p vs e cores as genuinely unequal backends, how should we pin the load generator, lb, backends and antagonists so they dont interfere, and how do we do the noise check you mentioned

Claude Code

Yes. P-cores vs E-cores is real heterogeneity, and it's arguably better than the synthetic slowness factor. They're different microarchitectures with different clocks, so the speed ratio depends on the workload, exactly like Cloudflare's hardware generations. It also makes your static-by-cores baseline wrong in a realistic way.

The two things to handle first are **turbo coupling** (backends affect each other's clock speeds) and the fact that **in a veth testbed, packet processing runs on whichever CPU sent the packet**, not where you pinned the LB.

1. P vs E as unequal backends

The i7-14650HX has **8 P-cores (with hyperthreading, 16 threads) and 8 E-cores (no hyperthreading)**. The E-cores sit in two clusters of 4, and each cluster shares an L2 cache.

Why it's good heterogeneity:

- The P/E speed ratio depends on your request type, so "cores" is the wrong unit, just like across real server generations.
- Static by cores will misjudge capacity, while the static oracle (measured) won't. That's the gap RQ1 is about.
- Keep the slowness factor as a second, controllable knob for sweeps.

The catch is turbo and power limits. With turbo on, a P-core's clock depends on how many other cores are busy and how hot the package is. That causes two problems:

1. **Noise.** Capacity drifts within and between runs.
2. **Coupling.** Loading backend A slows backend B. Your denominator (each backend measured alone) would then overstate capacity, and even a perfect LB would score below 1.0.

Fix: disable turbo for all experiments, and verify coupling is gone in the noise check (step 4 below). If it isn't, measure the denominator with all backends loaded at the same time.

2. Discover the real CPU numbering first

Hybrid CPUs aren't always numbered the way you'd guess. Check:

```
lscpu -e=CPU,CORE,CACHE,MAXMHZ                          # P threads show higher MAXMHZ
cat /sys/devices/cpu_core/cpus /sys/devices/cpu_atom/cpus   # P (core) vs E (atom) CPU lists
cat /sys/devices/system/cpu/cpu*/topology/thread_siblings_list   # hyperthread pairs
cat /sys/devices/system/cpu/cpu16/cache/index2/shared_cpu_list   # which E-cores share an L2
```

The plan below assumes the common layout: P-core threads 0–15 in sibling pairs (0,1), (2,3), … and E-cores 16–23, in clusters 16–19 and 20–23. **Substitute your real numbers.**

3. Where packet processing actually runs in a veth testbed

Sending into a veth processes the packet in **softirq context on the sending CPU**. Without intervention:

- The router's forwarding, the LB's XDP program, the bridge, *and* the backends' receive path for requests all run on the **load generator's** cores.
- The reply path runs on the backends' cores.

So pinning the LB process does nothing; there is no LB process.

Fix: use Receive Packet Steering (RPS) to move processing to the cores you choose. Its mask is a hex CPU bitmask per interface:

Interface (namespace)	RPS mask →	Effect
Router's client-facing veth (router ns)	**LB core**	Router forwarding runs there; the LB's veth processing (and so XDP) is scheduled on the same CPU, and the encapsulated packets continue from there
Each backend's veth (backend ns)	**That backend's cores**	Redirector + TCP receive run on the backend's own cores, as on a real server, so whole-core CPU includes the backend's networking
Client veth (client ns)	**Load generator cores**	Reply processing stays with the generator

```
# example: LB on CPU 6 → mask 0x40
ip netns exec rtr sh -c 'echo 40 > /sys/class/net/veth-cli/queues/rx-0/rps_cpus'
```

Verify it:

- Run `mpstat -P ALL 1` during a test: softirq time (`%soft`) should appear on the cores you chose.
- Check the LB's cost per packet with BPF runtime stats (`bpf_stats_enabled`, then `bpftool prog show`).

Make request cost dominate networking cost: about 0.5–2 ms of CPU per request keeps network processing a small fraction of each backend's CPU. Compare cgroup CPU (app only) with whole-core CPU to measure the remaining leakage.

4. Pinning plan (16 physical cores; P-core siblings left idle)

Role	CPUs (example)	Notes
Housekeeping: OS, desktop, controller, collectors, tracepoint reader, `turbostat`	0 (+ sibling 1)	Hardware interrupts here too

Role	CPUs (example)	Notes
Load generator (`wrk2 -t2`) + client receive (RPS)	2, 4	Must never be the bottleneck: keep it under about 50% busy, and check the achieved rate equals the target
Router + LB (RPS target)	6	One core is plenty for request-direction traffic
Backend P1	8, 10 (2 P-cores)	Siblings 9 and 11 idle
Backend P2	12, 14 (2 P-cores)	Siblings 13 and 15 idle
Backend E1	16–19 (whole E-cluster 0)	Don't split a cluster: its cores share L2
Backend E2	20–23 (whole E-cluster 1)	n/a
Antagonists	**Shared with the target backend's cores**, in their own cgroup	Contention on purpose

- **Never put two roles on the two threads of one P-core.** They share execution units and interfere in ways no counter shows.
- Either leave the siblings idle or take them offline (`echo 0 > /sys/devices/system/cpu/cpu9/online`) for less noise.
- Two P backends with 2 cores and two E backends with 4 cores gives static-by-cores a 1:1:2:2 split. Reality will differ, and that's your RQ1 gap.

Enforce it with cgroups (no reboot needed):

```
# confine everything else (incl. the desktop) to housekeeping
sudo systemctl set-property --runtime user.slice   AllowedCPUs=0,1
sudo systemctl set-property --runtime system.slice AllowedCPUs=0,1
sudo systemctl set-property --runtime init.scope   AllowedCPUs=0,1

# each experiment component in its own transient cgroup (gives you cpu.stat + cpu.pressure per
backend for free)
sudo systemd-run --scope --slice=exp.slice -p AllowedCPUs=8,10 \
     ip netns exec beP1 ./backend --slowness 1.0
sudo systemd-run --scope --slice=exp.slice -p AllowedCPUs=8,10 \
     stress-ng --cpu 2 --cpu-load 50          # antagonist: same cores, separate cgroup
```

Also stop `irqbalance` and point device interrupts at housekeeping (`echo 0-1 > /proc/irq/*/smp_affinity_list` ; some will refuse, which is fine).

A scenario this setup gives you for free (RQ3): an antagonist on the *idle sibling thread* of a P backend's core. It slows the backend's requests, but:

- it's not on the backend's CPUs, so whole-core CPU of those CPUs misses it;
- it's not in the backend's cgroup, so cgroup CPU misses it;
- nothing waits in the backend's run queue, so PSI misses it.

Only RIF sees it (service time goes up). It's realistic hyperthread interference and a clean test of the "signals that predict latency beat utilisation" hypothesis.

5. Frequency and power settings for every experiment

```
echo 1 | sudo tee /sys/devices/system/cpu/intel_pstate/no_turbo
sudo cpupower frequency-set -g performance
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/energy_performance_preference
# record during every run, keep with the results:
sudo turbostat --quiet --interval 5 --show CPU,Busy%,Bzy_MHz,PkgWatt,PkgTmp > turbostat.log
```

- Run on AC power, with the power profile on performance and the laptop raised for airflow.
- Ideally run experiments from a TTY, or at least close the browser.
- Use the `turbostat` log to **flag runs where clock speed dropped** (falling `Bzy_MHz` at constant load means thermal or power throttling). Exclude them, and say so in the report.

6. The noise check (phase 0, about a day)

Setup: one P backend, accessed directly with no LB, fixed request cost (e.g. 1 ms), `wrk2` at a constant rate (`-R`, which corrects for coordinated omission) with `--latency`, all the pinning and frequency settings above.

1. **Rough calibration:** ramp the rate quickly to find roughly where p99 crosses your SLO L. That's about 100% capacity.
2. **Repeatability at two load levels:** 50% and 80% of that capacity (80% is where SLO decisions are made, and where p99 is most sensitive).
   - **10 runs per level, alternating** (50, 80, 50, 80, …) so slow drift doesn't masquerade as a difference.
   - Each run: 20 s warm-up, then 60 s measured, then 10 s rest.
   - Record p50/p99/p99.9, achieved rate, and the `turbostat` log.
3. **Repeatability of the headline number itself:** run the full capacity-at-SLO search 5 times. **The spread of that capacity estimate is what decides which differences between policies you can claim.**
4. **Coupling and core type:** repeat step 3 on an E backend, and repeat the P backend while the other three backends run at about 70% load. If the capacity with neighbours busy is within the noise of the capacity alone, measuring each backend alone is valid for your denominator.
5. **Variation within a run:** compute p99 per 10-second window inside one run. Its variation tells you how strict your per-window SLO rule can reasonably be.

Pass criteria (suggested):

- p99 at 80% load: run-to-run coefficient of variation ≤ 5–10%.
- Capacity at SLO: coefficient of variation ≤ 2–3%.
- **Rule of thumb for your report:** don't claim differences between policies smaller than about twice the capacity spread you measure here.

If it fails, change one thing at a time and keep what helps:

- turbo off vs on (should already be off);
- siblings idle vs offline;
- desktop running vs TTY;
- 60 s vs 120 s measurement windows;
- higher per-request cost (less networking noise).

Write down the final configuration as your "measurement protocol" section. Reviewers like seeing that the noise was measured, not assumed.

Before the noise check, also check:

- the generator is never saturated (its CPU below 50%, achieved rate = target rate);

- no packet drops;
- `turbostat` shows a steady `Bzy_MHz` throughout.

After phase 2, rerun a shortened version through the LB to confirm the LB adds no significant noise of its own.

Prompt 17

perfect, the sibling-thread antagonist is a really nice scenario for rq3. lets start building with phase 0: set up the repo (git, a docs folder with decisions and a journal), the topology script with the pinning and rps from your plan, backend app v0, and then run the noise check. a few constraints: keep the code simple and readable since we need to explain every part. any system tweaks (turbo, governor, cpu confinement, irqbalance) should be runtime only, applied only during measurement runs, with a script that undoes them, because we keep vs code and other things open on this laptop between runs. and please put copies of the papers and blog posts into the repo too

Claude Code

I'll start by checking what this machine has: CPU topology, tools, and whether I can use sudo.

Now the shared config files, which are the single source of truth for the layout:

Now the backend app:

Now the open-loop load generator:

Several downloads need checking: the XDP paper came back as HTML, the three ~6 KB Medium/Twitter pages look like bot-block pages, and the Klein post timed out. Retrying:

Now the Python benchmark tools: shared helpers, the system monitor (replaces `turbostat` / `mpstat`), the noise-check orchestrator and its analysis.

The dispatcher's lag slice would also grow without limit in a long background run, so it gets the same treatment:

Now build everything and check syntax before touching the system:

Masks are correct (CPUs 8+10 → `0x500`, 16–19 → `0xf0000`). Next I'll bring up the topology and run its built-in check. This only creates network namespaces and changes no system settings:

Routing works and RPS puts the processing on the intended CPUs (single flow, so one CPU of each pair). The status line said `cli0 DOWN` even though pings pass, so I'll look at that more closely:

All links are up; `status` had just run before the link state settled. Next: calibrate, start one backend, and run a short load-generator test (still without system tweaks):

While the smoke test runs, I'll write the docs. First the decisions log, which records everything we settled in this conversation so the team can trace why the design looks the way it does:

While that runs (about 10 minutes), the rest of the docs: plan, measurement protocol, references index, README and `.gitignore`.

While the smoke test runs, I'll save two things worth remembering across sessions:

The smoke test is still running (about 10 minutes in total). I'll continue as soon as it finishes.

Now the shared work package and the antagonist tool:

The `local == 42` trick is clumsy. An atomic package-level sink is clearer, so I'll rewrite it that way:

Now switch the backend over to the shared package:

The flag variable `work` now shadows the `work` package inside `main`, so I'll rename it to `units`:

A safety net in the main cleanup (stop any antagonists), and removal of loadgen's now-unused `-discard`:

This diagnostic is worth keeping, since it's how the team can show service time no longer depends on load. I'll save it as a small script and run it:

The load generator has the same issue in miniature: its spinning dispatcher holds one of its two Go processors. It gets one spare as well:

The diagnostic run is still going (the E-backend half). I'll continue when it finishes; tweaks are undone automatically at the end of the run.

Rebuild, clear out the two stale smoke-test result folders (made before the fixes; their key numbers will go in the journal), and run the quick noise check again:

While it runs, I'll write the first journal entry, covering everything found today:

The journal entry is written. Waiting for the quick noise check to finish.

While the full run goes, a few doc touch-ups: the measured E/P ratio in H1, and a check of the repo state.

The full noise check is running (about 1 h 50 min). I'll write up the results when it finishes. Until then the laptop will feel slow: everything outside the experiment is confined to CPUs 0–1 and turbo is off. All of that is undone automatically at the end, and `sudo scripts/tune.sh undo` restores it by hand if needed.

Context limit reached. The session continued from an automatic summary.

If the experiment fails, the analysis shouldn't run inside `finally`; previously it ran only after success. I'll keep that and hand the files back in both places instead.

Phase 0 is done: the full noise check passed every limit, with plenty of margin. Afterwards the laptop was back to its normal settings and no test processes were left running.

Check	Result	Limit
p99 spread run to run, 80% load (10 runs)	CV 1.1% (4.83 ± 0.05 ms)	≤ 10%
Capacity spread, P backend (5 searches)	CV 0.8% (1630 ± 12 req/s)	≤ 3%
Capacity spread, E backend (5 searches)	CV 0.3% (1771 ± 5 req/s)	≤ 3%
P capacity when the other backends are busy	−0.2%	within noise
Thermal throttling	0 events	0

What this means for the project:

- **Smallest difference we can claim between policies: about 1.5%.** H1 predicts gaps of tens of percent between static-by-cores and the static oracle, so the testbed is precise enough for RQ1.
- **H1 holds so far.** Both backend types reach 82% of their theoretical capacity under the SLO. That makes the 4-E-core backend worth about 1.09× the 2-P-core one, as calibration predicted, not the 2× that core counts suggest.
- **We can measure each backend's capacity on its own.** Busy neighbours don't change it, so we don't need to load all backends at once to get the oracle's numbers.

Three things came up after the run, all written up in `docs/journal.md`:

1. **Quick runs overestimate capacity.** The quick check measured P/E capacity 3–5% higher than the full run, which is more than the noise. The likely cause is that 10 s measurements catch fewer slow requests than 40 s ones; we haven't tested that separately. I added a rule to `docs/measurement-protocol.md`: only compare capacities measured with the same settings, in the same session, with the configurations taking turns.

2. **The P cores looked slow, but weren't.** The summary showed them averaging 2010 MHz instead of the fixed 2200. I checked against the CPU's hardware cycle counters (APERF/MPERF): the clock stayed steady within ±0.2% over a minute, while the kernel's frequency reading jumped as low as 800. That reading isn't reliable here. The analysis now reports the median, which comes out at exactly 2200 / 1600 MHz.
3. **My first attempt at that check failed to undo its tweaks.** A `pkill` command in my cleanup step killed the script running the cleanup itself, so the tweaks stayed on (turbo off, desktop confined to CPUs 0–1). I saw it right away and undid them by hand. The journal records two lessons: stop test processes by their systemd unit name, and use `systemd-run` to put anything on the experiment CPUs while tweaks are on.

I also fixed `bench/noise_check.py`, which left `summary.md` owned by root, and marked phase 0 as done in `docs/plan.md`.

Nothing is committed yet. If you'd like, I can make an initial commit. It would include `references/` (20 MB) and the noise-check results (2.5 MB).

Prompt 18

nice, those numbers are much tighter than i expected, and the e backend ending up only 1.09x the p one is a good early sign for h1. the pkill thing is a good lesson too, stopping things by unit name from now on makes sense.

yes, commit it, but can we keep the pdfs out of git? maybe gitignore references/*.pdf and keep a references list with links and which part of the project each one is used for, 20mb of papers in the history seems unnecessary. the noise check results are fine to commit. and small commits from here on please, one per piece that works.

one thing i noticed while reading backend main.go: requests wait in the go channel and each worker is pinned to its own cpu, so when a backend is overloaded by requests the waiting happens inside our process, not in the kernel run queue. doesnt that mean psi cpu.pressure will never see a backend that is overloaded by requests, only one that is fighting an antagonist? if so the psi result in rq2 would mostly be about how we wrote the backend rather than about psi itself. should we add a second backend mode where the waiting happens in the kernel (like a thread per request, or more threads than cpus), or just document it as a limitation and explain it? whats your take, no need to build anything for it now if it belongs to a later phase.

also just making sure for phase 2: the loadgen needs connection churn (a new connection per request or every few requests), since with 64 persistent connections the lb only ever sees 64 flows and moving buckets cant move any load. i saw the workload model is already an open item in plan.md.

next, phase 1 part one: the xdp director with a static map in map table and our encapsulation, plus a small tool to load and configure it, and an end to end test through the lb with a static table. golden packet tests with bpf_prog_test_run from the start would be good too, so we can check the encap header byte by byte.

Claude Code

Now the PSI point. Before writing it down, I'll check what decisions.md already says about signals and the encapsulation header (I'll need the latter for phase 1).

The reference list now has links and says what each item is for. Next, D25 (PSI and where the backend queues) and the churn note, so the docs commit is complete.

Docs are in place. Now the commits, split by piece:

Committed in five pieces, with the PDFs and blog copies left out. Now phase 1. First, the toolchain:

The API has everything needed. Now the BPF side. The shared definitions go in one header, so the director and the backend-side decap (and later the redirector) all use the same layout:

I left a placeholder for the outer source address. It should be the LB's own IP, set by the loader like the VIP:

`IP_MF` / `IP_DF` / `IP_OFFSET` live in the kernel's internal headers, not the UAPI ones, so I'll define them in `lb.h`:

All three programs compile. Next is the Go side, `internal/lb`. It mirrors `lb.h` (constants, struct layouts, the hash), loads the programs, publishes tables, and reads the counters.

Now the golden packet. I'll compute the expected bytes with a separate Python jhash, so the literal in the test checks the C and Go versions against an implementation that shares no code with either:

All golden tests pass. That's the first working piece, so I'll commit it before writing the loader:

Next is `lbctl`, the tool that loads and configures everything. It needs `config.env` and `backends.conf`, so let me check their exact format:

Inline comments follow the values, so the parser strips anything after `#`. Next, a small Go config reader (the counterpart of `load_config` in `bench/testbed.py`) and `lbctl` itself:

Now the topology changes. The router needs a route sending the VIP to the LB, and `down` must remove the pinned objects:

Everything builds. First, a manual end-to-end run: testbed up, a backend in each namespace, load the LB, publish an equal table, then curl the VIP from the client.

I'll move that `metrics` helper into `testbed.py` (the e2e test needs it too) and let the loadgen helpers take a target IP, so they can aim at the VIP as well as at a backend:

Now the client checksum setting goes into `topology.sh`, with the reason, and then the end-to-end test script:

Before running it, I'll tear down the manual setup (backend units, LB, testbed) and check that `tb.run` passes the extra arguments through:

Now the decisions and journal entry for this piece:

The last commit message says D25–D28, but D25 already went in with the first docs commit. Since nothing is pushed, I'll correct the message:

Phase 1 part one works: the golden-packet tests and the end-to-end test through the LB both pass. Everything is committed in 8 small commits, and the testbed was left torn down with nothing loaded.

Repo and git

- `references/papers/` and `references/blogs/` are git-ignored. I left out the blog HTML/txt copies as well as the PDFs, for the same reason.
- `references/README.md` now lists every paper and post with its link and which phase or RQ uses it.
- The noise-check results are committed.

PSI. You're right, but it's RQ3 (signals), not RQ2. One addition: whole-core CPU and cgroup CPU have a related blind spot. Both stop at 100%, so they can't tell "slightly over capacity" from "far over". Only RIF measures the queue.

My take is to build the second mode rather than only document the limitation. Where a server queues is a real design difference: nginx, Envoy and thread pools queue inside the process, while thread-per-request servers queue in the kernel. Testing both turns "PSI fails" into "PSI only sees overload when the server queues in the kernel", which is a more useful result. The mode is small: many more unpinned workers than CPUs on the same FIFO queue. It belongs to phase 4 and is only needed for the PSI runs. Signals should be compared within one mode and capacities never across modes, because time-slicing changes the latency distribution. I recorded this as D25.

Connection churn. Yes, it's needed, and `loadgen -reqs-per-conn 1` already does it; the e2e test uses it. There's a catch for phase 2 that I added to the open items in `plan.md`. The client closes first, so every connection leaves a 60 s TIME_WAIT. At 2000 connections/s that's 120k sockets against about 28k ephemeral ports, and new connections start failing after about 15 s. The fix is two settings inside the `cli` namespace only (`tcp_tw_reuse=1` and a wider `ip_local_port_range`), so it doesn't touch the host.

What's built

- `bpf/director.c`: the XDP director. It hashes each packet to one of 4096 buckets, looks up the hops in the map-in-map table, adds our 40-byte encapsulation and sends it with XDP_TX.
- `bpf/decap.c`: a minimal backend-side decap, so end-to-end works now. The redirector replaces it in part two.
- `cmd/lbctl` with `load | table equal|NAME=W,... | show | stats | unload`. The programs stay attached after it exits.
- `make test-bpf`: 7 golden tests run in the kernel with `BPF_PROG_TEST_RUN`.
  - The main one checks a SYN's encapsulation byte by byte against bytes computed by a separate Python implementation of the hash and checksums.
  - A mismatch is reported by field name, e.g. `udp.sport` or `hdr.bucket`.
  - 2000 random flows confirm the C hash and its Go copy pick the same buckets.
- `bench/e2e_lb.py`, run with sudo. Results:
  - The director is attached in native XDP mode.
  - Without a table, VIP traffic is dropped and counted.
  - With an equal table, backends got 24.7 / 25.3 / 25.9 / 24.1% of about 4000 fresh connections.
  - With `beP1=1`, beP1 got 100%. With `beP1=1,beE1=3`, the split was 24.4 / 75.6%.
  - Zero errors. The LB received 79k VIP packets and only 4 others, so replies bypass it.

First end-to-end try failed: the backends rejected every packet as a checksum error. A veth leaves TCP checksums unfilled by default and marks the packet as "checksum still to do", and that mark doesn't survive XDP_TX. I fixed it by having the client compute its own checksums, as a real NIC would (D26). Fixing checksums in the director instead would add work to every packet that a real LB never does. D27 and D28 cover the other director and loader choices.

Next is part two: the TC redirector (socket lookup, second hop, the `F_HOPPED` loop flag, the syncookie check), the reset counter, the table generator with second hops, and the offline controller and simulator.

Prompt 19

great, and fair point on psi being rq3 not rq2, my mistake. building the second backend mode in phase 4 sounds right, and i like how it turns "psi fails" into a statement about where servers queue. the time_wait thing is a good catch too, keeping those sysctls inside the cli namespace only is fine.

i read director.c and lb.h, its really readable. one small thing: inner_len is taken from

data_end, so on a real nic a short frame that got ethernet padding (like a bare ack, 54 bytes padded to 60) would carry the padding inside the outer packet. its harmless after decap since the kernel trims to the inner tot_len, but using ip->tot_len (and checking it against data_end) seems more correct, and we will run on real laptops later. maybe add a golden test with a padded frame.

for part two id like to keep it focused, the controller and simulator can be the next step after this. so: the tc redirector replacing decap (decap first, then socket lookup, keep it on a syn or when we have a socket, otherwise forward to the second hop with the F_HOPPED flag set so it never goes a third time), the syncookie case, and the reset counter with the tracepoints. the table generator only needs enough second hop logic for testing here.

then the test i care most about for rq4: start long lived connections (say a few hundred, sending a request every 100 ms or so), switch to a table that moves a lot of buckets to different first hops, and count broken connections and resets, once with second hops filled in and once with them empty. if the design works it should be close to zero vs lots. could you also force the syncookie path in one backend (tcp_syncookies=2 in its namespace) and check that those connections survive too?

Claude Code

Plan for part two:

1. Fix the padding issue in the director.
2. Write the TC redirector and its golden tests.
3. Add the reset monitor and the table logic for second hops.
4. Run the move test.

First, what the tcp reset tracepoints give us on this kernel:

The director now uses `tot_len` and trims any padding before encapsulating. The padded-ACK golden test goes in with the redirector tests below. Now the redirector, which replaces `decap.c`:

Now the reset monitor's BPF side. It's two tracepoint programs that write one event per reset into a ring buffer:

Both compile. Now the expected bytes for the redirector's forward case, again from the independent Python code. Backend 1 (10.1.0.11) forwards a non-SYN packet to its second hop, 10.1.0.13:

Now the Go side: loading the redirector and the second-hop table logic in `lb.go`.

Slicing the hex string in half is fragile, so I'll give the inner IPv4 header its own constant:

I left some junk in `TestRedirectorKeeps` (unused variables from a first idea). Rewriting it cleanly, with the socket cases added:

A test-helper bug: `encapsulated` calls `As4()` on the "no second hop" address before checking that it's valid. Fixing it:

Still all passing with the redirectors in place. Committing this piece:

A quick plumbing check: connecting to a closed loopback port makes the kernel send a reset and the client receive one.

Both events show up with the right ports. One small thing: with no VIP involved, every reset is labelled "client", which is fine for our traffic. Committing:

`move_test.py` calls `tb.give_back`, which so far lives only in `noise_check.py`. I'll move it into `testbed.py` for both scripts to use, and rename a loop variable that shadowed the run name:

The fields exist under those names, so CO-RE can find them. Rewriting the send side of `resets.c`:

Before rerunning, I'll add the client namespace's own `TcpOutRsts` to the results. That will show whether the "client sent" resets in run B are real.

`resetmon` didn't start, and the script swallowed its error. Running it directly:

`tp_btf` programs attach with `link.AttachTracing`, not `link.Tracepoint`. Fixing that, and making the test fail loudly if `resetmon` dies:

It loads and attaches; the loopback resets are now filtered because they don't involve the VIP. The move test again:

Both retransmitting flows here came from beP2, which matches beP2's 10 retransmissions. Now the same breakdown for the full test's run B:

Now the journal entry:

Final checks before committing: vet, unit tests and the in-kernel golden tests.

Part two is done and committed in 7 small commits. On your main test: **with second hops, a table change that moved half the buckets broke 0 of 300 long-lived connections. Without second hops it broke 146.** The forced-SYN-cookie backend's connections also survive, and switching the cookie check off shows why it's needed. The testbed was left torn down.

Padding fix. You were right. The director now takes the inner length from `tot_len`, cuts any Ethernet padding with `bpf_xdp_adjust_tail` before encapsulating, and drops packets whose `tot_len` is longer than the frame. There are two new golden tests: a bare ACK padded from 54 to 60 bytes, and an oversized `tot_len`.

What's built

- **Redirector** (replaces `decap.c`). It keeps a packet if any of these hold: `F_HOPPED` is already set, there's no second hop, it's a SYN, the socket lookup finds a connection, or only the listener matches but the ACK carries a valid SYN cookie. Otherwise it forwards the packet once to the second hop with `F_HOPPED` set.
  - The golden tests open real loopback connections, because the socket lookup runs in the test's own namespace. The same ACK is kept when an established socket matches and forwarded when only the listener does.
  - All 15 in-kernel tests pass.
- **Second hops for testing.** `lbctl table` gives each bucket whose first hop changes the old first hop as its second hop. `-second none` publishes the table without second hops.
- **Reset monitor.** `cmd/resetmon` records every reset on a connection to the VIP, with its bucket. Which backend sent it comes from each namespace's own `TcpOutRsts`.

Move test (`bench/move_test.py`). 300 long-lived connections each send a request about every 100 ms, alongside 200 new connections/s. beE1 runs with `tcp_syncookies=2`. After 12 s the table switches to one without beP1, which moves 2047 of 4096 buckets.

Run	Long-lived broken (of 300)	New connections broken (of ~6000)	Backend resets, all in moved buckets
A: second hops	**0**	**0**	0
B: no second hops	**146**	0	192

Run	Long-lived broken (of 300)	New connections broken (of ~6000)	Backend resets, all in moved buckets
C: second hops, cookie check off	0	**671**	1290

The cookie connections survive in A: beE1 kept 598 handshake-completing ACKs only because their cookie was valid. Run C shows what happens without that check. New connections to beE1 in buckets that moved *to* it break: their last handshake ACK finds only the listener and gets forwarded to the old first hop, which resets it. beE1 sent 1971 cookies and got 1300 back, and the missing 671 are exactly the broken connections. It happens with no overload at all.

Two things the test turned up

- **My first reset monitor missed the important resets.** When a packet hits only the listener, the standard tracepoint reports the listener's own address (`[::]:8080`), not the connection. It reported 0 resets while the kernel counters said 1214. I fixed it by reading the addresses from the packet being answered. It now matches `TcpOutRsts` exactly.
- **In run B the client also sends resets later.** I first guessed these answered the old backends' retransmissions, but the counters ruled that out (14 retransmissions for 123 client resets). Packet captures and counters showed what happens:
  - Each broken connection leaves an orphaned socket on its old backend.
  - It wakes up either after 15 s idle, for Go's default keepalive probe, or for a retransmission when the client's last ACK was already redirected. Run B had 146 keepalive probes, one per broken connection.
  - The client answers with a reset, but the reset goes to the *new* first hop, so the orphan keeps probing for minutes.
  - So for RQ4, we count broken connections as distinct client sockets that received a reset, not total resets. That count equals the loadgen's errors in every run.

Details are in D29–D31 and the journal. Next on the plan is the controller offline: a table generator with rendezvous second hops, the signal plugins and the simulator.

Prompt 20

0 vs 146 is exactly the result we hoped for, and run c is a really nice extra, it shows the cookie check isnt optional even without overload. the orphaned socket and keepalive story was interesting to read too, and counting distinct client sockets that got a reset makes sense as the rq4 metric.

next: the controller, offline first like you said. the parts id expect:
- the table generator with rendezvous second hops for the initial table, and the bookkeeping per bucket (when it was last changed)
- the update rule from our plan: normalise each backends signal to load relative to capacity, aim everyone at the capacity weighted mean, target_i = n_i * (mean/u_i)^gain with a clamp and a floor, largest remainder rounding, swaps first and then reassign buckets whose second slot is empty or oldest
- hysteresis and dwell times, a cap on buckets changed per period (globally and per backend), and the low load gate so it doesnt react to noise when everything is idle
- signal plugins behind one interface (whole core cpu, rif, cgroup cpu, psi, and fake ones for testing), with publishing through the same map in map swap lbctl uses
- a simulator with backends of different capacities to check it converges, doesnt oscillate, and how fast

one thing i want the simulator to report: how many buckets get their first hop changed again before the connections from their previous change have ended (within the connection lifetime). with only two hops those are the ones where connections would actually break, so it would be nice if the controller makes that impossible by design (never touch a bucket again within that time) rather than just unlikely, and the simulator proves it. also if cpu saturating at 100% makes the cpu controller converge slower or get stuck when a backend is overloaded, id like the simulator to show that too, since thats part of what rq3 is about.

Claude Code

Before writing the controller, two things to check: how `testbed.py` names the backend cgroups (the cgroup-CPU and PSI signals read files there), and what the plan says about controller defaults.

The p99 model checks out against the M/M/1 closed form (9.21 ms). Now the reworked analysis in `ctlsim`, plus the short-dwell control run:

Now the helpers it uses: `InNetns` (moved out of lbctl), a CPU-list parser, and the in-namespace metrics fetch.

No response requested.

Prompt 21

sorry, my laptop ran out of battery while you were working on the controller, so your last turn got cut off and the machine rebooted (the testbed and the tune settings are gone too). the controller, signals and simulator code is still there uncommitted, along with the smoke test and sim results. can you check where you were, finish that step and commit it in small pieces like before, and tell me what the simulator showed?

after that, one design change we want to make on purpose. earlier we asked about xdp vs tc for the lb and you recommended xdp partly because our native xdp laptop would run it in a physical demo. we checked and that was wrong on our side: none of our laptops can do native xdp (mine only has wifi, no ethernet port), and raspberry pis arent going to happen. so on real hardware the lb would always run in generic xdp, which you said costs about the same as tc but brings the gro/mtu problem and tcpdump not seeing what it sends. so we want the lb in tc as a deliberate decision, not a fallback:
- port the director to a tc program (you kept the header code shareable for this), with bpf_skb_adjust_room and the encap flags, and update the golden tests and the end to end test
- write a decision entry that replaces d9 and explains the reasoning, including what tc costs us compared to native xdp
- all experiments from now on use the tc lb. keep the xdp version only if its cheap to keep around, for one small side measurement of per packet cost between the two hooks on veth, since we also want to be able to explain the difference between hook points properly
- check whether the client checksum workaround (d26) is still needed with tc, since the skb survives now

Claude Code

I'll start by checking what survived the reboot: the uncommitted files, whether everything still builds and passes, and that the reboot really cleared the testbed and tuning.

It works. Now the docs for the controller step:

Both compile. Now the Go side: `LoadDirector` takes the hook, and the golden tests run once per hook. The current test file:

I'll rewrite the director tests so each one runs once per hook. The expected bytes stay the same for both; only the return codes differ (`TC_ACT_REDIRECT` vs `XDP_TX`, and so on).

The kernel's test runner refused one of the inputs outright (`invalid argument`), before the program even ran. The cases run in random map order, so I'll find out which input it was:

That XDP check came out convoluted. I'll simplify it: `bpftool` must list the TC program, and `ip link` must show no XDP program. Also fixing the docstring:

A last sweep for stale XDP references in live code and configs (journal history stays as written):

Both parts are done and committed: the interrupted controller step in 7 commits, and the TC port in 4. Nothing was lost in the reboot, and the tuning and testbed were already gone as designed. Everything is torn down again now.

What the simulator showed

It ran four backends of unequal capacity, five seeds per case, judging results by p99 latency as well as utilisation imbalance.

- **Safety, which you wanted guaranteed by design:** 0 re-changes within the connection lifetime and 0 broken connections, over about 50 million simulated connections. That includes heavy-tailed lifetimes. The controller only touches buckets whose first hop hasn't changed for at least `Dwell`, so a move can never strand a connection unless one outlives `Dwell`.
  - To show the check isn't vacuous, a control run removes every protection (no dwell, no caps, flapping capacity). It gets 99k–288k re-changes and 27–824 broken connections.
  - In practice, the "oldest change first" ordering and the 2%-per-period cap already make re-changes rare; the dwell is what makes them impossible.
- **Convergence:** within about 22–35 s with the CPU and true signals, with no sustained oscillation at 2% signal noise.
- **CPU saturation:**
  - **With a big overload it doesn't slow the CPU controller at all:** 34 s vs 34 s for the true signal, and 26 vs 26.
  - **Near capacity it does.** At 90% load the CPU controller takes 35 s against 14 s. At 95% it gets stuck with a backend at ρ = 1.07 (p99 26 s).
  - The true signal also ends over capacity at 95% (ρ = 1.02). So the main culprit is the controller's *relative* dead band; saturation makes it worse.
  - An overload override I added (off by default) unsticks it, but at 90% load noise alone triggers it constantly. Tuning it is a phase-3 decision.
- **RIF** gives 2-core backends lower utilisation, but the same p99 as CPU. In the model its readings are much noisier, so it never stops moving buckets, and at 95% load it oscillates.
- **Live smoke test:** both the CPU and RIF controllers worked end to end with no errors. But real per-reading noise is about 7–15%, not 2%, so after converging the controller keeps moving around 40 buckets per period. The simulator at 10% noise reproduces exactly that. Next step for phase 3: measure the real signal noise, then choose period, smoothing and band with the simulator.

The LB is now at TC (D35, replaces D9)

- **Port:**
  - All the director logic now lives in `bpf/director.h`. `director_tc.c` (used) and `director_xdp.c` (kept for the comparison) differ only in how they make room and send.
  - Every golden test runs against both programs with the **same expected bytes**: 31 test runs pass.
  - One gap: the kernel's test runner won't feed a truncated IPv4 frame to a TC program, so that case is tested on XDP only. The check it exercises is shared code.
  - `lbctl load` attaches the TC version; `lbctl load -hook xdp` attaches the XDP one, and still works live.
- **End-to-end and RQ4 on TC:** everything passes. The move test gives the same picture as with XDP: 0 / 130 / 632 broken, against 0 / 146 / 671 before.
- **What TC costs:** on a NIC with an XDP driver, the XDP paper measures 24 Mpps for native XDP against 4.8 Mpps at the earliest point of the regular stack, a 5× gap, and TC sits at about that point. Our LB handles tens of thousands of packets per second, so that only rules out a line-rate claim, which isn't our research question.

The client checksum workaround (D26) isn't needed with TC. I removed it and turned the client's checksum and segmentation offload back on. A capture shows 65 KB GSO packets passing through the LB encapsulated and being decapsulated at the backend, and 20 uploads of 1 MiB arrive complete. The workaround is only needed again for the XDP run of the side measurement.

Not done yet: the per-packet comparison of the two hooks itself. The method is in `plan.md`: the same packet rate through each hook, comparing the LB CPU's time spent per packet. Expect a smaller gap than the paper's, because on veth even "native" XDP starts from a packet buffer the kernel has already built.

Prompt 22

nice, the sim results are really useful, especially that the dwell makes re-changes impossible rather than just rare, and that the cpu controller only gets stuck near capacity. and good to know the checksum workaround went away with tc.

one thing before the live comparisons: since the real signal noise is 7-15% and rif looked much noisier in the sim, can we choose the period, smoothing and dead band with the same procedure for every signal (measure each signals real noise, then pick its settings with the simulator using the same rule)? otherwise rq3 would partly be comparing how well we tuned each signal rather than the signals themselves. keep that tuning step short.

change of plans on our side: we want to finish the whole thing in about 10 more hours, so the benchmarks need to be quicker than what we planned, but we still want them as rigorous as possible within that time (we will also rerun the important ones on our 4 laptops later). so before building more, can you redesign the benchmark campaign with a budget of about 3 to 4 hours of machine time total on this laptop, spending the time where it buys the most confidence:
- short runs (something like 20-30 s per load step and 1-2 min phases), 2-3 repeats only where it matters, and a coarse capacity search
- keep only the experiments that answer the research questions: static baselines (equal, by cores, oracle), the live controller with cpu vs rif on a step change and the antagonist scenario, the connection survival under the change cap, and psi/cgroup cpu only as a

quick extra on the antagonist case
- drop the stateful baseline and the big timescale sweep, or shrink them to one point
- give a table with each experiment, configs x repeats x duration and total time, and say what each one can and cant claim with the shorter runs
also anything still missing in the backend (cost distribution, logging the harness needs) should go into this step, not a separate one

Claude Code

Two things are needed before more gets built: a tuning rule that's identical for every signal, and a campaign that fits about 3–4 hours. I'll write both into `docs/campaign.md` and the decisions log, and also build the backend and logging pieces the campaign needs. First, the backend's current flags and handlers, since the campaign needs three additions there:

I've redesigned the campaign to fit about 2.5 hours of testbed time, plus about 45 minutes of reserve for one rerun. The plan is in `docs/campaign.md`, with D36 for the tuning rule and D37 for the campaign and workload. The backend additions and the logging tool are built, checked and committed.

One tuning rule for every signal (D36)

1. **Measure noise.** Fixed oracle table, no controller, all four signals logged every second for 150 s, at 50% and 80% load. Averaging those 1-s readings gives each signal's noise σ at every period P ∈ {1, 2, 4, 8} s, from one run.
2. **Set the dead band from the noise**, the same way for every signal: start correcting above 3σ, stop below 1.5σ.
3. **Pick the period in the simulator**, with its noise set to the measured σ: the smallest P that moves at most 1% of the table per minute in steady state. Among those, take the one that recovers fastest after a step.
4. **Everything else is identical** for all signals: gain, caps, dwell, gates.

A noisier signal ends up with a wider band or a longer period, which is a fact about the signal, not about our tuning effort. Smoothing *is* the averaging period; a separate smoothing knob would do the same job twice. This takes about 6 minutes of testbed time plus a few minutes of simulation.

PSI is logged in every run but not used as a controller input. A quick check showed its values only reach about 0.1 even with an antagonist on the backend. That's below the controller's low-load gate, because our backend queues requests in-process (D25).

The campaign

Settings throughout:

- **Load steps:** 20 s measured after a 5 s warm-up.
- **Capacity search:** 5 probes, about 2.3 minutes, 1.25% resolution.
- **Backends:** beP1 is slowed 1.6×, so that neither *equal* nor *by cores* is the right split.

Session	Experiment	Configs × repeats	Time
S1	End-to-end check	1	2 min
S1	Each backend's own capacity (denominators; also gives the short protocol's CV)	4 × 2	18 min
S1	Signal noise for tuning	2 load levels	6 min
—	Tuning in the simulator	computation only	~5 min
S2	Static policies: equal ×1, by cores ×1, oracle ×3	5 runs	12 min

Session	Experiment	Configs × repeats	Time
S2	Controller-found tables (converge 90 s, freeze, search): CPU ×2, RIF ×2	4 runs	15 min
S3	Step: beP2 slows 2× at 60 s. Static oracle ×1, CPU ×3, RIF ×3	7 runs	25 min
S3	Antagonist on beE1, 60–180 s: oracle ×1, CPU ×3, RIF ×3, cgroup ×1; PSI logged passively	8 runs	36 min
S3	No-protection contrast (dwell 0, no caps): shows live what the dwell prevents	1	5 min
S4	10-s flapping antagonist (second timescale point), SMT-sibling antagonist, TC vs XDP cost	2 + 1 + 6 runs	17 min
	Setup between runs		~10 min
	Total		**~2.5 h**

S4 is dropped first if time runs short. The stateful baseline is dropped. The timescale sweep shrinks to two points: the 120-s antagonist and the 10-s flapping one.

Connection survival (RQ4) costs no extra runs. Every dynamic run carries 100 long-lived connections that outlive the 20-s dwell, alongside the short churn connections. So every run counts broken connections, and the long-lived ones give the real trade-off number.

What the shorter runs can and can't claim:

- **Absolute capacities:** 20-s probes read a few % high, but the usable-capacity *fractions* compare capacities measured the same way, so most of that cancels.
- **Equal / by cores vs oracle:** expected 10–35% apart, so one run each is enough.
- **Controller vs oracle:** we can claim "matches within about 3%", nothing finer.
- **Dynamic runs:** 5-s windows give recovery times to about ±5 s and p99 to about ±15%. Three repeats can separate "recovers in 30–40 s" from "never", but not 10-s or 30% differences.
- **cgroup / PSI:** we can say which signals *see* the antagonist, not rank them as controller signals.
- **RQ4:** gives exact counts of broken connections, not a model of how breakage grows with the change rate.
- **Cross-machine repeatability:** comes from your later 4-laptop reruns.

Built in this step

- **Backend:**
  - `-cost exp`: exponential request cost. Checked: 50,155 units per request on average for a mean of 50,000.
  - `/slowness?set=` changes slowness at runtime, so the step scenario doesn't have to restart a backend and lose its connections.
  - `work_units_total`: how much work each backend actually did.
- `cmd/sigmon`**:** logs all four signals plus each backend's request and work rate every second. Checked with an antagonist on beE1: whole-core CPU went from 0.13 to 0.64, while cgroup CPU only went from 0.10 to 0.12. It's blind to the neighbour, as H5 predicts.

Next: the harness. That's the capacity search through the LB, the scenario runner, the tuning tool, and controller flags for the dead band. Then S1.

Prompt 23

this campaign looks great, and deriving the dead band from each signals measured noise is a clean answer to the fairness question. the cgroup cpu check (0.10 to 0.12

with a neighbour) is already a nice small result for h5.

one thing i noticed got dropped: the second backend mode from d25, where requests wait in the kernel run queue instead of our go queue. without it the psi result just says "psi doesnt see our backend", which we already know from the design. could you add one cheap run in s4 with that mode under request overload (and the antagonist if its free), only logging the signals, so we can show psi does see overload when the server queues in the kernel? it should only cost a few minutes.

then go ahead: build the harness (capacity search through the lb, scenario runner, the tuning tool and the dead band flags) and run s1 and s2, including the tuning step. show me the numbers and plots, and commit as you go

Claude Code

A 1.6× slowdown at 1000 req/s, an antagonist, and now a kernel-queue mode. I'll go in this order:

1. Add the D25 kernel-queue mode to the backend and to S4 in the plan.
2. Build the harness: capacity search through the LB, scenario runner, tuning tool, dead-band flags.
3. Dry-run everything at tiny durations, then run S1, the tuning step, and S2.

First, what's reusable from the noise check, and whether plotting is available:

The sample landed after the burst had already finished, so it's not a result. Retrying with a longer burst and an earlier sample:

Unchanged now. Next, `cmd/ctltune`:

Now the harness. First, the existing helpers it will build on:

Next, `bench/tune_signals.py`: it turns the S1 noise logs into σ(P) per signal and runs `ctltune` with the measured capacities.

Before the dry run, one change: skip the end-to-end gate in `--dry` mode, since it's itself a 2-minute test. Then a dry run of S1, the tuning step and S2:

Confirmed: every request was served, but random arrivals produced only 96.6% of the target count in that 4-s window. Fixing all three, plus the docs:

That one is a gap in the harness: `sudo` created the campaign root folder as root, and only the session subfolder gets handed back. The fix is to hand back the whole root:

These are static PNGs for the repo and the write-up, so light mode only. The charts need up to 5 categorical series (5 policies), so I'll validate the first 5 slots:

The analysis runs. Step 7 of the dataviz procedure is to look at the rendered charts, so here are the two densest:

S1 is running (about 30 minutes). I'll wait for it rather than run anything heavy alongside: everything I start now shares CPUs 0–1 with the harness's orchestration.

Now session S2, about 30 minutes: 5 static searches (oracle ×3, equal, by cores), then 4 controller-found tables (CPU ×2, RIF ×2, each converging for 90 s from by-cores shares before its search).

S2 is running. I'll pick up with the analysis and plots when it finishes.

The safety check blocked that command because `$R` was unquoted in an `rm`, so none of it ran. Redoing it with literal paths:

Committed. S1 ran without the cancellation change, but at loads that meet the SLO no client gives up, so its numbers aren't affected. Rerunning S2 (about 30 minutes):

S2 is running again with the fixes; I'll analyse it when it completes.

The third S2 run is in progress, about 30 minutes. S1 and the tuning results are committed; I'll analyse S2 when it finishes.

The `if True:` lines are a leftover of how I split the function. That's not code anyone should have to explain, so I'll dedent the two functions properly:

The first function didn't dedent, because its `return` line already had the shallower indent. Fixing that one by hand:

Now I'll set aside the old noise runs, tuning and the third S2 attempt as evidence, commit, and rerun the noise part of S1 (about 7 minutes):

S2 is running a fourth time, with the TIME_WAIT fix and the final tuning (about 30 minutes). I'll analyse it when it finishes.

The labels are clear now. Last step: the journal entry, a short decision entry for the testbed fixes the campaign forced, and the commits.

S1, the tuning step and S2 are done and committed; the kernel-queue PSI run is added to S4. The numbers below come from the fourth S2 run. The first three were invalid because of testbed problems, which are now fixed and kept as evidence. Machine time so far is about 3 hours, against the 1 hour planned for S1 and S2.

RQ1, static policies (S2)

Usable capacity fraction at p99 ≤ 20 ms:

policy	runs	usable fraction	range
equal	1	**62.5%**	
by cores	1	**86.7%**	
oracle	3	**95.1%**	94.0–96.2%
table found by the CPU controller	2	**92.8%**	89.5–96.0% (89.5 is a lower bound)
table found by the RIF controller	2	**86.6%**	84.9–88.3%

- **H1 holds.** By cores is 8.4 points below the oracle, and equal is 33 points below. The smallest difference we can claim is about 5%, because the backends' own capacities varied by about 2.5% between repeats.
- **H2 holds for CPU.** The CPU controller's tables are within that 5% of the oracle.
- **RIF's tables lose about 8.5 points, and the ranges don't overlap.** RIF gives the 4-core E backends more load than the oracle does. At the same utilisation, a 4-core backend has fewer requests queued per core, so it looks less loaded to RIF. But its cores are 1.84× slower, so it misses the SLO first.
  - This is the opposite of what H4 expected for RIF.
  - It may come from how we normalise RIF (dividing by the core count, D10) rather than from RIF itself. I haven't changed anything.
- The oracle reaches 95%, not 100%. With 512 connections, each backend's load moves 7–11% from second to second, and the p99 follows whichever backend is busiest.

Tuning: the same rule for every signal

signal	period	dead band (start / stop)
CPU	2 s	0.17 / 0.08

signal	period	dead band (start / stop)
cgroup CPU	2 s	0.16 / 0.08
RIF	1 s	0.57 / 0.29

RIF is about 3× noisier than CPU at 80% load, so it gets a band about 3× wider.

I changed the rule twice before any live comparison, and D36 records both together with what the original rule would have chosen:

- The original's fallback was a cliff: it gave CPU an 8-second period (61 s recovery in the simulator) in exchange for 10 fewer bucket moves per minute.
- The simulator's own noise was higher than the measured noise. The tuning can only add noise, not remove it, so it would have judged CPU noisier than it is. It now simulates 20× finer connections.

Why S2 took four runs (D38)

Each invalid run had a different testbed cause, and each fix was checked:

1. **Queue carry-over between probes.** A probe after an overloaded one ran into its queue. The harness now waits for idle backends, and the backend drops requests whose client has gone.
2. **The load generator ran out of CPU above about 4500 req/s.** It now also uses the second hyperthread of its two cores.
3. **The client's TIME_WAIT sockets.** Once about 40% of its ports were in TIME_WAIT, opening a connection sometimes stalled for tens of milliseconds. A direct test: the same 4700 req/s failed 2 of 5 probes before the fix and passed 5 of 5 after. I first blamed lumpy per-connection load, which was wrong. This also inflated the original noise measurements, so I redid them.

Plots: in `results/campaign-20261102/`: `s2-capacity.png`, `s2-p99.png`, `s2-convergence.png`, `s1-denominators.png`, `tuning-noise.png` and `tuning-choice.png`. All numbers are in `summary.md` next to them.

Your decision: S3 and S4

S3 plus S4 would take about another 1.5 hours, which puts the campaign at about 4.5 hours instead of 3–4. To get back to about 1 hour, I'd suggest:

- 2 repeats instead of 3 for CPU and RIF in the step and antagonist scenarios;
- keep the kernel-queue PSI run (7 min);
- drop the TC vs XDP hook-cost measurement.

The alternative is to run everything as planned. Which do you prefer?

Prompt 24

nice, four runs is annoying but every one of those bugs would have poisoned s3 and s4 too, so good that they got caught. the rif result is interesting, i think it's a real finding and not just a bug: at the same utilisation a 4 core backend has a shorter queue per core, so anything queue-based looks lighter there even though its slow cores miss the latency target first. can you add one cheap s2 repeat with rif not divided by cores, so we can tell if the problem is the normalisation or rif itself, and write it up as a finding either way. small things i noticed: unitsDone is counted when a request is queued, so requests skipped after the client left still count as served work, count it after the work instead. the one probe with a 20 ms send delay was ctl-cpu at 7048 req/s, which was above capacity anyway so it doesn't matter, but say so in the summary. and the ctl-cpu row in the s2 table has an extra column at the end.

for s3 and s4 i'd rather run the full plan, 3 repeats for cpu and rif. repeats matter here since the smallest difference we can claim is about 5%. keep the tc vs xdp cost too, but make it a short run per hook, it's what backs up running the lb in tc on purpose. so: the step and antagonist scenarios with cpu vs rif (cgroup cpu and psi logged), the no-protection contrast, the flapping antagonist, the sibling antagonist, the kernel-queue backend run for psi, and tc vs xdp. then a short results summary per research question, with the honest caveats from the short runs

Claude Code

Plan for this turn:

1. The three small fixes.
2. A **total-RIF** signal (RIF not divided by cores), tuned with the same D36 rule, plus one extra S2 search with its controller-found table.
3. The S3/S4 harness: the scenario runner, kernel-queue PSI and TC vs XDP. Dry-run everything, then run S3 and S4 at full repeats.
4. Analysis, plots, and a results summary per research question.

Starting with the backend's work counter:

The tuning chart doesn't know `rif-total` yet. I'll give it its own colour and a place in the chart order:

Now a dry run of all three new sessions (S2-extra, S3, S4) in a scratch root that reuses the real S1 and tuning results, with short durations and no tuning:

The three sessions are running back to back (about 1 h 45 min). When they finish, I'll check each for testbed problems the same way as for S2, then write the analysis and the per-research-question summary.

The step chart is honest now. Last, the results write-up per research question (`docs/results.md`), the journal entry, and the commits:

That check caught an overstatement in what I just wrote: **not** every live move was a swap; at least one step run had reassignments. Per run:

Everything ran at full repeats and is committed. The write-up per research question is in `docs/results.md`, and every table and plot is in `results/campaign-20261102/summary.md`. No testbed problems this time. The whole campaign took about 4 h 50 min of machine time, including the three invalid S2 runs, against about 2.5 h planned.

Your RIF question: the normalisation is the problem

A controller using RIF **not** divided by cores (tuned by the same rule) found a table close to the oracle's. It passed every probe up to 95.1% usable capacity, so that's a lower bound, against 86.6% for RIF per core. That's one run, but far outside the per-core range (84.9–88.3%).

This confirms your explanation: dividing by cores makes the 4 slow E-cores look lighter than they are. That reverses H4 for per-core RIF; with the right normalisation, RIF does as well as CPU here.

Results by research question

RQ1, usable capacity (static):

policy	usable fraction
equal shares	62.5%
shares by cores	86.7%

policy	usable fraction
oracle (3 runs)	95.1%
CPU controller's table (2 runs)	92.8%
RIF-per-core controller's table (2 runs)	86.6%
total-RIF controller's table (1 run)	≥ 95.1%

- H1 and H2 hold, H2 for CPU and total RIF.
- The smallest difference we can claim is about 5%.

RQ2, dynamic controller vs static oracle (75% load):

disturbance	static oracle	CPU controller	RIF controller
step: beP2 needs 2× CPU per request	collapses	back within SLO after 30–35 s	after 15–25 s
120-s antagonist on beE1	p99 ~200 ms throughout	after 10 s	after 5–10 s
10-s flapping antagonist (1 run each)	50% of windows over SLO	12%	38%

The controller also helps at 10-s flapping, which goes against half of H3, but on a single run.

RQ3, which signal:

- **H5 holds.** cgroup CPU is blind to a neighbour in another cgroup: it even drops (0.91×), and a controller using it never recovered.
- **PSI depends on where the server queues.** With our pool backend it stays at or below 0.14 and *falls* under request overload (0.05 at 130%). With the kernel-queue backend it reaches 0.68 at 90% and 0.99 at 130%.
- **H6 couldn't be tested.** The sibling antagonist ran but didn't slow beP1 at all, because our request work is a dependency chain that leaves the core idle for its sibling.

RQ4, connection survival:

- **No long-lived connection broke, and no backend sent a reset, in all 20 live runs.**
- Of 55,172 live moves, all but 29 were swaps. With rendezvous second hops in every bucket, the controller almost never needs a reassignment, and swaps can't break connections.
- The no-protection contrast made 16,311 moves, all swaps. So it never exercised the risky move type, and live breakage evidence still comes only from the move test and the simulator.

Hook cost: on veth, XDP costs 15–22% *more* CPU per packet than TC: 5121 vs 4439 ns at 2000 req/s, and 4413 vs 3621 at 4000, one run per hook. Here even "native" XDP rebuilds the packet buffer, while TC reuses it. That backs D35 on our hardware, and says nothing about NICs with real native XDP.

Caveats that apply throughout

- Short runs: 20-s probes, 5-s windows, 1–3 repeats, one laptop, one workload, one load level for the dynamic runs.
- Window p99 only counts successful requests, so runs with errors get no recovery time. I found this when the static oracle appeared to "recover" while 12% of its requests were timing out.
- The three small fixes are in: work is counted after it's done, the S2 table has no stray column, and the summary says the one send-delayed probe (ctl-cpu at 7048 req/s) had already missed the SLO.

I can also publish `docs/results.md` with its plots as a shareable page for the team, if that's useful.

Prompt 25

no need to publish it as a page for now, the repo is enough.

the rif-total result is the most interesting thing so far, but it rests on one run that hit the top of the search range. a few things before the 4-laptop scripts:

1. give rif-total 2 more s2 repeats with the search range opened up, so we get an actual number and not a lower bound. it should also replace per-core rif in the dynamic comparisons, so run the step and antagonist scenarios with rif-total too (2 repeats each is fine)

2. the no-protection contrast doesn't test what rq4 is about. i get why swaps alone never break anything, the {first, second} pair stays the same, but that only holds because the redirector is on. the real contrast is the same antagonist run with the cpu controller and the redirector detached from the backends, so a swap sends existing connections to a backend that doesn't have them. one run should be enough to show the breakage live. and since the controller almost never reassigns, say clearly in the results that this is a property of the design (rendezvous second hops + swaps), which is a nice result on its own

3. the sibling test: i don't think the antagonist was the right kind. if our work is a dependency chain that leaves the core idle for its sibling, then an antagonist that runs the same burn loop (or something heavy on the same execution units or memory bandwidth) should actually contend. one oracle run with that, so h6 either gets tested or we can say why it can't be on this cpu

4. then the timescale sweep we cut earlier: flapping periods from fast to slow (something like 5, 20, 60, 180 s), cpu vs rif-total, so rq2/h3 has a curve instead of one 10 s point. repeats where you can, but keep this whole message's runs to about an hour, tell me the estimate first and what you'd cut if it's over

update results.md after

Claude Code

Budget first, as you asked:

item	runs	estimate
1. total-RIF S2 repeats, wider search range	2 × (90 s converge + 5–6 probes)	~10 min
1. total-RIF in step and antagonist	2 + 2	~16 min
2. antagonist + CPU controller, redirector *not forwarding*	1	~5 min
3. sibling antagonist that contends	1 oracle run, two antagonist kinds in sequence	~4 min (after a 1-min local check)
4. flapping sweep, CPU vs total RIF vs static oracle	periods 5, 20, 60, 180 s, 1 repeat each	~40 min
session setup (2 sessions)		~3 min
total		**~78 min**

That's about 18 minutes over. What I'd cut: **the 180-s period from the sweep.** Its slow end is already covered by S3's antagonist runs: 120 s on, 60 s off, a 240-s cycle, with CPU ×3, per-core RIF ×3, oracle ×1, and the

new total-RIF ×2. Reusing those as the slowest point saves 12 minutes. Shortening the fast periods' disturbance windows to 120 s brings the total to **~66 minutes**, with 1 repeat per sweep point. That fits "about an hour", so I'll go ahead on that basis.

Note on item 2: "detaching" the redirector entirely would also remove the decapsulation, so no traffic would work at all. Instead it gets a load-time switch: it still decapsulates, but never forwards to the second hop.

Starting with the antagonist, which needs kinds of load that actually compete for the hardware a hyperthread sibling shares:

All tests pass, including the new one: with forwarding off, a packet whose connection isn't on this backend is decapsulated and kept, exactly like a missing second hop. Committing both pieces:

One bug there: `flapping-5` toggles every 2 s, a 4-s period, because the half-period was rounded down to an integer. Using exact half-periods (2.5 s):

The analysis works on the existing data. Dry runs of the two new sessions next (the dry root needs the current tuning, which now includes total RIF):

The real runs have started: the total-RIF S2 repeats first, then S5. That's about 63 minutes, inside the hour you set. I'll update `results.md` when they finish.

All of S5 ran cleanly, and the tuning was undone afterwards. The sibling section was cut off by my filter, so here it is in full, plus a check that the flapping toggles actually happened on time:

Now updating `results.md`: RQ1 (total RIF with real numbers), RQ4 (the redirector contrast and the design property), RQ2 (the sweep), RQ3 (total RIF in the dynamic runs, and H6 now tested):

The sweep chart reads as intended: the oracle sits far above both controllers at every period, while CPU and total RIF stay at 8–17%. Journal entry and commits:

All four follow-ups are done and committed, and `docs/results.md` is updated. They took about 63 minutes of machine time, against my 66-minute estimate. To fit the hour I cut the 180-s sweep point as I said; S3's antagonist run (120 s on, 60 s off) serves as the slow end.

1. Total RIF now has real numbers, and it matches CPU but doesn't beat it. With the wider search range, the two new runs landed at 90.7% and 92.7%. With the first run's lower bound of ≥95.1%, the three average 92.8%, the same as CPU. My earlier reading that total RIF "reached the oracle" rested on that one lower-bound run and was too strong. The finding that holds is about the normalisation: total RIF (90.7–95.1%) doesn't overlap with RIF per core (84.9–88.3%).

In the dynamic runs (2 repeats each), total RIF recovers from the antagonist like CPU (10 s), with the lowest late p99 (13.5–15.6 ms). It recovers more slowly from the step: 40–45 s, against 30–35 s for CPU and 15–25 s for RIF per core.

2. The RQ4 contrast now tests what it should. I couldn't detach the redirectors entirely, because they also decapsulate and no traffic would get through. Instead they were loaded with forwarding switched off, so they decapsulate but never send anything to the second hop. One run with the CPU controller and the antagonist:

- **36 of the 100 long-lived connections broke**, plus 160 short ones, and the backends sent 391 resets;
- all 1359 moves were swaps.

With forwarding on, 0 connections broke in 34 runs. `results.md` now states the design property explicitly: of 74,928 live moves, 74,899 were swaps, because every bucket starts with a rendezvous second hop. Swaps keep both backends in the bucket, so with forwarding they can't strand a connection, with or without a dwell; the dwell only protects the rare reassignment.

3. H6 is now tested, and it's partly wrong. A local check first showed which neighbour actually contends on a hyperthread sibling:

sibling runs	slowdown of our request loop
the same loop	2%
independent multiplies	1%
a 64 MiB walk	**39%**

The walk wins because it evicts the caches the two hyperthreads share. Run on beP1's siblings, it changed beP1's signals like this:

signal	change
RIF	+25%
whole-core CPU	+10%
cgroup CPU	+9%
PSI	none

So H6's "only RIF notices" doesn't hold: the CPU signals measure time, and slower work takes more of it. p99 barely moved (10.7 → 11.0 ms), but beP1 was only at about 63% utilisation.

4. The sweep: the controllers beat the static oracle at every period, down to 5 s. Share of the disturbance spent over the SLO, one run per point unless noted:

period	5 s	10 s	20 s	60 s	240 s
static oracle	96%	50%	50%	50%	100%
CPU controller	12%	12%	17%	8%	17% (3 runs)
total-RIF controller	12%	–	12%	12%	10% (2 runs)

There's no crossover between 5 and 240 s, so H3's "for fast disturbances nobody helps" isn't supported. At fast periods the controller doesn't follow each flap; it settles on a table that gives beE1 less load on average, which keeps the on-phases within the SLO. The chart is `results/campaign-20261102/sweep.png`.

The caveats in `results.md` now include that the sweep points and the redirector contrast are single runs, and that nothing faster than 5 s was tested.

Prompt 26

this is a much better set of results, especially the redirector contrast, 36 of 100 broken without forwarding vs 0 in 34 runs with it is exactly the picture rq4 needed. and good that the open search pulled total rif back to 92.8% instead of us claiming it beats cpu off one run. h6 being partly wrong is fine, it's a real answer

last big piece: we want to rerun the main experiments on our 4 real laptops later. all linux, wifi only (no ethernet ports on two of them), so probably 1 client, 1 lb, and 2 laptops each running 2 backends to keep the 4 backend setup. can you write scripts for that: one config file with the machines ips/interfaces, a setup script per role, and a single driver we run from one laptop over ssh that does the short version of the campaign end to end in under about an hour and collects the results into the same format so the same analysis and plots work. keep it simple, reuse the existing harness where you can instead of a second copy

and a short runbook with checks before starting. things i can think of: mtu (the encap adds 40 bytes and wifi drivers often can't raise the mtu, so probably lower it or clamp mss on the client), wifi power save, clocks, the ap blocking peer traffic (client isolation) or dropping replies from the backends since they use the vip as source, a phone hotspot vs the college wifi, and no generic xdp needed since the lb is tc. plus what we should expect to differ from the single laptop numbers. you can't test it on real machines here, so test what you can locally (e.g. the driver against the namespace testbed or a dry run) and say clearly what's untested

Claude Code

This is the biggest change to the harness so far, so here is the design before writing code. The constraint that shapes it: **on wifi, a laptop can only send frames with its own MAC address.** So the backends can't sit on the wifi network with IP addresses of their own, the way they share the bridge in the namespace testbed. Instead:

- **Every backend host is a small router.** Each backend runs in its own network namespace with a private subnet, as now (e.g. h1: `10.11.1.0/24` for beP1 and beP2). The laptop forwards between its wifi interface and the namespaces.
  - The director already writes the destination MAC itself. It uses the *host's* wifi MAC as next hop and the backend's private IP as the outer destination, so the director's code doesn't change.
  - Second-hop forwarding works the same way: the redirector hands the packet to its host, which routes it to the other host.
  - Replies go out from the backend through its host to the client, with the VIP as source address, exactly as now.
- **The LB laptop** runs the director at TC ingress of its wifi interface, plus the controller.
- **The client laptop** runs the load generator, with a route that sends the VIP via the LB. That route gets a lower MTU so the 40-byte encapsulation fits (details in the runbook).
- **Signals come over the network.** The controller on the LB laptop can't read another laptop's `/proc/stat` or cgroup files, so each backend also exports raw counters for all four signals on its metrics port. Total RIF already comes from there.
- **One harness.** `testbed.py`'s helpers become the seam: in cluster mode they run commands over ssh on the right host, while `campaign.py`'s experiments stay as they are. A driver, `bench/cluster.py`, sets up the hosts, runs the short campaign, collects the files and undoes everything.
- **Testable here:** I can stand in for the four laptops with four network namespaces on a bridge (the "access point"). The driver then reaches each fake host through `ip netns exec` instead of ssh, and everything else runs the real cluster code path.

Starting with the Go side. First, the current testbed config reader and the backend's metrics handler:

I dropped the `package` line when I replaced the file's comment. Putting it back:

Now the signals: in cluster mode every signal comes from the backend's metrics over plain HTTP, with no namespace switching on the controller's machine.

Line 266 is garbage I wrote by mistake. It wouldn't even compile, and it does nothing. Removing it:

One more lbctl detail: in cluster mode the redirectors' counters live on the backend machines, so `stats` on the LB machine has to skip them instead of failing.

Go side done. Before the Python side, a regression check that the local testbed still works end to end after the lbctl split and the backend changes:

No regression: e2e all passed, and the golden tests pass (32). Committing the Go side:

The routing works: from the client "laptop", beE1's namespace on h2 answers a ping through h2. Next, a check that `down` really restores everything, then the whole driver in dry mode:

The local testbed still works after the refactor, and the cleanup is complete: no namespaces, tuning not applied. Now the runbook, a decision entry (D39), README and journal:

Context limit reached. The session continued from an automatic summary.

Docs are written. Before committing, I'll check why `results/campaign-20261102/summary.md` changed, probably from rerunning the analysis after the SLO-from-data change:

The summary diff is wording only (no numbers change), so it goes in with the analysis commit. Committing in five pieces:

I've written the scripts for the four-laptop rerun and committed them in five commits, on top of the three Go commits from earlier. The working tree is clean. A dry run of the whole driver passed on four machines emulated with network namespaces; nothing has run on real laptops or wifi yet.

How it works

- **The network trick:** a wifi laptop can only send frames with its own MAC address, so the backends can't have addresses on the wifi network. Each backend laptop keeps its two backends in namespaces behind a bridge on a private subnet (10.11.1.0/24, 10.11.2.0/24) and routes to them. The load balancer sends a backend's packets to that laptop's MAC. The director and redirector code didn't change. This is written up as decision D39.
- **Signals:** the controller runs on the LB laptop and can't read the other laptops' `/proc` or `/sys`. So each backend now publishes its raw counters on its metrics port, and the signals read them from there with the same arithmetic.
- **One harness, no copy:** `bench/testbed.py`'s `command()` decides where each command runs: in a namespace locally, or over `ssh … sudo` on the right laptop. `bench/campaign.py`'s experiments are reused unchanged.

What's there

- `cluster/cluster.conf.example`: the one config file. Fill in each laptop's IP, wifi interface and CPUs, and commit it so all four have the same copy (the driver checks).
- `scripts/cluster/setup.sh up|down NAME`: setup for each machine's role, every change recorded and undone exactly on `down`. The client's route to the VIP gets MTU 1460, so the 40-byte encapsulation still fits in 1500. Wifi power save is turned off during the run.
- `bench/cluster.py`: the driver, run from the client laptop as your user (not with sudo). In order:
  1. checks that every laptop is reachable and has the same commit and config;
  2. runs the setup on every machine;
  3. S1: a test of 10 requests through the VIP, each backend's capacity, and signal noise;
  4. tuning;
  5. S2: static oracle ×2, equal, by cores, plus the CPU and total-RIF controller tables;
  6. S3: the antagonist with the oracle, CPU and total RIF, and the redirectors-not-forwarding contrast;
  7. copies the result files back, undoes the setup on every machine, and runs the analysis.

  The output format is the same, so `analyze_campaign.py` and the plots work as they are. The only change there is that the analysis now reads the SLO from the data.

- `docs/cluster-runbook.md`: what to do before the day, checks on the day, how to run it, what to expect to differ, and the untested list.

Time: about 50 minutes for the full run (my estimate from the step lengths; only the dry run has been executed), plus about 10 minutes for the dry run. Do the dry run first. If the full run goes over, cut the second oracle repeat and the "by cores" run first (about 4 minutes).

What was tested here

- A dry run of the whole driver on four emulated machines (`scripts/cluster/emulate.sh` and `cluster/emulated.conf`):
  - checks, setup and undo;
  - the 10-request test, 10 of 10 through the VIP;
  - S1, tuning, S2 and S3;
  - file layout and analysis.
- The redirector contrast came out as expected: 15 long-lived connections broken without forwarding, 0 with it.
- Nothing was left behind afterwards.
- The single-laptop harness still works: the end-to-end and golden tests pass, and a dry S3 completed.

Untested, because it needs the real laptops

- ssh and rsync themselves. The commands they wrap are the ones the emulation ran.
- Anything about wifi: TC on real drivers, the MTU, turning power save off with `iw`, and how the access point behaves. The biggest risks are client isolation and dropping backend replies whose source address is the VIP. The 10-request test catches both and prints what to check. A phone hotspot or your own router is safer than the college wifi.
- `tune.sh` on non-Intel laptops. It assumes `intel_pstate`, so it may need a change for AMD.
- The full-length timing.
- Whether the scaled workload suits your wifi: requests cost 20 ms instead of 1 ms, the SLO is 400 ms instead of 20, and there are 64 connections. These are guesses. If the dry run's p99 at low load is already near the SLO, raise `COST_MS` and `SLO_P99_MS` together.

What to expect to differ from the single laptop: request rates about 20× lower, a wifi floor and noisier tails in every p99, and more run-to-run noise. Real differences between laptops replace the P/E cores plus slowness factor. The tuning rule will also pick different controller settings. Ratios like the usable capacity fraction and recovery relative to the SLO can be compared with the single-laptop results; absolute numbers can't.

Prompt 27

nice, the emulated dry run catching the redirector contrast (15 vs 0) is a good sign the driver does the real thing. one small thing: not all our laptops are intel, so give tune.sh a fallback for amd-pstate / acpi-cpufreq instead of assuming intel_pstate, with the same undo

then finally a quick review: go over the setup and results against the methodology from the papers we read (unimog, prequal, and the measurement ones) and fix anything misleading, rerunning something only if a result is actually wrong or unsupported. check especially that every number in results.md matches the data files, that single-run results are labelled as such, and that the hypotheses we changed after seeing data are marked as changed. then update the readme so someone new can understand the problem, the design, how to run it on one laptop and on 4 laptops, and the main results

Claude Code

Status works on Intel. I'll tidy the turbo line, then exercise the AMD and acpi-cpufreq paths plus undo against fake sysfs trees, since this laptop only has `intel_pstate`:

The test copy sourced `lib.sh` relative to its own location. I'll fix the copy and rerun:

Something is hanging. I'll check the output so far and what `lib.sh` does when sourced:

The `pkill` stopped it. In the first attempt the sourcing failed, so the functions never ran; the hang must be in the functions themselves. I'll trace one case with a timeout:

Now the review. First, which papers we're holding ourselves to, and the hypotheses as written before the campaign:

Before checking numbers, I'll see whether the plan's hypotheses were edited after any data came in:

The 173 M sum double-counts, because each phase row repeats its run's total. Counting each (scenario, signal) once:

Only the sweep changed. I'll check the regenerated plot:

The 20-s fix shows correctly. One readability problem remains: oracle (teal) and total RIF (green) are hard to tell apart. Checking the palette:

Two ratios I wrote need tightening against the table. The no-protection run made 7.6–11× the tuned CPU runs' moves (not "10×"), and total RIF's step moves are 2–2.8× CPU's (not "2–2.5×"):

Now `plan.md` (outcome pointer, stale SLO line) and `campaign.md` (connection count, flapping period):

The journal is a dated log, so its old entries stay and a correction entry will point to the fixes. I'll check whether the corrected claims also appear elsewhere:

Clean. Now the README. First, the single-laptop campaign's actual command line:

Both targets exist and `cluster.conf` isn't ignored, so "commit it" holds. Journal entry for the review, then commits:

Both are done: `tune.sh` has the AMD and acpi-cpufreq fallback, and the review fixed a mislabelled sweep point plus several wrong or overstated claims in `results.md`. No result was wrong enough to rerun anything. The README is rewritten, and everything is committed in four commits.

tune.sh on non-Intel laptops

- It now finds whichever turbo switch exists:
  - `intel_pstate/no_turbo` on Intel;
  - the global or per-policy `boost` file on `amd-pstate` and `acpi-cpufreq`;
  - `amd_pstate/cpb_boost` on some 6.11 kernels.
- EPP is only recorded where it exists. `acpi-cpufreq` has none, and the old script would have stopped with an error there.
- Every change is recorded and undone as before. If no switch is found, it warns and carries on; `status` shows the driver and the switch.
- I tested it against fake sysfs layouts for all of those drivers, and undo restored every file exactly. It hasn't run on a real AMD laptop. I didn't run a real apply/undo here, to keep tweaks to measurement runs; the Intel path is unchanged apart from the EPP check.

Fixed in the results

- **Wrong sweep point:** S4's flapping run (10 s on, 10 s off) is a 20-s period, but the analysis plotted it at 10 s. It now counts as a second 20-s run, so there is no 10-s point. The conclusion stands: the controller beat the

static oracle at 5, 20, 60 and 240 s.

- **"No sustained oscillation" was wrong:**
  - CPU settles: 0–143 moves/min after the step.
  - Per-core RIF keeps moving buckets back and forth: 1,279–1,929 moves/min, 7–11 reversals per minute. Total RIF does too, less.
  - p99 stays within the SLO and every move is a swap, so nothing breaks.
  - The tuning simulator predicted at most 40 moves/min, so it underestimates live noise.
- **Stale simulator numbers:** they came from the run before the RIF model fix. Without protection it's 27–6,711 broken connections (not 27–824), and the protected runs covered about 173 M connections per noise level (not ~50 M).
- **Redirector contrast:** 36 breaks among the long-lived connections, not "36 of the 100" (a reconnected one could break twice). The backends' counters show 395 resets sent; 391 is the reset monitor's count.
- **Smaller fixes:**
  - The 7–11% load fluctuation behind the oracle's 95% isn't in the data: the runs show 5–8% or 12–13%. It's now also marked as an untested explanation.
  - Averages that include a lower-bound run now read "≥".
  - Total RIF moves 2–3× as many buckets as CPU after the step, not "about as many".
  - Its lower late p99 is within noise.
  - The campaign plan said 128 connections; the harness always used 512.

Labelled

- Every result now gives its number of runs.
- A new hypothesis table gives each hypothesis's outcome. The hypothesis text never changed (the git history shows that), but four tests were added after seeing data and are marked as exploratory: total RIF, the cache-thrashing sibling neighbour, the sweep, and the redirectors-not-forwarding contrast.
- Comparisons across sessions weren't interleaved, which our own protocol asks for; calibration stayed within 0.4% across all seven sessions.
- From Prequal: our total RIF is Prequal's definition, and dividing by cores was our own choice. Our controller isn't a test of Prequal, which picks a server per request from fresh probes of RIF and latency.

README now covers the problem, the design with a diagram, the main results per research question, and how to run it on one laptop and on four.

One thing I left alone: in the sweep plot, oracle and total RIF are both green. They pass the colour check for normal vision and use different markers, so I didn't recolour.

Prompt 28

> good review, the per-core rif oscillation is worth having in there honestly. two last small things: since the intel path did change (the epp check), run a real tune.sh apply, status and undo once here and confirm everything is back to how it was. and give total rif its own colour in the sweep plot, two greens is confusing when it's printed. that's it from me for now

Claude Code

Running the real `tune.sh` apply/status/undo first, with a snapshot of everything it touches taken before and after:

The diff is only line order: the idle-state lines came out in a different order. Comparing the sorted snapshots:

None of the free palette slots passes against all three: amber and pink sit too close to orange, and violet too close to blue. I'll check the reference palette's other steps:

Both plots now show total RIF in a dark violet that stands apart from the teal, blue and orange. Committing:

Both are done and committed.

tune.sh on this laptop: I ran a real `apply`, `status` and `undo`, with a snapshot of 223 values before, during and after:

- turbo, governors, EPP and min/max clocks for every CPU policy;
- every idle state's on/off switch;
- the CPUs allowed for `user.slice`, `system.slice` and `init.scope`;
- every interrupt's CPU affinity, and whether irqbalance runs.

While applied, 120 values had changed as expected: turbo off, `performance` governor, 44 deep idle states disabled, 48 interrupts moved, and the desktop confined to CPUs 0–1. After undo, all 223 values match the before state exactly, and `status` shows "applied: no".

Total RIF colour: it's now a dark violet (`#4a3aa7`) in every chart. None of the palette's free colours could be told apart from blue, orange and teal, so I used this darker violet from the same palette family; it passes the colour-blindness and normal-vision checks against all three. The same fix also helps the tuning plot, where total RIF was green next to cgroup's teal. The redirector-contrast panel moved to the freed green, so the two violets never mean different things. I regenerated the plots and checked the sweep and tuning images.

Prompt 29

back after a few days, the laptop rebooted in between so tune.sh and the testbed need setting up again. i went through the whole repo and results again before calling it finished and found some things to fix. all of it in one go please.

code:
1. controller startup breaks the safety claim. cmd/controller/main.go publishes InitialTable over the live table without WithSecondHops, and internal/controller sets LastChanged to -inf. scenario_run installs the static table with lbctl and starts the controller and the loadgens right after, so those race, and a connection opened before the publish can land on a bucket whose new second hop isn't its old first hop. the resets csvs show it never happened in our runs, but the claim should hold anyway. build the first table from the active one with WithSecondHops and mark changed buckets as changed at time 0, add a test for it. the same startup is also why some runs show moves at t=1-2 s (the first signal reading covers the ramp-up), so discard that first reading
2. smaller ones while you're there: the director accepts inner ihl > 5 but the redirector drops it, so make the director drop it too. and the campaign doesn't notice if the controller dies in the middle of a run (one failed signal read is a log.Fatal, then it carries on with a frozen table), so make the harness check the controller is still alive at the end and mark the run invalid if not. that one matters more on wifi

new runs, about an hour total, tell me the estimate first:
3. static table re-weighted for the disturbance. our oracle is the table for the undisturbed system, so "controller beats oracle" after the step is partly just that the old table is now wrong. run a hindsight static table with the capacities that hold during the step and during the antagonist, so the comparison is fair, and report both

4. denominators with all the backends busy. right now each backend's capacity is measured alone without the lb, so the usable fraction mixes the policy with lb cost and contention between backends on one laptop, and that might be the oracle's missing 5%. measure it (our own protocol has a neighbours-busy check we didn't apply to the denominators) and say whether it explains the gap. keep the old denominators too and show both, don't silently swap them
5. cpu vs total rif s2 tables, properly interleaved in one session, as many repeats as fit. right now that comparison is across sessions

write-up, so results.md and readme say what the data supports:
6. h3 is mislabelled. its threshold is disturbances longer than the control period and connection lifetime, those are 1-2 s and ~0.5-2 s, and our fastest on-phase is 2.5 s. so every point is above the threshold. it should say first half consistent, second half untested, not "not supported"
7. "best static split" in the readme is wrong, nobody searched for it, it's shares proportional to standalone capacity, and one ctl-cpu run beat it. reword that and "beat the oracle" everywhere to say what it really is
8. d19 says no udp with no reason. write the real one: the second hop check depends on tcp state (socket lookup, syn cookies), udp has none of that, and connected udp sockets or quic connection ids would be the way to do it
9. h6 says every time based signal notices the sibling, but cpu and cgroup only rose ~10% in one run and the dead band is 17%, so no controller would act on it, say so. and "-39% speed not saved" has no data behind it, drop it or measure it. the 33x rif ratio in h5 is measured while the backend is overloaded, and cpu is capped at 1, so those two ratios aren't comparable, add that. hook cost is also just 39 vs 45 and 64 vs 78 ticks of 10 ms in one run, including router and bridge work, so state that
10. numbers that don't match the data: "0-440 moves/min" before a disturbance holds only for s3/s4 (flapping-20 made 726 and 520, noprotect 4000). load cv "5-8%" is really 4.4-12.5% per backend across s3/s4, and s1's noise run is 11.4-12.8%. flapping-5 beE1 "~1150 to ~860-970" is 1190 to 862-1011 for cpu. and the readme says "the same run broke 196 connections", it was a different run (s5) and 196 is failed requests (36 long lived + 160 churn)
11. the docs say interleaved a b c a b c, but in s2 all static policies ran before the controller tables, and in s3 the oracle ran once at the start of each block. say what actually happened. also RESOLUTION in the analysis assumes a +-20% search range and the real ranges differ, and the 5% claim rests on 4 standalone pairs while p99 near 20 ms isn't monotonic with load (oracle probes at 87-98% go between 13 and 27 ms). fix the constant and be honest about how firm 5% is. and the protocol asks for at least 5 searches and "never report a single run", so say plainly where we fell short

plots:
12. the 240 s sweep point covers only the on-phase, so it isn't comparable, don't join it with a line and mark it. the dotted rif line runs through points with no data, it should only connect real data. show run counts. draw lower bounds in s2-capacity as lower bounds, not normal dots. cpu is yellow in s2-capacity and blue in the timelines, make colours follow the signal everywhere like the comment says, and mark the runs from another session in s3-step

the data for the old runs predates the startup fix, note that where it matters, no need to rerun them since they had 0 resets. keep everything in results.md consistent with what you rerun, and keep the old numbers visible where the new ones change a conclusion

Claude Code

Starting with the current repo state and the code these items touch. I'll give the run estimate before starting any runs.

Dry session works. Next, check its outputs (first reading discarded, per-backend summaries, second hops), and a live test that a killed controller gets flagged:

A killed controller marks the run invalid and a healthy run isn't flagged, and the first reading is discarded. Committing the code, then starting the real S6 so the data matches a commit:

S6 is running; it needs about 70 minutes. Meanwhile I'll do the analysis and plot changes (light work, on the housekeeping CPUs only). First a quick look at the formats I need:

Your numbers all reproduce once the windows are set: load CV 4.4–12.5% and S1 11.4–12.8% counting from 5 s, and beE1 862–1011 from 36 s on. Now the analysis changes:

Now the documents that don't depend on the new data (D19, a new D40, the plan, campaign order, protocol shortfalls, runbook):

The greys now separate; the only remaining fails are the lightness and chroma checks, because they are neutral on purpose. The S6 session is still running, with about an hour left. When it finishes I'll run the analysis, write the results and README from the new data, and test the dead-controller check on the emulated cluster.

Prompt 30

while s6 runs: the ppt has to be submitted very soon, so i need this now and can't wait for the runs to finish. the presentation itself is a lot later, so we could still update numbers by then. rules from the course: 15 minutes total split however we want between talking and a demo, plus 5 min q&a. 5 slides, can extend by one, in this order:
1. problem statement & objectives
2. architecture / mechanism, our design and how it works
3. extension / issues fixed / evaluation, what we built on top, bugs resolved, and results
4. non-functional testing parameters, performance, scalability, reliability etc
5. challenges faced

we're making the slides ourselves, so please don't make a pptx or html or any slide file, and don't write finished slide text. just give us content and ideas for each slide: the few points that matter most, which number or which existing plot would make each point best, and what you'd leave out. use the corrected wording from my last message (h3, "best static split", etc) even where results.md isn't updated yet, and mark anything the s6 runs could still change so we use safe wording for it now. for slide 4 be straight about what we actually measured and what we didn't, like scalability, instead of filling the category. for slide 5 pick the challenges that are actually interesting from the journal, not all of them. also suggest how to split the 15 minutes and what a short demo could show that's reliable on one laptop, with a fallback if it misbehaves live. rough notes are fine, we'll write it in our own words. don't touch s6 or run anything heavy for this, and carry on with my last message once s6 is done

Claude Code

Here are the notes. S6 is still running and I haven't touched it; I'll carry on with your last message when it finishes. Anything tagged **[S6]** could still change, so use the safe wording for now.

Plots: don't use `sweep.png` or `s2-capacity.png` from `results/`, because they still have the problems you listed. Corrected versions from the old data are in `/tmp/claude-1000/-home-m4nny-Code-Courses-cs331-lb/43eb207f-3643-4f43-b206-23432dca2abe/scratchpad/plots-for-slides/`. Copy them out, since that folder is temporary. They're fine for submission, but the `n=` labels collide at the right of the sweep and the total-RIF marker is thin. I'll fix both after S6.

Slide 1: problem and objectives

- **Problem:** an L4 balancer only chooses a server when a connection starts. Servers differ and slow down, and we want no per-connection state and no broken connections.
- **Number to open with:** usable capacity at p99 ≤ 20 ms:
  - equal shares 62.5% (1 run)
  - shares by cores 86.7% (1 run)
  - shares proportional to standalone capacity 95.1% (3 runs)
- **Objectives:** the four research questions, one line each: usable capacity, timescales, which signal, connection survival.
- **Leave out:** the hypotheses list and related-work survey.

Slide 2: architecture

- **Figure:** use the diagram from the README, redrawn.
- **Walk one packet after a swap**, which explains the whole design:
  - Bucket `[A|B]` becomes `[B|A]`.
  - B has no socket for the old connection, so its redirector forwards the packet once to A.
  - New connections stay on B.
- **Points to make:**
  - 4096 buckets, each with a first and second hop
  - UDP encapsulation (40 bytes) and direct server return
  - keep rule: a socket, a SYN or a valid SYN cookie; otherwise forward once
  - controller reads one signal every 1–2 s and mostly does swaps
- **One line on TC:** we chose it because our laptops have no NIC with native XDP.
- **Leave out:** map-in-map, rendezvous maths, tuning-rule details.

Slide 3: extension, fixes, results (use the extra slide here)

3a: built and fixed

- **Built on top:**
  - load-aware controller with swappable signals
  - one tuning rule for all signals, based on noise and a simulator
  - redirector with second hops
  - four-laptop scripts
- **Bugs worth naming (3–4 at most):**
  - client port stall from TIME_WAIT, which invalidated a run
  - load generator saturating its CPUs
  - controller startup race: never fired (0 resets) and now fixed with a test

- director accepting IP options that the redirector then dropped

3b: results

Question	Best number	Plot	Safe wording
RQ 1	CPU table ≥ 92.8%, capacity-proportional 95.1%	`s2-capacity.png`	Never say "best static split": nobody searched for it. One CPU run (96.0%) was above its mean but not above its best run (96.2%). **[S6]** may move the CPU and total-RIF values
RQ 4	Move test: 0 of 300 broken with second hops, 130–146 of 300 without (1 run per director)	none, numbers are enough	Strongest result, and S6 doesn't affect it
RQ 4	34 live runs, 0 long-lived broken; 74,899 of 74,928 moves were swaps	none	These runs predate the startup fix, and S6 has no controller run with long-lived connections. Say "before the fix; the fix closes a race these runs didn't hit"
RQ 4	Redirectors not forwarding, 1 separate run: 196 failed requests (36 long-lived + 160 short)	none	Not "196 connections", and not "the same run"
RQ 2	Step recovery: CPU 30–35 s (3 runs), RIF per core 15–25 s (3), total RIF 40–45 s (2). Table for the old capacities: 12% errors	`s3-step.png`	**[S6]** Say "a table set for the capacities before the disturbance fails". Don't say "the controller beats the oracle"
RQ 2	Sweep: old table 50–96% of windows over the SLO, controllers 8–17% (mostly 1 run per point)	`sweep.png`	Same **[S6]** caution; show the 240-s point as not comparable
RQ 3	Total RIF and CPU both ≥ 92.8%; RIF per core 86.6%	`s2-capacity.png`	**[S6]** Say "no difference we can resolve so far, measured across sessions". The rerun could separate them
RQ 3	CPU settles (0–143 moves/min); RIF per core keeps hunting (1,279–1,929). cgroup CPU is blind to a neighbour (0.91×, 1 run)	`s3-signals-antagonist.png`	The 33× RIF ratio is measured under overload and CPU is capped at 1, so don't compare the two

- **H3 wording:** "first half consistent, second half untested". Our fastest on-phase is 2.5 s, against a 1–2 s control period and connections of about 2 s.
  - Lifetime from the data: 512 connections at 235 new per second gives 2.2 s, so we're only just above the threshold.
- **Don't explain the 95% yet [S6]:** say "about 5% is unexplained; we're measuring how much comes from contention and how much from the LB". Don't name hashing or contention as the cause.
- **Hindsight table [S6]:** don't predict it. It may well match or beat the controller once the disturbance is known. A safe line for later is that the controller gets there without being told.

Slide 4: non-functional, measured and not measured

A two-column table works well here, and being plain about the gaps will come across better than filling every category.

Category	Measured	Not measured
Performance	Capacity at SLO, open-loop p99, recovery time	Maximum LB throughput, controller overhead
LB cost	TC vs XDP: 39 vs 45 and 64 vs 78 ticks of 10 ms, 1 run, includes router and bridge work. Say "no sign TC costs more on veth" and skip the "15–22%" headline	NICs with native XDP
Reliability	Connection survival; a dead controller is now detected and the run marked invalid	Backend or LB failure, failover (no health checks, by design)
Scalability	**Nothing.** Say so: 4 backends, 1 LB, one laptop, 1 load level for dynamic runs	More backends, multiple LBs, real networks
Repeatability	Calibration within 0.4% across the earlier sessions; capacity CV 2.5% from 4 pairs	2–3 runs mostly, where our own protocol asks for ≥ 5
Correctness	Golden-packet tests in the kernel, simulator with 0 broken in ~173 M connections per noise level	The simulator predicted ≤ 40 moves/min; live runs made up to 440 in S3/S4 and 726 in S5

- **How firm the ~5% limit is:** treat it as a rule of thumb. With 4 pairs it could be anywhere from 3% to 14%.
- **Four laptops:** scripts exist, tested only on emulated machines. Say "not run on real wifi yet", and update it if you do run them before the talk.
- **Leave out:** security, usability, anything else with no data behind it.

Slide 5: challenges (pick 3–4)

1. **A test that couldn't fail.** The "no protection" contrast broke nothing, because all 16,311 moves were swaps. That exposed the wrong test and led to the key insight: swaps are safe only because the redirector forwards. This is the best story.
2. **The tool was the problem.** S2 took four runs, with oracle capacities first ranging over 3,739–5,452 req/s. The first explanation was wrong; a direct test found the client's port selection stalling on TIME_WAIT. After the fixes, repeats agreed within 1.2%.
3. **XDP to TC.** The checksum offload failure, then realising none of our laptops has native XDP, turned a bug into a design decision.
4. **Our own claims didn't survive review.** "No sustained oscillation" was wrong, a sweep point was mislabelled, and the "oracle" comparison flatters the controller.

- **Leave out:** battery death, small fixes, tuning-simulator corrections.

Splitting the 15 minutes

Part	Time
Slide 1	1.5 min
Slide 2	2.5 min
Demo (right after slide 2, when the mechanism is fresh)	3.5 min
Slides 3a + 3b	4 min
Slide 4	1.5 min
Slide 5	1.5 min
Buffer	0.5 min

Demo: connection survival

This should be reliable because it counts broken connections rather than latency, so desktop noise barely matters. It needs no network, since everything runs in namespaces.

1. **One request:** `curl` through the VIP, then `lbctl stats` to show the LB forwarding it (about 30 s).
2. **With second hops:** 300 long-lived connections at low rate, move 50% of the buckets, 0 errors.
3. **Without second hops:** same move with `-second none`; expect roughly 130–146 errors, as in the move test.

- **Optional, only if rehearsal is solid:** controller plus antagonist, showing beE1's bucket count falling. Show buckets, not p99.
- **Setup:**
  - Don't apply `tune.sh`; it confines the desktop to two CPUs, which would hurt screen sharing.
  - Bring the testbed up beforehand, on AC power.
  - Starting backends by hand is fiddly, so this needs a small script.
- **Not tested:** I haven't run this exact sequence on the current build, and can't while S6 holds the testbed. I can write and test the script once S6 is done, if you want it.
- **Fallback:**
  - Screen-record a good rehearsal run.
  - If there's no result within about 30 s live, switch to the recording and don't debug on stage.
  - Keep `results/move-20261102-030314/results.json` as the last resort: 0 vs 130 broken.

Likely Q&A: why not 100%, what about scalability, and whether a static table would do just as well if you knew the disturbance. The last one is the hindsight runs, so you'll have a real answer once S6 is in.

Context limit reached. The session continued from an automatic summary.

S6 has finished. I'll check that the machine is clean and the runs are valid, run the analysis, then work through the write-up, the plots, the cluster check and the commits.

I've got the S6 results and I'm fixing the last plot problems (hidden single-run markers, thin markers, colliding labels), then rewriting `results.md`.

I'm finishing the docs (`journal.md`, `campaign.md`, decisions), then I'll run the emulated cluster check, since the `tool_alive` harness change hasn't been tried in cluster mode yet.

All 12 items are finished and committed (six commits, `9199590` to `50f863f`). S6 changed one headline: **a static table that knows the disturbance beats every controller we ran it against.**

The machine is back to normal: `tune.sh` is undone, no namespaces are left and the LB is unloaded. All S6 runs were valid, including the third CPU/total-RIF pair. Firefox was open throughout S6, confined to CPUs 0–1.

What S6 found

Hindsight table vs controllers (2 runs each, S6 against S3/S5 runs from days earlier):

	Hindsight: recovery, windows over SLO	Controllers: recovery, windows over SLO
Step	0 s, 0–4%	15–45 s, 17–50%
Antagonist	0 s, 0–4%	5–10 s, 8–25%

- **New claim:** the controller doesn't beat static tables in general. It beats a table that's out of date, and it doesn't need to know which server is hit, when or by how much.
- **Hindsight's limits:** its table only suits that one disturbance. From its shares it should also lose capacity when nothing is wrong (about 85–90% of the sum), but that's computed, not measured. It wasn't run on

flapping.

The oracle's missing 5%: busy backends explain at most about half.

- **Busy backends:** the sum of capacities falls 2.0%, which moves the oracle from 95.7% to 97.6% of the same session's sum. That's one search per backend, and the fall is no bigger than the difference between two alone searches.
- **Through the LB:** the same shares without the LB reach 99–101%, so most of the gap appears only through the LB. Requests also arrive less evenly there (CV 4.9–8.6% vs 1.6–4.3%). We didn't separate hashing, per-connection choice and packet work.
- **Firmness:** both effects are under the ~5% limit, so they show direction, not size.

CPU vs total RIF, interleaved (3 runs each): no difference we can resolve.

- CPU reached 92.7% and total RIF 90.9%, and their runs overlap.
- **H2 got weaker for total RIF:** it's now 5.7 points below the oracle, just past the limit, so I changed it from "supported" to "on the edge".
- One CPU table spans 87–100%, and one reached 100.1%, above both oracle runs in that session. So the oracle isn't the best split, and 100% isn't a ceiling.

Changes to your slides

Replace the safe wording I gave you with this:

- **Slide 1:** the oracle is now 95–97% (S2 95.1%, S6 96.7%).
- **Slide 3:**
  - Add the hindsight result. It also answers the likely Q&A question about whether a static table would do as well.
  - Use 92.7% vs 90.9% for CPU vs total RIF.
  - Say the gap is "at most half from busy backends, the rest appears only through the LB".
- **Slide 4:** I gave you "3% to 14%" for the limit; it's **3% to 15%**.
- **Plots:** the ones in `results/campaign-20261102/` are now fixed, so use those instead of the scratchpad copies. `s6-tables.png` is new.

Other things to know

- **Still untested:**
  - **Startup fix:** no controller run with long-lived connections has happened since the fix. The "0 broken in 34 runs" predates it, and `results.md` says so. S6's support is a unit test, six table runs with 0 errors (short connections only), and four hindsight runs with no moves.
  - **H3:** its second half is still untested. Connections live about 2.2 s, measured, and the fastest on-phase is 2.5 s, so that point is only just above the threshold.
- **Emulated cluster dry run:** it passed in 5 min 40 s with a clean undo, and healthy runs aren't flagged. Flagging a dead controller was only tried on the single laptop (kill -9); on the cluster it relies on `systemctl is-active`.
- `-39%` **claim:** I dropped it rather than measuring it.
- **Older journal entries:** they keep their old wording as history. The new entry lists the corrections.

Questions for you

- **Do you want the startup-fix rerun?** A few runs of the antagonist scenario, about 15 minutes, would let you say the 0 broken also holds after the fix. You said no rerun was needed for the old data, so I didn't do it.

- **Should I write the demo script?** I'd script the connection-survival sequence from the notes and test it now that the testbed is free. You should still rehearse it and record the fallback yourselves.

Prompt 31

yes to both, the rerun and the demo script. thanks for the slide corrections, we can still edit the ppt.

1. startup-fix rerun: the few antagonist runs with cpu and total rif, so the final code has its own "0 broken" with long lived connections. add them to results.md as what they are
2. demo script: the connection survival sequence from your notes, as one script in scripts/, no tune.sh, brings up what it needs and cleans up after itself, prints the numbers big enough to read on a projector. run it at least twice here and tell me how long it takes and if the numbers came out as expected

also a friend read through the repo on an older copy (from before s6, so ignore anything s6 already covers). most of it is outdated now, but these small things are still true, please fix them in a quick pass:
- cmd/controller -signal help doesn't list rif-total
- internal/lb/lb.go still has two comments saying the table generator / second hops come later
- campaign.md says the hook cost used 3 rates, it was 2 (2000 and 4000), and the "still to build for this" section is long done, delete it
- plan.md phase 1 still says "XDP LB" and it's TC now (D35). check the open items and the phase table are current too
- plan.md's ~1.5% and results.md's ~5% are different measurements (full noise check vs 20 s campaign probes), add a clause so they don't look like a contradiction
- check D29 names bpf_skb_change_tail for TC, not only the xdp call
- readme: add the libbpf headers to the requirements, that the go tools build on linux only (check that's true first), the s5 command, and the helper scripts missing from the layout
- a .gitattributes forcing lf for scripts and source (*.sh *.py *.go *.c *.h *.md, Makefile), one of us is on windows and a crlf shell script would break on the laptops. don't use "* text=auto", 13 sysmon.csv files in results are committed with crlf and i don't want git touching the data files

then at the end run make test and the golden tests (make test-bpf) on the final code, and e2e once, and tell me they pass

Claude Code

I'll do the small fixes first since they don't need the testbed, then run the antagonist session in the background. The demo script comes after that, because it needs the testbed too, and the final test suite runs last on the finished code.

The quick-pass edits and `scripts/demo.py` are written but not committed, because the demo can't be tested until S7 releases the testbed. I'll carry on when S7 finishes, in about 20 minutes.

S7 came back with 0 broken connections in all four runs. Now I'll test the demo script.

I'm fixing a table the last edit broke in `campaign.md`, then running the final tests on the finished code: `make test`, the golden tests, e2e, and the demo twice more.

All done and committed (`a445747` to `d443b1b`). `make test` , `make test-bpf` and e2e all pass on the final code, and the machine is clean: no namespaces, `tune.sh` undone.

Test	Result
`make test`	passes, including `go vet`
`make test-bpf` (golden tests)	passes, no failures or skips (32 PASS lines including subtests)
`bench/e2e_lb.py` , once	ALL PASSED (21 checks)

1. Startup-fix rerun (S7, 16 min)

Four antagonist runs on the final code, in the order CPU, total RIF, total RIF, CPU, all valid:

- **Connections:** 0 of 400 long-lived connections broken and 0 backend resets. All 6,233 bucket moves were swaps, and 4.5 M short-lived requests had 0 errors.
- **Recovery:** 5–10 s with 8–12% of windows over the SLO, in line with S3 and S5. Hindsight is still better at 0–4%, and S7 ran about 20 minutes after S6, so that comparison no longer spans a reboot.
- **How `results.md` describes it:** one scenario, not a repeat of the 34 runs. It also doesn't force the startup race, because the first table is probably published before the load generators connect, so the unit test is what covers that logic.

For your slide you can now say "0 of 400 on the final code", next to the 34 older runs.

2. Demo script

Run it with `sudo python3 scripts/demo.py` ; add `--no-pause` to skip the Enter prompts. It takes **40 s** without pauses, so there's plenty of room to talk over it in the 3.5-minute slot.

I ran it to the end 7 times, and the numbers came out as expected every time:

	Connections broken, of 300
With second hops	**0** in all 7 runs
Without second hops	133, 151, 149, 156, 154, 142, 146

- **Expected range:** say "about 150" on stage rather than the 130–146 from my notes, because half the buckets move and the runs went up to 156.
- **What to point at:** a live table shows each backend's requests per second. After the table change, beP1 stays near 770/s with second hops and drops to 0 without them. The audience will see that before the big numbers appear.
- **Other cases tested:**
  - All 24 CPUs kept busy by other work: same outcome.
  - In a real terminal with pauses and colour: worked.
  - Interrupting it partway with SIGINT: it cleans up. That test found the load generator kept running afterwards, which I fixed.
  - Starting it with a testbed already up: it refuses.
  - It exits 1 and says to use the recording if the numbers are wrong.
- **Rehearse on the presenting laptop:** I've only run it on this one, so check `config.env` and `backends.conf` there, and record your fallback.

3. Your friend's list

All fixed. A few notes:

- **Linux-only claim:** confirmed by cross-compiling; Windows and macOS builds fail on the namespace and CPU-affinity system calls.
- `plan.md`**:** phases 2 and 4 had stale statuses, so I updated them. I also split the open items into what's still open and what's settled.
- `.gitattributes`**:** no existing file was renormalised. There are now 14 CRLF `sysmon.csv` files, the 13 you mentioned plus S7's new one, and git leaves them alone.
- **Beyond your list:**
  - The README also gets the `s2-extra`, `s2-rif-total`, `s7` and demo commands, since RQ1's total-RIF numbers can't be reproduced without the first two.
  - The layout also lists `cluster/`, `bpf/lb.h`, `bpf/xdp_pass.c`, `internal/lb`, `internal/testbed` and `internal/work`.

Question for you

Should `.gitattributes` also cover `*.env`, `*.conf` and `*.example`? I kept to your list, but `config.env` is sourced by the shell scripts, and I'd expect `backends.conf` and `cluster.conf` to break the same way with CRLF. That's the same Windows risk you were guarding against. All three are LF today, so adding them would change nothing now, and I'd add them.

Prompt 32

yes add env, conf and example to the gitattributes too. and one last small add, keep it to about 10 minutes total and don't expand it:

1. readme: a short section on what we borrowed and what's ours. buckets, two hops and load feedback from unimog, the table idea from maglev, the second hop forwarding from beamer, rif as a signal from prequal, each with what we changed. then what's our own: tc on wifi only laptops, one noise based tuning rule across signals, the cpu vs rif comparison on unequal cores. check each claim against references/ and decisions.md rather than trusting my list

2. short packet captures in results/captures/ (small, a few packets each, -c limit): encapsulated packet from lb to backend, a second hop forward after a swap, and a direct reply from backend to client that skips the lb. a few lines saying what to look at in each, so we can put one on a slide

commit when done

Claude Code

I'll add the three patterns to `.gitattributes`, check your borrowed-vs-ours list against `references/` and `decisions.md`, then take the captures.

The captures came out clean, so now I'll write them up along with the README section. Checking your list against the sources changed some of it, and I'll list what changed in my summary.

All three are done and committed (`cb59744` to `8418837`). The testbed is down, `tune.sh` wasn't used, and the 14 CRLF data files are untouched. `.gitattributes` now also covers `*.env`, `*.conf` and `*.example`; all of those files were already LF, so nothing changed in them.

README: what the sources changed in your list

The new section is "What we borrowed and what's ours".

Your claim	What the sources say	What the README says
Table idea from Maglev	Only partly true. Maglev has a large lookup table, but filled by permutation and paired with connection tracking, and we use neither. `decisions.md` only cites Maglev for a metric. Our table follows Unimog.	The general idea only, and it says what we didn't take
Second hop from Beamer	True, but it reached us through Unimog, whose post credits Beamer. We dropped Beamer's generation numbers and longer chains.	"Beamer, through Unimog"
Buckets, two hops, load feedback from Unimog	Confirmed. Unimog's own signal is processor utilisation.	As you listed, with our changes
RIF from Prequal	Confirmed, but Prequal probes RIF and latency per request.	RIF alone, read every 1–2 s to move buckets; dividing by cores was our change, and it lost
TC on wifi-only laptops as ours	Choosing TC for the LB is our decision, but all the numbers are on veth on one laptop, and wifi has only run on emulated machines.	Says exactly that
Noise-based tuning rule, CPU vs RIF comparison	Both ours.	Adds that live churn was higher than the simulator predicted, and that it isn't a test of Prequal

Two things your list was missing, both borrowed:

- **The swap trick:** this is Unimog's, and its post calls it a "novel trick". Since nearly all our moves are swaps, leaving it out would have made it look like ours.
- **Rendezvous second hops:** these come from GLB.

"Ours" is worded as our work in this project, not a claim that nobody did it first. I couldn't check the SYN-cookie check against the sources, because the Unimog post doesn't mention one. So the README just lists it as our change to the keep rule and cites D8, without saying where the idea came from.

Captures

They're in `results/captures/`, 20 KB in total, with a `README.md` saying what to look at in each and a small `decode.py`, because tcpdump shows our header only as a UDP payload.

1. `1-encapsulated.pcap` **(on the LB):** the same SYN going in plain and coming out wrapped, with the second hop visible in the header.
2. `2-second-hop-forward.pcap` **(on beP1):** this is the one I'd put on a slide, since it shows the design in two lines. Only the outer addresses and the HOPPED flag change:

   ```
   outer 10.1.0.10 > 10.1.0.11 UDP 7700 | flags 0x00 bucket 879 second_hop 10.1.0.12 | inner
   10.0.0.2:40568 > 10.2.0.1:8080
   outer 10.1.0.11 > 10.1.0.12 UDP 7700 | flags 0x01 (HOPPED) bucket 879 second_hop 10.1.0.12 |
   inner 10.0.0.2:40568 > 10.2.0.1:8080
   ```

   It's a true swap: all 2,047 changed buckets exchanged their two hops exactly.
3. `3-direct-reply.pcap` **(on beP2):** replies from the VIP straight to the client, including port 40568, the same connection as in capture 2. While this was captured, the LB saw 312 packets and none had the VIP as source. That count came from a separate 3-second capture that I didn't save, and the notes say so.

The capture commands are in the notes, but I didn't commit the script that ran them.

Prompt 33

one more small one, about 5-10 min, then that's it for this session. while reading through the code i noticed a few things:

1. TestNeverChangesABucketWithinDwell passes even if you remove the dwell check from plan, only the NewFrom test catches it. D32 cites this test as support for the safety rule, so it should actually fail when the rule is gone. fix the test (make a scenario where the controller really wants to change a settled bucket again) and confirm by removing the check temporarily that it fails, then put the check back
2. stale lines in docs/campaign.md: search range [0.8, 1.2] doesn't match the code (s1 0.5-1.0 x theoretical, s2 0.7-1.05, s6 0.85-1.15), the tuning step's <= 40 moves/min was revised in D36, and connection lifetime "~0.5-2 s" is about 2.2 s measured
3. D10 and plan.md's rq3 mention a "connections" signal, there's none in internal/signals. mark it as dropped
4. from reading the code, with whole core cpu on one machine a dead backend's cores look idle, so the controller would give it more buckets. we didn't test failures, so add one line next to where results.md says what wasn't tested, saying this comes from reading the code and not from a run

run make test after and commit

Claude Code

All four are done and committed (`246f502`, `3a08bdc`), and `make test` passes.

1. The dwell test. You were right about why it passed: `plan` takes the oldest buckets first, and with 4096 buckets it never reaches a recently changed one, so the dwell check never mattered.

- **New scenario:** one backend starts with only 32 buckets and the load keeps reversing, so soon only recently changed buckets are left to move. The test also fails if the controller never wanted to move a bucket and was held back, so it can't pass without exercising the rule.
- **Confirmed by removing the check:** it now fails with `t=6: bucket 32 changed again 6 s after its last change (dwell 20)`. `TestNewFromKeepsConnectionsOfTheActiveTable` fails too. I put the check back, and `controller.go` matches the committed version.
- **Old scenario:** it still passes without the check, as you found. I kept it as `TestMovesKeepTheOldFirstHop`, because it still checks that a move never drops the old first hop. D32 now cites the new test and says what it does.

2. `campaign.md`**.**

- **Search ranges:** now listed per session, taken from the code. S6's denominators use 0.8–1.2, so I listed those too.
- **Resolution:** the same line claimed 1.25%, so it now says about 1–2%, and a later row says about 1%. D37 still says 1.25%, which I left as what was decided then.
- **Churn rule:** now D36's revised rule.
- **Connection lifetime:** about 2.2 s, as measured.

3. Connections signal. Marked as dropped in D10's table, D4's RQ3 and `plan.md`'s RQ3. D10 says it was never built and doesn't give a reason, since I don't know one.

4. `results.md`**.** One new bullet in the "applies to every claim" list, next to the line about what wasn't run, saying this comes from reading the code and not from a run. I checked the code: on one machine, whole-core CPU

reads `/proc/stat`, so a dead backend reads as idle and the update rule gives it more buckets.

I also added a short journal entry about the test, which you didn't ask for.

Prompt 34

> sorry, one more and it's a quick one, i was reading bench/cluster.py for the laptop run and two things look like they'd bite on real machines but not in emulation, since emulation runs under sudo:
> 1. remote_dirs does mkdir over ssh with sudo, on the client too, and the driver runs from the client as the normal user. so results/.../s1 would end up owned by root and the driver can't write runs.jsonl there. check if that's right and fix it, and make the emulated dry run able to catch this kind of thing if that's easy
> 2. setup("up") is outside the try at the start of main, so if it fails on the third machine the first two stay set up and the next attempt stops with "already set up". move it inside so the undo runs
>
> test what you can with the emulated cluster, say what's still only checkable on real machines, and commit. that's it

Claude Code

Confirmed, and the emulated dry run now catches it: `PermissionError` on `runs.jsonl`, with `s1` owned by root. Next I'll reproduce #2 on the old code, then apply both fixes.

Both fixes work, and the full dry run passes as a normal user. I'll tear down the emulation, update the runbook and commit.

You were right on both, and both are fixed and committed (`6d660b3`, `35a0c45`). A full emulated dry run now passes as a normal user in 6 min 15 s, with no root-owned files left and no invalid runs. `make test` still passes, and the machine is clean.

1. Root-owned results directory

- **Reproduced:** with the driver as a normal user, the run stopped with `PermissionError` on `runs.jsonl`, and `s1` was owned by root.
- **Fix:** a new `results_owned` makes the directory on the LB and client machines and hands the whole results folder to the user who ran sudo there. It runs before each session, before `collect`, and at the end, including after a failure. That last step also takes care of the root-owned files the tools write.
- **Catching this in emulation:** it was easy. The emulated transport now uses `sudo` per command like the real one, and the driver refuses to run as root, so the command no longer has `sudo` in front:

  ```
  python3 bench/cluster.py cluster/emulated.conf --no-tune --dry
  ```

  Any other step that only works because the driver is root would now fail the dry run too. This one passed, so there were no others on the paths the dry run covers.

2. `setup("up")` outside the `try`

I forced a failure on the third machine by putting a namespace called `beP1` in the way:

	After the failure	Next attempt
Before	state files left for client, lb and h1	stopped with "already set up"
After	no state files left, all machines undone	no leftover state to stop it

I didn't run a second attempt after the fix, because with no state files left nothing can trigger that message.

Only checkable on real machines

- **ssh:** `ssh … sudo`, connection sharing, and the quoting of the new `sh -c` command.
- `rsync` **in** `collect`**:** emulation skips it because everything shares one file system.
- **Ownership on a separate LB machine:** `SUDO_UID` has to be the ssh user there, and `collect` has to be able to read the tools' files. A site's sudoers settings could affect this, so look at the owner of `results/cluster-*` on the LB after the `--dry` run.
- **A machine that becomes unreachable partway:** `setup("down")` can't undo it and doesn't report the failure, so you'd use the manual steps in the runbook. I noted this but didn't change it.
- **Already on the runbook's list:** wifi and `tune.sh` on AMD laptops.

Prompt 35

50 minutes on the real laptops is too long for us, can you add a --quick mode to bench/cluster.py that takes about 30, leaving the current full run as the default. from the code i think the time goes s1 ~11 min, s2 ~16, s3 ~16, so:
- 4 probes per search instead of 5, the wifi noise will swamp the extra resolution anyway
- in s2 only oracle and equal for the static tables, keep both controller tables (cpu vs total rif is the main question)
- in s3 keep all four runs (oracle, cpu, total rif, redirector contrast), but make the antagonist scenario shorter: on at 30 s, off at 120 s, end at 150 s. only in quick mode, the single laptop scenario stays as it was

check that the analysis still works with those (the 30 s before window, the late p99 window, the shorter run, no cores row) and doesn't state anything that's wrong for the shorter scenario. run the emulated dry run with --quick as a normal user to test the plumbing, give me your real time estimate from the step lengths, say in the runbook what quick gives up (single runs, coarser search, no by cores comparison), and commit. keep it small

Claude Code

I'll start by reading how the search, the scenario timings and the analysis windows use these values, since a shorter scenario could quietly break the late-p99 window.

`--quick` is in and committed (`468faed`, `046c1ea`, `3602508`). The default full run and the single-laptop scenario are unchanged. The emulated `--quick --dry` run passed as a normal user in 5 min, with no root-owned files left and everything cleaned up.

Time estimate: about 33 minutes, not 30

This comes from the step lengths, with a probe taking about 27 s (S6's timestamps show 27.3 s). It hasn't been run at real length.

Step	Full	Quick
S1	11 min	9 min
S2	17 min	10 min
S3	16 min	10 min
Tuning, setup, analysis	~4 min	~4 min
Total	**~48–50 min**	**~33 min**

Your breakdown matched the code. Ssh, a busy network and searches that end at the edge of their range (one extra probe each) will add some time, so plan for 35.

I took "only oracle and equal" to mean one oracle search, not the full run's two. If you want the second one back, it adds about 2 min and gives you a repeat to compare against.

Analysis: what your check found

The windows you asked about needed fixes, and so did some text that was wrong:

- **30 s before window:** with onset at 30 s, "p99 before" was empty and the signal baseline would have included start-up. Both now use the second half of the lead-in (15–30 s here), which is the same 30–60 s window for 60-s scenarios.
- **Late p99 window:** it works, at 90–120 s, but rests on 30 s of data instead of 120 s.
- **Shorter run:** the single antagonist run was being presented as a "sweep", described as 120 s on. That section now only appears when flapping runs exist.
- **No cores row:** the capacity plot had an empty row for each missing policy; only policies with data now get a row. The axis also hid a 49% result, so it now follows the data.
- **Wrong in every cluster run, including the full one:**
  - "CV pooled over 0 pairs: nan%" now says this run has no CV.
  - The non-monotonic p99 claim was hard-coded and now appears only if the data shows it.
  - "n/a (0% errors)" now shows the real rate, such as 0.11%.

`--dry` shortens the timing, so it can't test 30/120/150. I checked those windows with synthetic records instead, and they used exactly the expected windows.

One side effect on your existing results: S5's flapping rows and the stream-sibling row in `summary.md` had `nan` for "p99 before" all along and now have values. Every other number is unchanged, and `results.md` doesn't use those cells.

What quick gives up (now in the runbook)

- **Single runs:** one oracle search, and every table and config runs once, so nothing in the run says how much of a difference is noise.
- **Coarser search:** resolution about 2% in S2 and 4% in S1.
- **No by-cores table:** H1 can't be checked, but oracle vs equal remains.
- **Shorter antagonist:** a recovery slower than 75 s reads as "never". The numbers aren't directly comparable with a full run or the single laptop, so don't put them in one table without saying so.

Prompt 36

first real problem from the laptops. a friend's laptop on ubuntu (clang 21.1.8, kernel 7.0.0-42-generic, libbpf-dev 1.6.3) builds fine, and make test passes, but make test-bpf fails on all 8 director tests on both hooks, redirector tests pass:

director_test.go:207: file ../../bin/bpf/director_tc.o: load BTF maps: map tables: can't parse BTF map definition of inner map: can't get size of BTF value: type *btf.Fwd: type is unsized

so clang 21 emits the inner map's value type as a forward declaration, and clang 23 here emits it in full, so it never showed up. i think the fix is to stop depending on that, like key_size/value_size in the inner map definition instead of __type(value, ...) (cilium/ebpf reads those), but check it, and check every other map with a struct value for

the same risk since this error may just be the first one hit. keep it small, the laptops are waiting on it.

test here with make test-bpf and e2e. since we don't have clang 21 here, if you can get it quickly (a container, or a package) build with it and run the golden tests, otherwise check the built objects so no map definition depends on a forward declared type, and say clearly which of those you did. commit, and tell me if the lbctl load path needs anything else. no need to rerun any campaign, but note in the journal and results that the map definition changed and why that doesn't affect the numbers, if that's true

Claude Code

I'll try to get a real clang 21 first, using an Ubuntu container matching your friend's setup. A real build is much stronger evidence than inspecting objects from clang 23.

No response requested.

Prompt 37

sorry, the laptop rebooted in the middle of that (an omarchy update), so your fix in bpf/director.h is uncommitted and anything running got killed. please finish it. the friend's ubuntu laptop is the real clang 21 test and everyone is waiting, so skip getting clang 21 here unless you already had it working. check the other struct value maps are fine (backends in director.h) and the built objects have no map definition relying on a forward declared type, run make test-bpf and e2e, check nothing is left over from the killed run, commit, and keep the journal note short

Claude Code

The reboot may have cleared the scratchpad and changed the kernel, so I'll check what survived first: the fix itself, any leftovers, and whether sudo works.

The fix is committed (`ac0ac64` , `f6cbf84`) but **not pushed**, so your friend won't get it until you run `git push` . `make test` , `make test-bpf` and e2e all pass here on clang 23.

Your friend can then test it with `git pull && make && make test-bpf` . The header change triggers the object rebuild.

Check	Result
`make test`	passes
`make test-bpf`	exit 0, 32 PASS lines, 0 failures, 0 skips, both hooks
`bench/e2e_lb.py`	ALL PASSED, 20 checks (I told you 21 last time, but my count included the "ALL PASSED" line)

What I checked

- `backends` **:** it's the only other struct-valued map, and it's fine. It's a top-level map and stayed a full 12-byte struct on both compilers; only the nested map hit the problem.
- **Forward declarations:** I walked the BTF from every map definition in all five objects, and no map relies on a forward-declared type. `bpf_flow_keys` and `bpf_sock` do appear as forward declarations, but no map reaches them.
- **Clang 21:** this is two different things.
  - **Before the reboot:** I had a real clang 21.1.8 (Arch's build) working. It reproduced your friend's exact error on the old source, and the fixed source parsed correctly with the right sizes. The objects only went

through the loader's parser, not into the kernel, and the walk above didn't run on them.
    - **Now:** that copy was wiped with the scratchpad and I didn't re-fetch it, so today's walk and tests are on clang 23 only.
- **Leftovers from the killed run:** none. No namespaces, units or BPF pins, and `tune.sh` isn't applied. Nothing was left over after the tests either.

Does `lbctl load` need anything else?

No code changes that I can see. It goes through the same parser, and the tables it publishes use `innerTableSpec` in `lb.go`, whose sizes still match the map. E2e exercises that path here.

The one thing I can't rule out is the kernel verifier rejecting clang 21's code on kernel 7.0, because nothing has loaded clang 21 objects into a kernel yet. Your friend's redirector tests already pass there, which is encouraging. If `make test-bpf` fails again, send me the new error.

The numbers aren't affected

Built with the same compiler, old and new source give identical instructions, relocations and map sizes in all five objects; only the debug type information differs. `results.md` and the journal both say so, and no rerun is needed.

My byte comparison from before the reboot was invalid, because it compared against objects built earlier and even showed `redirector` as changed. This result comes from redoing it properly. The results note covers the source change only, since I no longer have the exact objects the campaign used.

Context limit reached. The session continued from an automatic summary.

Prompt 38

the real 4 laptop run worked. results are in /home/student/mini-unimog/results/cluster-20261106-085033 on this laptop (use sudo to read, student owns them), the first dry run that crashed is cluster-20261106-083954, the clean dry is cluster-20261106-084421, and the cluster.conf used is in /home/student/mini-unimog/cluster/cluster.conf. client was this i7, lb .8 (thin i5), h1 .185 (i5, arch), h2 .243 (i5, ubuntu), phone hotspot, --quick, 1 run per config.

what we changed on the way: first dry run the tuning crashed with nan noise because sigmon samples were 2.6-8.8 s apart, the hotspot looked saturated at ~250 req/s and capacity through the lb was well under the sum. so i doubled the cost and slo in cluster.conf (COST_MS=40, SLO_P99_MS=800, CONNS=32). ping lb to backends was ~140 ms avg, up to 362 ms, at idle before the run. tune.sh warned the power profile wasn't performance on three laptops (power-saver, unknown, balanced) and that was left alone.

please, in about 10 minutes and no new runs:
1. copy the three result folders into the repo under results/ (and the cluster.conf used, keep it next to the real run), mark the two dry ones as dry
2. add a real laptops section to results.md and a short one to the readme, stating what it does and doesn't show. things i see: forwarding on 0 long lived broken vs 12 with it off, cpu controller late p99 2.9 s and 1 error vs 12.9 s and 281 errors for the oracle table, total rif 10 s and 249 errors, nothing got back under 800 ms during the antagonist, oracle 91.8% cpu 87.8% and equal and total rif both >=80% with search resolution 15-58%, tuning gave rif a 8 s period with a huge band since rif noise is 0.6-0.7 on wifi.

check those against the data, and check the send delay warnings where a probe met the slo, plus whether the reachable bottleneck looks like wifi rather than the backends. say plainly that hardware was close to identical, one run each, and not comparable in one table with the single laptop numbers. the h2 backends reached only 69% of theoretical while h1's reached 91-97%, look at why if the data says, otherwise say you don't know
3. update the runbook with what we learned (COST_MS 40 as the starting point on a phone hotspot, paste lines one at a time around sudo -iu, ufw and ssh, hostname missing on arch is harmless, set the power profile first), update cluster.conf.example to match, and the summary's "work units = 1 ms" line prints the per request work when COST_MS isn't 1, fix that label
4. plan.md status and journal entry, short, then commit. then tell me the numbers you're confident in for the report

Claude Code

I'll copy the three result folders in first. Then I'll check each number you listed against the raw data before writing anything, and do the docs, the label fix and the commit after that.

All four items are done and committed in four small commits, not pushed. Rerunning the analysis on the real run changed only the calibration label; the clean dry's title also picked up its new folder name.

Numbers for the report

Confident (the mechanism or several readings back them, though each is still one run):

Claim	Numbers
Long-lived connections broken, forwarding on vs off	**0 of 20** (1223 and 511 moves) vs **12 of 20** (1409 moves)
Backend RSTs	0 and 1 vs 34
CPU controller finds the table by itself	733 moves from equal, shares within 1.7 points of the oracle's
h2's CPU speed compared with h1's, clocks fixed	about 0.72 (83–89k vs 113–123k work units per busy core-ms, across all five runs)
RIF tuning	8 s period, band 1.82/0.91, **0 moves in S2**
Run length with `--quick`	about 32 min

Usable only as "in this one run":

- **Late p99:** CPU 2.9 s, no forwarding 5.4 s, total RIF 10.0 s, oracle 12.9 s. The direction is believable because CPU cut beE1 from 968 to 613 buckets and its p99 was falling. The sizes are not solid, because most of that time is waiting for one of the 32 client connections: the oracle's p99 is 13.3 s, of which 12.8 s is send delay. Long-lived connections on their own pool stayed at 1.2–1.4 s in all four runs.
- **Capacity:** oracle 91.8%, CPU 87.8%, equal and total RIF between 80% and 102%.

Don't claim:

- that the oracle beats equal or CPU
- anything about RIF as a signal from S2
- that wifi is the limit at 40 ms
- any comparison with the single-laptop numbers

What I corrected from your list

The rest of your numbers match the data.

- **Errors:** 281, 249 and 219 are mostly requests never sent, still queued when the run ended (281, 248 and 200 of them). Real failures once sent were oracle 0, CPU 1, total RIF 1, no forwarding 19.
- **Resolution "15–58%":** the real spacing is 2.4% for oracle and CPU, and 27.7% for equal and total RIF. The 15% is a median of those two groups, and 58% is twice beP2's standalone spacing.
- **Equal and total RIF are the same table:** RIF never moved in S2, so this is one table measured twice. At 149 req/s it gave p99 2.4 s with no errors, then 7.6 s with 26% errors. That noise is bigger than the gap between oracle and CPU.
- **"Nothing got back under 800 ms":** true as no recovery. The oracle had 2 windows and no forwarding had 1 under 800 ms, right after onset. CPU had none, but was down to 1.5 s when the antagonist ended and under 800 ms by 130–135 s.
- **RIF noise "on wifi":** not shown. CPU noise also rose about 4×, and RIF averages only 1–3 per backend, which is noisy anyway.
- **Sigmon gaps of 2.6–8.8 s:** I can't find these in the files. The first dry's CSV shows 1.5–2.8 s and the real run about 1 s, up to 2.6 s. The `nan` came from the dry run's short window leaving only 4 samples, so a full run wouldn't hit it.

The questions you asked me to check

- **Send delay warnings:** all 13 passing S2 probes are flagged, but none passed wrongly, because latency counts from the scheduled send time. Nine are 78 ms or less. The other four, near capacity, are 187–407 ms because the client's connection pool was filling, so oracle and CPU capacity may be slightly understated.
- **Wifi or backends:** at 40 ms it looks like the backends.
  - A rough queueing model with no network at all predicts p99 of about 860 ms at 134 req/s (measured 695).
  - The same hotspot carried 147 req/s at p99 332 ms in the clean dry run, though that was a short probe with turbo on.
  - Wifi shows up as noise instead. At 20 ms it may be the limit, but I couldn't rule out the LB's CPU or the 64-connection pool.
- **h2 at 69%:** this is CPU speed. Calibration only ran on h1, so "theoretical" assumed h1's speed everywhere; at h2's own speed its backends reached 90–94%, like h1's. The gap only appeared with `tune.sh` applied. I don't know why the fixed clocks differ, but `lscpu` and `scripts/tune.sh status` on h1 and h2 would likely tell you without a new run.

"Hardware close to identical" is in the docs as you asked, along with this speed gap.

Files

- **Dry runs:** they are now `results/cluster-dry-20261106-083954` and `results/cluster-dry-20261106-084421`, each with a `DRY.md`.
- **Added beyond your list:**
  - Each run folder has its `run.log`, because the power-profile warnings and the crash trace exist only there.
  - D39 in `decisions.md` now has the new workload values.
- `cluster.conf`**:** the copy next to the real run is verbatim, so its comment still says 20 ms while its setting is 40. The example config is corrected.